

SENTRA GATEWAY

Enterprise API Security and Observability Gateway

Professional Technical Project Documentation

Architecture | Security Design | Implementation Blueprint | Evaluation Plan

Prepared by
Omar Ahmed

Version 1.0 | June 2026

Repository: sentra-gateway
Package: com.omar.sentra.gateway
Document Status: Final professional project documentation package

Document Control

Field	Value
Project Name	Sentra Gateway
Subtitle	Enterprise API Security and Observability Gateway
Author	Omar Ahmed
Document Type	Professional technical documentation, implementation blueprint, and evaluation plan
Primary Stack	Java 25, Spring Boot 4, Spring Cloud Gateway, PostgreSQL, Redis, Resilience4j, Micrometer, Prometheus, Grafana, Docker Compose, k6
Scope Note	This documentation package defines the system architecture, security model, implementation plan, evaluation strategy, operational model, and release-ready documentation structure for Sentra Gateway.

Project Scope and Ownership

This document presents the design and planned implementation of Sentra Gateway, an enterprise API security and observability gateway created as a professional backend engineering project. It is intended to serve as a comprehensive project report, implementation blueprint, portfolio reference, and long-term technical guide. Any external libraries, specifications, frameworks, or protocols used in the final implementation should be documented clearly in the references section and README.

Implementation sections define the expected behavior, design decisions, configuration model, validation strategy, benchmark targets, and verification evidence required for a production-style build.

Engineering Context

Sentra Gateway is inspired by real backend infrastructure concerns: security enforcement, request governance, observability, resilience, testing discipline, and operational readiness. This document is written for a portfolio-grade backend infrastructure project, but the design tries to follow professional engineering habits rather than a small demo approach.

This section records the engineering context behind the project and can be extended with contributor notes, review notes, implementation decisions, and release history as the project evolves.

Executive Summary

Modern backend applications are increasingly built as microservice-based systems. This architecture improves modularity and scalability, but it also creates security and operational challenges. When every service independently implements authentication, authorization, rate limiting, request validation, logging, resilience policies, and monitoring, the result can be duplicated code, inconsistent security behavior, weak visibility, and higher maintenance cost.

Sentra Gateway is proposed as an enterprise-grade API security and observability gateway that acts as the centralized entry point for client traffic. It validates JSON Web Tokens, supports API key management, applies route-level authorization, enforces Redis-backed rate limits, validates HMAC-style request signatures, manages IP allow and block policies, detects suspicious request patterns, records auditable security events, exposes metrics through Micrometer and Prometheus, and applies reliability controls such as retries, timeouts, circuit breakers, and fallback responses.

The project is designed using Java, Spring Boot, Spring Cloud Gateway, PostgreSQL, Redis, Docker Compose, OpenAPI, Prometheus, Grafana, and k6. The system emphasizes separation of concerns through modules for routing, authentication, API clients, rate limiting, request signing, IP filtering, risk scoring, audit logging, observability, resilience, and administration.

This project documentation presents the problem analysis, requirements, architecture, data model, security design, implementation strategy, testing plan, deployment approach, and future work for Sentra Gateway. The intended outcome is a production-style backend infrastructure project that demonstrates how centralized gateway controls can improve consistency, maintainability, observability, and security in microservice environments.

Keywords: API Gateway, Spring Boot, Spring Cloud Gateway, Microservices, JWT, OAuth2, OIDC, API Key Management, Rate Limiting, Redis, PostgreSQL, Request Signing, Audit Logging, Observability, Prometheus, Grafana, Resilience4j, k6.

Table of Contents

This document is organized as a complete engineering reference for Sentra Gateway. The major sections cover business problem analysis, requirements, architecture, security, data design, API governance, implementation strategy, testing, operations, and future roadmap.

Document Control	1
Project Scope and Ownership	2
Engineering Context	3
Executive Summary	4
Table of Contents	5
Lists and Terminology	6
List of Figures	7
List of Tables	7
Abbreviations and Terminology	7
Chapter 1: Introduction	8
1.1 Overview	9
1.2 Motivation	9
1.3 Problem Statement	9
1.4 Aim and Objectives	10
1.5 Scope	11
1.6 Document Organization	11
Chapter 2: Background and Technology Review	12
2.1 Microservices and Cross-Cutting Concerns	13
2.2 API Gateway Pattern	13
2.3 Spring Cloud Gateway	13
2.4 JSON Web Tokens	14
2.5 API Keys	14
2.6 OAuth2 and OIDC	15
2.7 Redis and Fast Security State	15
2.8 PostgreSQL and Policy Persistence	16
2.9 Observability	16
2.10 Resilience Engineering	17
2.11 Technology Stack Summary	17
Chapter 3: Problem Analysis and Requirements	18
3.1 Existing Problem Context	19
3.2 Stakeholders	19
3.3 Functional Requirements	19
3.4 Non-Functional Requirements	20
3.5 Constraints and Assumptions	21
3.6 Success Criteria	21
3.7 Requirements Traceability	22
Chapter 4: System Analysis	22
4.1 System Context	23

4.2 Actors	23
4.3 Use Cases	24
4.4 Request Lifecycle	24
4.5 Without vs With Sentra Gateway	25
4.6 Sequence: JWT User Request	25
4.7 Sequence: API Key Partner Request	26
Chapter 5: Architecture and System Design	26
5.1 Architectural Style	27
5.2 Module Decomposition	27
5.3 Layered Design	28
5.4 Filter Ordering	28
5.5 Route Model	29
5.6 Trusted Header Propagation	29
5.7 Docker Compose Topology	30
5.8 Architecture Quality Attributes	30
Chapter 6: Security Design	31
6.1 Security Goals	32
6.2 JWT Validation	32
6.3 API Key Security	32
6.4 Request Signing	33
6.5 Rate Limiting	33
6.6 IP Filtering	34
6.7 Bot Detection and Risk Scoring	34
6.8 Secure Error Handling	35
6.9 Secret Management	35
6.10 Threat Model Summary	36
Chapter 7: Database and Data Model Design	36
7.1 Persistence Strategy	37
7.2 Database Table Catalog	37
7.3 gateway_routes Table	37
7.4 api_clients and api_keys Tables	38
7.5 rate_limit_policies Table	38
7.6 audit_events Table	39
7.7 Flyway Migration Strategy	39
7.8 Indexing Strategy	40
7.9 Retention Strategy	40
Chapter 8: API Design	41
8.1 API Design Principles	42
8.2 Route Categories	42
8.3 Route Management APIs	42
8.4 API Client APIs	43
8.5 Security Policy APIs	43

8.6 Audit APIs	44
8.7 Metrics APIs.....	44
8.8 Error Response Contract.....	45
8.9 API Versioning	45
8.10 Endpoint Catalog	46
Chapter 9: Implementation Strategy.....	46
9.1 Implementation Phases	47
9.2 Configuration Profiles	47
9.3 Filter Implementation Pattern	48
9.4 Service Components	48
9.5 Repository Design	49
9.6 Redis Key Design	49
9.7 DTO and Validation Design	50
9.8 Development Workflow.....	50
9.9 Implementation Risks	51
Chapter 10: Observability and Reliability	51
10.1 Observability Objectives	52
10.2 Metrics Design	52
10.3 Prometheus and Grafana Pipeline	52
10.4 Structured Logging	53
10.5 Audit Event Taxonomy	53
10.6 Reliability Patterns	54
10.7 Alerting Strategy	54
10.8 Operational Trade-offs.....	55
10.9 Metrics Catalog.....	55
Chapter 11: Testing and Evaluation Plan.....	56
11.1 Testing Philosophy	57
11.2 Unit Testing.....	57
11.3 Integration Testing.....	57
11.4 Security Regression Testing	58
11.5 Contract Testing	58
11.6 Performance Testing with k6.....	59
11.7 Resilience Testing	59
11.8 Evaluation Reporting	60
11.9 Security Test Matrix.....	60
Chapter 12: Deployment and Operations.....	61
12.1 Local Deployment.....	62
12.2 Production-Like Deployment	62
12.3 Environment Variables	62
12.4 Operational Runbooks.....	63
12.5 Backup and Recovery	63
12.6 Release Governance.....	64

12.7 Production Hardening.....	64
12.8 Environment Variable Catalog.....	65
Chapter 13: Project Management, Roadmap, and Future Work.....	65
13.1 Work Breakdown Structure	66
13.2 Suggested Timeline	66
13.3 Risk Management.....	66
13.4 Minimum Viable Demonstration	67
13.5 Future Work	67
13.6 Future Work Roadmap	68
Chapter 14: Conclusion	68
Appendix A: Extended Requirements Traceability Matrix	69
Appendix B: Detailed Endpoint Catalog	71
Appendix C: SQL Migration Examples	72
Appendix D: Security Checklist	73
Appendix E: k6 Performance Test Plan	74
Appendix F: README	75
Appendix G: Glossary.....	76
Appendix H: References and Standards	77
Appendix I: Extended Engineering Notes.....	78
1.1 Route Governance	79
1.2 Security Event Taxonomy.....	79
1.3 Safe Defaults	80
1.4 Policy Caching	80
1.5 Downstream Trust Boundary.....	81
1.6 Performance Boundaries	81
1.7 Incident Review	82
1.8 Portfolio Evaluation.....	83
1.9 Operational Maturity	83
1.10 Project Portfolio Value	84
1.11 Security Review Meetings	84
1.12 Release Discipline	85
1.13 Documentation Hygiene	85
1.14 Dashboard Storytelling	86

Lists and Terminology

List of Figures

- Figure 1. High-level Sentra Gateway architecture
- Figure 2. Gateway request processing pipeline
- Figure 3. Centralized security control comparison
- Figure 4. API key validation flow
- Figure 5. JWT validation and trusted header propagation
- Figure 6. Request signing and replay protection flow
- Figure 7. Token bucket rate limiting lifecycle
- Figure 8. IP filtering decision flow
- Figure 9. Risk scoring lifecycle
- Figure 10. Circuit breaker state transition model
- Figure 11. Observability pipeline
- Figure 12. Docker Compose deployment topology
- Figure 13. Logical database relationship model
- Figure 14. CI/CD and release governance pipeline

List of Tables

- Table 1. Core project objectives
- Table 2. Stakeholder roles
- Table 3. Functional requirements
- Table 4. Non-functional requirements
- Table 5. Threat model summary
- Table 6. Route policy matrix
- Table 7. API key data model
- Table 8. Audit event taxonomy
- Table 9. Rate limiting policy examples
- Table 10. Database table catalog
- Table 11. Admin API catalog
- Table 12. Security test matrix
- Table 13. Performance test scenarios
- Table 14. Environment variable catalog
- Table 15. Future work roadmap

Abbreviations and Terminology

Term	Meaning
API	Application Programming Interface
API Gateway	Centralized entry point that routes and controls API requests before backend services.
JWT	JSON Web Token used to carry signed authentication claims.
OAuth2	Authorization framework commonly used for delegated access.

Term	Meaning
OIDC	OpenID Connect, an identity layer built on top of OAuth2.
JWKS	JSON Web Key Set used to verify signed tokens.
HMAC	Hash-based Message Authentication Code used for request signing.
CIDR	Classless Inter-Domain Routing used to represent IP ranges.
RBAC	Role-Based Access Control.
TTL	Time To Live.
SLO	Service Level Objective.
MTTR	Mean Time To Recovery.

Chapter 1: Introduction

1.1 Overview

Backend systems increasingly use microservices to separate user, order, payment, notification, reporting, and administration capabilities. In Sentra Gateway, this topic is important because the gateway is the first component that receives untrusted traffic. A weak design at this layer can expose every internal service, while a strong design can centralize control and reduce repeated security code across the system.

The design decision for this section is to make behavior explicit, measurable, and testable. Every important action should either produce a route decision, an audit event, a metric, or a safe error response. This gives developers and evaluators evidence that the system is behaving according to the documented requirements.

A production-style gateway must balance strict security with usability. If rules are too loose, the system becomes unsafe. If rules are too strict or poorly documented, legitimate clients will fail unexpectedly. Sentra Gateway therefore uses route-level policies, subject-specific limits, and documented response codes.

The module should be implemented through small services rather than one large filter. The filter extracts request data and delegates to services; the services perform validation, policy checks, logging, and decision making. This structure allows unit testing and reduces the chance of hidden coupling.

The design favors explicit policy definitions over hidden behavior. A gateway that silently permits or denies traffic without clear policy ownership becomes difficult to operate. For that reason, Sentra Gateway models routes, permissions, API keys, rate limits, IP rules, and audit event types as first-class concepts.

Observability is treated as part of the architecture instead of an afterthought. A security gateway must explain its decisions, reveal latency patterns, expose upstream failures, and support investigation during incidents. Metrics and logs are therefore aligned with request identifiers and route identifiers.

- Define the policy in a persistent or external configuration model.
- Expose the decision through audit events and metrics.
- Add positive and negative tests for the behavior.
- Document the expected error response and operational impact.

1.2 Motivation

The motivation behind Sentra Gateway comes from the repeated engineering problem of duplicated cross-cutting concerns. In Sentra Gateway, this topic is important because the gateway is the first component that receives untrusted traffic. A weak design at this layer can expose every internal service, while a strong design can centralize control and reduce repeated security code across the system.

The design decision for this section is to make behavior explicit, measurable, and testable. Every important action should either produce a route decision, an audit event, a metric, or a safe error response. This gives developers and evaluators evidence that the system is behaving according to the documented requirements.

A production-style gateway must balance strict security with usability. If rules are too loose, the system becomes unsafe. If rules are too strict or poorly documented, legitimate clients will fail unexpectedly. Sentra Gateway therefore uses route-level policies, subject-specific limits, and documented response codes.

The module should be implemented through small services rather than one large filter. The filter extracts request data and delegates to services; the services perform validation, policy checks, logging, and decision making. This structure allows unit testing and reduces the chance of hidden coupling.

The design favors explicit policy definitions over hidden behavior. A gateway that silently permits or denies traffic without clear policy ownership becomes difficult to operate. For that reason, Sentra Gateway models routes, permissions, API keys, rate limits, IP rules, and audit event types as first-class concepts.

Observability is treated as part of the architecture instead of an afterthought. A security gateway must explain its decisions, reveal latency patterns, expose upstream failures, and support investigation during incidents. Metrics and logs are therefore aligned with request identifiers and route identifiers.

- Define the policy in a persistent or external configuration model.
- Expose the decision through audit events and metrics.
- Add positive and negative tests for the behavior.
- Document the expected error response and operational impact.

1.3 Problem Statement

Modern microservice systems often expose multiple backend services that duplicate authentication, authorization, rate limiting, request validation, logging, and resilience logic. In Sentra Gateway, this topic is important because the gateway is the first component that receives untrusted traffic. A weak design at this layer can expose every internal service, while a strong design can centralize control and reduce repeated security code across the system.

The design decision for this section is to make behavior explicit, measurable, and testable. Every important action should either produce a route decision, an audit event, a metric, or a safe error response. This gives developers and evaluators evidence that the system is behaving according to the documented requirements.

A production-style gateway must balance strict security with usability. If rules are too loose, the system becomes unsafe. If rules are too strict or poorly documented, legitimate clients will fail unexpectedly. Sentra Gateway therefore uses route-level policies, subject-specific limits, and documented response codes.

The module should be implemented through small services rather than one large filter. The filter extracts request data and delegates to services; the services perform validation, policy checks, logging, and decision making. This structure allows unit testing and reduces the chance of hidden coupling.

The design favors explicit policy definitions over hidden behavior. A gateway that silently permits or denies traffic without clear policy ownership becomes difficult to operate. For that reason, Sentra Gateway models routes, permissions, API keys, rate limits, IP rules, and audit event types as first-class concepts.

Observability is treated as part of the architecture instead of an afterthought. A security gateway must explain its decisions, reveal latency patterns, expose upstream failures, and support investigation during incidents. Metrics and logs are therefore aligned with request identifiers and route identifiers.

- Define the policy in a persistent or external configuration model.
- Expose the decision through audit events and metrics.
- Add positive and negative tests for the behavior.
- Document the expected error response and operational impact.

Problem statement: Microservice systems need a centralized, auditable, and observable gateway that can enforce security policies, control traffic, and protect downstream services before untrusted requests reach internal APIs.

1.4 Aim and Objectives

Objective ID	Objective	Description
OBJ-01	Centralized Routing	Route external requests to internal services through controlled route definitions.
OBJ-02	Authentication Enforcement	Validate JWT tokens and API keys before allowing protected access.
OBJ-03	Authorization Control	Enforce route-level roles and scopes for users and API clients.
OBJ-04	Traffic Throttling	Apply Redis-backed rate limits by IP, user, API key, tenant, route, and method.
OBJ-05	Request Integrity	Validate request signatures and prevent replay attacks.
OBJ-06	Risk Reduction	Apply IP policies and suspicious behavior rules before routing.
OBJ-07	Auditability	Record structured decisions for security review and incident investigation.
OBJ-08	Observability	Expose metrics and dashboards for traffic, latency, errors, and blocked requests.
OBJ-09	Reliability	Use timeouts, retries, circuit breakers, and fallback responses.
OBJ-10	Operational Readiness	Provide Docker Compose setup, OpenAPI docs, tests, and runbooks.

Table 1. Core project objectives

1.5 Scope

The scope includes gateway routing, security enforcement, policy administration, audit logging, observability, and performance evaluation. In Sentra Gateway, this topic is important because the gateway is the first component that receives untrusted traffic. A weak design at this layer can expose every internal service, while a strong design can centralize control and reduce repeated security code across the system.

The design decision for this section is to make behavior explicit, measurable, and testable. Every important action should either produce a route decision, an audit event, a metric, or a safe error response. This gives developers and evaluators evidence that the system is behaving according to the documented requirements.

A production-style gateway must balance strict security with usability. If rules are too loose, the system becomes unsafe. If rules are too strict or poorly documented, legitimate clients will fail unexpectedly. Sentra Gateway therefore uses route-level policies, subject-specific limits, and documented response codes.

The module should be implemented through small services rather than one large filter. The filter extracts request data and delegates to services; the services perform validation, policy checks, logging, and decision making. This structure allows unit testing and reduces the chance of hidden coupling.

The design favors explicit policy definitions over hidden behavior. A gateway that silently permits or denies traffic without clear policy ownership becomes difficult to operate. For that reason, Sentra Gateway models routes, permissions, API keys, rate limits, IP rules, and audit event types as first-class concepts.

Observability is treated as part of the architecture instead of an afterthought. A security gateway must explain its decisions, reveal latency patterns, expose upstream failures, and support investigation during incidents. Metrics and logs are therefore aligned with request identifiers and route identifiers.

- Define the policy in a persistent or external configuration model.
- Expose the decision through audit events and metrics.
- Add positive and negative tests for the behavior.
- Document the expected error response and operational impact.

1.6 Document Organization

The document is organized as a full technical project book covering background, requirements, design, implementation, testing, operations, and future work. In Sentra Gateway, this topic is important because the gateway is the first component that receives untrusted traffic. A weak design at this layer can expose every internal service, while a strong design can centralize control and reduce repeated security code across the system.

The design decision for this section is to make behavior explicit, measurable, and testable. Every important action should either produce a route decision, an audit event, a metric, or a safe error response. This gives developers and evaluators evidence that the system is behaving according to the documented requirements.

A production-style gateway must balance strict security with usability. If rules are too loose, the system becomes unsafe. If rules are too strict or poorly documented, legitimate clients will fail unexpectedly. Sentra Gateway therefore uses route-level policies, subject-specific limits, and documented response codes.

The module should be implemented through small services rather than one large filter. The filter extracts request data and delegates to services; the services perform validation, policy checks, logging, and decision making. This structure allows unit testing and reduces the chance of hidden coupling.

The design favors explicit policy definitions over hidden behavior. A gateway that silently permits or denies traffic without clear policy ownership becomes difficult to operate. For that reason, Sentra Gateway models routes, permissions, API keys, rate limits, IP rules, and audit event types as first-class concepts.

Observability is treated as part of the architecture instead of an afterthought. A security gateway must explain its decisions, reveal latency patterns, expose upstream failures, and support investigation during incidents. Metrics and logs are therefore aligned with request identifiers and route identifiers.

- Define the policy in a persistent or external configuration model.
- Expose the decision through audit events and metrics.
- Add positive and negative tests for the behavior.
- Document the expected error response and operational impact.

Chapter 2: Background and Technology Review

2.1 Microservices and Cross-Cutting Concerns

Microservice architecture divides a system into independently deployable services, but cross-cutting concerns appear repeatedly across them. In Sentra Gateway, this topic is important because the gateway is the first component that receives untrusted traffic. A weak design at this layer can expose every internal service, while a strong design can centralize control and reduce repeated security code across the system.

The design decision for this section is to make behavior explicit, measurable, and testable. Every important action should either produce a route decision, an audit event, a metric, or a safe error response. This gives developers and evaluators evidence that the system is behaving according to the documented requirements.

A production-style gateway must balance strict security with usability. If rules are too loose, the system becomes unsafe. If rules are too strict or poorly documented, legitimate clients will fail unexpectedly. Sentra Gateway therefore uses route-level policies, subject-specific limits, and documented response codes.

The module should be implemented through small services rather than one large filter. The filter extracts request data and delegates to services; the services perform validation, policy checks, logging, and decision making. This structure allows unit testing and reduces the chance of hidden coupling.

The design favors explicit policy definitions over hidden behavior. A gateway that silently permits or denies traffic without clear policy ownership becomes difficult to operate. For that reason, Sentra Gateway models routes, permissions, API keys, rate limits, IP rules, and audit event types as first-class concepts.

Observability is treated as part of the architecture instead of an afterthought. A security gateway must explain its decisions, reveal latency patterns, expose upstream failures, and support investigation during incidents. Metrics and logs are therefore aligned with request identifiers and route identifiers.

- Define the policy in a persistent or external configuration model.
- Expose the decision through audit events and metrics.
- Add positive and negative tests for the behavior.
- Document the expected error response and operational impact.

2.2 API Gateway Pattern

An API gateway acts as the controlled public entry point for a system and applies policies before forwarding requests. In Sentra Gateway, this topic is important because the gateway is the first component that receives untrusted traffic. A weak design at this layer can expose every internal service, while a strong design can centralize control and reduce repeated security code across the system.

The design decision for this section is to make behavior explicit, measurable, and testable. Every important action should either produce a route decision, an audit event, a metric, or a safe error response. This gives developers and evaluators evidence that the system is behaving according to the documented requirements.

A production-style gateway must balance strict security with usability. If rules are too loose, the system becomes unsafe. If rules are too strict or poorly documented, legitimate clients will fail unexpectedly. Sentra Gateway therefore uses route-level policies, subject-specific limits, and documented response codes.

The module should be implemented through small services rather than one large filter. The filter extracts request data and delegates to services; the services perform validation, policy checks, logging, and decision making. This structure allows unit testing and reduces the chance of hidden coupling.

The design favors explicit policy definitions over hidden behavior. A gateway that silently permits or denies traffic without clear policy ownership becomes difficult to operate. For that reason, Sentra Gateway models routes, permissions, API keys, rate limits, IP rules, and audit event types as first-class concepts.

Observability is treated as part of the architecture instead of an afterthought. A security gateway must explain its decisions, reveal latency patterns, expose upstream failures, and support investigation during incidents. Metrics and logs are therefore aligned with request identifiers and route identifiers.

- Define the policy in a persistent or external configuration model.
- Expose the decision through audit events and metrics.
- Add positive and negative tests for the behavior.
- Document the expected error response and operational impact.

2.3 Spring Cloud Gateway

Spring Cloud Gateway provides routing and filter-chain capabilities that fit the Sentra Gateway design. In Sentra Gateway, this topic is important because the gateway is the first component that receives untrusted traffic. A weak design at this layer can expose every internal service, while a strong design can centralize control and reduce repeated security code across the system.

The design decision for this section is to make behavior explicit, measurable, and testable. Every important action should either produce a route decision, an audit event, a metric, or a safe error response. This gives developers and evaluators evidence that the system is behaving according to the documented requirements.

A production-style gateway must balance strict security with usability. If rules are too loose, the system becomes unsafe. If rules are too strict or poorly documented, legitimate clients will fail unexpectedly. Sentra Gateway therefore uses route-level policies, subject-specific limits, and documented response codes.

The module should be implemented through small services rather than one large filter. The filter extracts request data and delegates to services; the services perform validation, policy checks, logging, and decision making. This structure allows unit testing and reduces the chance of hidden coupling.

The design favors explicit policy definitions over hidden behavior. A gateway that silently permits or denies traffic without clear policy ownership becomes difficult to operate. For that reason, Sentra Gateway models routes, permissions, API keys, rate limits, IP rules, and audit event types as first-class concepts.

Observability is treated as part of the architecture instead of an afterthought. A security gateway must explain its decisions, reveal latency patterns, expose upstream failures, and support investigation during incidents. Metrics and logs are therefore aligned with request identifiers and route identifiers.

- Define the policy in a persistent or external configuration model.
- Expose the decision through audit events and metrics.
- Add positive and negative tests for the behavior.
- Document the expected error response and operational impact.

2.4 JSON Web Tokens

JWTs carry signed authentication claims that the gateway can validate before forwarding user requests. In Sentra Gateway, this topic is important because the gateway is the first component that receives untrusted traffic. A weak design at this layer can expose every internal service, while a strong design can centralize control and reduce repeated security code across the system.

The design decision for this section is to make behavior explicit, measurable, and testable. Every important action should either produce a route decision, an audit event, a metric, or a safe error response. This gives developers and evaluators evidence that the system is behaving according to the documented requirements.

A production-style gateway must balance strict security with usability. If rules are too loose, the system becomes unsafe. If rules are too strict or poorly documented, legitimate clients will fail unexpectedly. Sentra Gateway therefore uses route-level policies, subject-specific limits, and documented response codes.

The module should be implemented through small services rather than one large filter. The filter extracts request data and delegates to services; the services perform validation, policy checks, logging, and decision making. This structure allows unit testing and reduces the chance of hidden coupling.

The design favors explicit policy definitions over hidden behavior. A gateway that silently permits or denies traffic without clear policy ownership becomes difficult to operate. For that reason, Sentra Gateway models routes, permissions, API keys, rate limits, IP rules, and audit event types as first-class concepts.

Observability is treated as part of the architecture instead of an afterthought. A security gateway must explain its decisions, reveal latency patterns, expose upstream failures, and support investigation during incidents. Metrics and logs are therefore aligned with request identifiers and route identifiers.

- Define the policy in a persistent or external configuration model.
- Expose the decision through audit events and metrics.
- Add positive and negative tests for the behavior.
- Document the expected error response and operational impact.

2.5 API Keys

API keys support machine-to-machine integration when a human user token is not appropriate. In Sentra Gateway, this topic is important because the gateway is the first component that receives untrusted traffic. A weak design at this layer can expose every internal service, while a strong design can centralize control and reduce repeated security code across the system.

The design decision for this section is to make behavior explicit, measurable, and testable. Every important action should either produce a route decision, an audit event, a metric, or a safe error response. This gives developers and evaluators evidence that the system is behaving according to the documented requirements.

A production-style gateway must balance strict security with usability. If rules are too loose, the system becomes unsafe. If rules are too strict or poorly documented, legitimate clients will fail unexpectedly. Sentra Gateway therefore uses route-level policies, subject-specific limits, and documented response codes.

The module should be implemented through small services rather than one large filter. The filter extracts request data and delegates to services; the services perform validation, policy checks, logging, and decision making. This structure allows unit testing and reduces the chance of hidden coupling.

The design favors explicit policy definitions over hidden behavior. A gateway that silently permits or denies traffic without clear policy ownership becomes difficult to operate. For that reason, Sentra Gateway models routes, permissions, API keys, rate limits, IP rules, and audit event types as first-class concepts.

Observability is treated as part of the architecture instead of an afterthought. A security gateway must explain its decisions, reveal latency patterns, expose upstream failures, and support investigation during incidents. Metrics and logs are therefore aligned with request identifiers and route identifiers.

- Define the policy in a persistent or external configuration model.
- Expose the decision through audit events and metrics.
- Add positive and negative tests for the behavior.
- Document the expected error response and operational impact.

2.6 OAuth2 and OIDC

OIDC integration allows the gateway to validate tokens issued by an external identity provider. In Sentra Gateway, this topic is important because the gateway is the first component that receives untrusted traffic. A weak design at this layer can expose every internal service, while a strong design can centralize control and reduce repeated security code across the system.

The design decision for this section is to make behavior explicit, measurable, and testable. Every important action should either produce a route decision, an audit event, a metric, or a safe error response. This gives developers and evaluators evidence that the system is behaving according to the documented requirements.

A production-style gateway must balance strict security with usability. If rules are too loose, the system becomes unsafe. If rules are too strict or poorly documented, legitimate clients will fail unexpectedly. Sentra Gateway therefore uses route-level policies, subject-specific limits, and documented response codes.

The module should be implemented through small services rather than one large filter. The filter extracts request data and delegates to services; the services perform validation, policy checks, logging, and decision making. This structure allows unit testing and reduces the chance of hidden coupling.

The design favors explicit policy definitions over hidden behavior. A gateway that silently permits or denies traffic without clear policy ownership becomes difficult to operate. For that reason, Sentra Gateway models routes, permissions, API keys, rate limits, IP rules, and audit event types as first-class concepts.

Observability is treated as part of the architecture instead of an afterthought. A security gateway must explain its decisions, reveal latency patterns, expose upstream failures, and support investigation during incidents. Metrics and logs are therefore aligned with request identifiers and route identifiers.

- Define the policy in a persistent or external configuration model.
- Expose the decision through audit events and metrics.
- Add positive and negative tests for the behavior.
- Document the expected error response and operational impact.

2.7 Redis and Fast Security State

Redis is used for low-latency counters, nonces, temporary blocks, and cached key data. In Sentra Gateway, this topic is important because the gateway is the first component that receives untrusted traffic. A weak design at this layer can expose every internal service, while a strong design can centralize control and reduce repeated security code across the system.

The design decision for this section is to make behavior explicit, measurable, and testable. Every important action should either produce a route decision, an audit event, a metric, or a safe error response. This gives developers and evaluators evidence that the system is behaving according to the documented requirements.

A production-style gateway must balance strict security with usability. If rules are too loose, the system becomes unsafe. If rules are too strict or poorly documented, legitimate clients will fail unexpectedly. Sentra Gateway therefore uses route-level policies, subject-specific limits, and documented response codes.

The module should be implemented through small services rather than one large filter. The filter extracts request data and delegates to services; the services perform validation, policy checks, logging, and decision making. This structure allows unit testing and reduces the chance of hidden coupling.

The design favors explicit policy definitions over hidden behavior. A gateway that silently permits or denies traffic without clear policy ownership becomes difficult to operate. For that reason, Sentra Gateway models routes, permissions, API keys, rate limits, IP rules, and audit event types as first-class concepts.

Observability is treated as part of the architecture instead of an afterthought. A security gateway must explain its decisions, reveal latency patterns, expose upstream failures, and support investigation during incidents. Metrics and logs are therefore aligned with request identifiers and route identifiers.

- Define the policy in a persistent or external configuration model.
- Expose the decision through audit events and metrics.
- Add positive and negative tests for the behavior.
- Document the expected error response and operational impact.

2.8 PostgreSQL and Policy Persistence

PostgreSQL stores durable configuration, API client records, route policies, and audit events. In Sentra Gateway, this topic is important because the gateway is the first component that receives untrusted traffic. A weak design at this layer can expose every internal service, while a strong design can centralize control and reduce repeated security code across the system.

The design decision for this section is to make behavior explicit, measurable, and testable. Every important action should either produce a route decision, an audit event, a metric, or a safe error response. This gives developers and evaluators evidence that the system is behaving according to the documented requirements.

A production-style gateway must balance strict security with usability. If rules are too loose, the system becomes unsafe. If rules are too strict or poorly documented, legitimate clients will fail unexpectedly. Sentra Gateway therefore uses route-level policies, subject-specific limits, and documented response codes.

The module should be implemented through small services rather than one large filter. The filter extracts request data and delegates to services; the services perform validation, policy checks, logging, and decision making. This structure allows unit testing and reduces the chance of hidden coupling.

The design favors explicit policy definitions over hidden behavior. A gateway that silently permits or denies traffic without clear policy ownership becomes difficult to operate. For that reason, Sentra Gateway models routes, permissions, API keys, rate limits, IP rules, and audit event types as first-class concepts.

Observability is treated as part of the architecture instead of an afterthought. A security gateway must explain its decisions, reveal latency patterns, expose upstream failures, and support investigation during incidents. Metrics and logs are therefore aligned with request identifiers and route identifiers.

- Define the policy in a persistent or external configuration model.
- Expose the decision through audit events and metrics.
- Add positive and negative tests for the behavior.
- Document the expected error response and operational impact.

2.9 Observability

Observability enables engineers to understand gateway behavior through logs, metrics, dashboards, and audit records. In Sentra Gateway, this topic is important because the gateway is the first component that receives untrusted traffic. A weak

design at this layer can expose every internal service, while a strong design can centralize control and reduce repeated security code across the system.

The design decision for this section is to make behavior explicit, measurable, and testable. Every important action should either produce a route decision, an audit event, a metric, or a safe error response. This gives developers and evaluators evidence that the system is behaving according to the documented requirements.

A production-style gateway must balance strict security with usability. If rules are too loose, the system becomes unsafe. If rules are too strict or poorly documented, legitimate clients will fail unexpectedly. Sentra Gateway therefore uses route-level policies, subject-specific limits, and documented response codes.

The module should be implemented through small services rather than one large filter. The filter extracts request data and delegates to services; the services perform validation, policy checks, logging, and decision making. This structure allows unit testing and reduces the chance of hidden coupling.

The design favors explicit policy definitions over hidden behavior. A gateway that silently permits or denies traffic without clear policy ownership becomes difficult to operate. For that reason, Sentra Gateway models routes, permissions, API keys, rate limits, IP rules, and audit event types as first-class concepts.

Observability is treated as part of the architecture instead of an afterthought. A security gateway must explain its decisions, reveal latency patterns, expose upstream failures, and support investigation during incidents. Metrics and logs are therefore aligned with request identifiers and route identifiers.

- Define the policy in a persistent or external configuration model.
- Expose the decision through audit events and metrics.
- Add positive and negative tests for the behavior.
- Document the expected error response and operational impact.

2.10 Resilience Engineering

Resilience patterns prevent downstream failures from becoming full system failures. In Sentra Gateway, this topic is important because the gateway is the first component that receives untrusted traffic. A weak design at this layer can expose every internal service, while a strong design can centralize control and reduce repeated security code across the system.

The design decision for this section is to make behavior explicit, measurable, and testable. Every important action should either produce a route decision, an audit event, a metric, or a safe error response. This gives developers and evaluators evidence that the system is behaving according to the documented requirements.

A production-style gateway must balance strict security with usability. If rules are too loose, the system becomes unsafe. If rules are too strict or poorly documented, legitimate clients will fail unexpectedly. Sentra Gateway therefore uses route-level policies, subject-specific limits, and documented response codes.

The module should be implemented through small services rather than one large filter. The filter extracts request data and delegates to services; the services perform validation, policy checks, logging, and decision making. This structure allows unit testing and reduces the chance of hidden coupling.

The design favors explicit policy definitions over hidden behavior. A gateway that silently permits or denies traffic without clear policy ownership becomes difficult to operate. For that reason, Sentra Gateway models routes, permissions, API keys, rate limits, IP rules, and audit event types as first-class concepts.

Observability is treated as part of the architecture instead of an afterthought. A security gateway must explain its decisions, reveal latency patterns, expose upstream failures, and support investigation during incidents. Metrics and logs are therefore aligned with request identifiers and route identifiers.

- Define the policy in a persistent or external configuration model.
- Expose the decision through audit events and metrics.
- Add positive and negative tests for the behavior.
- Document the expected error response and operational impact.

2.11 Technology Stack Summary

Layer	Technology	Purpose
Language	Java 25	Main implementation language.
Framework	Spring Boot 4	Application framework, validation, configuration, testing, and actuator support.

Layer	Technology	Purpose
Gateway	Spring Cloud Gateway	Routing, predicates, and filter chain.
Database	PostgreSQL	Routes, policies, API clients, keys, permissions, and audit events.
Cache	Redis	Counters, nonces, temporary blocks, and cached key material.
Reliability	Resilience4j	Circuit breakers, retries, timeouts, and fallbacks.
Observability	Micrometer, Prometheus, Grafana	Metrics and dashboards.
Testing	JUnit, Spring Test, Testcontainers, jquik, k6	Automated, integration, property-based, and load testing.
Deployment	Docker Compose	Local multi-service environment.

Table 2. Technology stack summary

Chapter 3: Problem Analysis and Requirements

3.1 Existing Problem Context

A backend platform composed of user-service, order-service, payment-service, notification-service, and admin-service can become difficult to protect if each service is exposed independently. In Sentra Gateway, this topic is important because the gateway is the first component that receives untrusted traffic. A weak design at this layer can expose every internal service, while a strong design can centralize control and reduce repeated security code across the system.

The design decision for this section is to make behavior explicit, measurable, and testable. Every important action should either produce a route decision, an audit event, a metric, or a safe error response. This gives developers and evaluators evidence that the system is behaving according to the documented requirements.

A production-style gateway must balance strict security with usability. If rules are too loose, the system becomes unsafe. If rules are too strict or poorly documented, legitimate clients will fail unexpectedly. Sentra Gateway therefore uses route-level policies, subject-specific limits, and documented response codes.

The module should be implemented through small services rather than one large filter. The filter extracts request data and delegates to services; the services perform validation, policy checks, logging, and decision making. This structure allows unit testing and reduces the chance of hidden coupling.

The design favors explicit policy definitions over hidden behavior. A gateway that silently permits or denies traffic without clear policy ownership becomes difficult to operate. For that reason, Sentra Gateway models routes, permissions, API keys, rate limits, IP rules, and audit event types as first-class concepts.

Observability is treated as part of the architecture instead of an afterthought. A security gateway must explain its decisions, reveal latency patterns, expose upstream failures, and support investigation during incidents. Metrics and logs are therefore aligned with request identifiers and route identifiers.

- Define the policy in a persistent or external configuration model.
- Expose the decision through audit events and metrics.
- Add positive and negative tests for the behavior.
- Document the expected error response and operational impact.

3.2 Stakeholders

Stakeholder	Interest	Gateway Benefit
End User	Reliable and secure API access.	Receives consistent responses and protection from service instability.
API Client Developer	Clear access rules and predictable limits.	Uses documented routes, scopes, keys, and errors.
Backend Developer	Reduced duplicated security logic.	Focuses on domain logic while gateway handles edge controls.
Security Administrator	Central visibility into access decisions.	Reviews audit logs, IP rules, keys, and suspicious traffic.
System Operator	Operational dashboards and alerts.	Monitors latency, error rate, circuit breakers, and upstream health.
Project Evaluator	Evidence of design and implementation depth.	Reviews architecture, tests, policies, and evaluation results.

Table 3. Stakeholder roles and interests

3.3 Functional Requirements

ID	Requirement	Description
FR-01	Route Management	Route requests to configured downstream services.
FR-02	Dynamic Route Configuration	Allow administrators to create, update, disable, and delete routes.

ID	Requirement	Description
FR-03	JWT Validation	Validate signature, expiration, issuer, audience, and claims.
FR-04	API Key Validation	Validate machine clients using hashed key material.
FR-05	API Key Rotation	Rotate and disable API keys safely.
FR-06	Route Permissions	Enforce roles and scopes per route.
FR-07	Rate Limiting	Apply Redis-backed limits by IP, user, key, tenant, route, and method.
FR-08	Request Signing	Validate signed requests using method, path, body hash, timestamp, and nonce.
FR-09	Replay Protection	Reject repeated nonces within the replay window.
FR-10	IP Policies	Support allowlists, blocklists, CIDR ranges, and temporary blocks.
FR-11	Risk Rules	Calculate suspicious behavior risk scores.
FR-12	Audit Logging	Persist structured gateway decisions.
FR-13	Metrics Exposure	Expose metrics for Prometheus.
FR-14	Resilience Policies	Support timeout, retry, circuit breaker, and fallback behavior.
FR-15	Admin APIs	Expose protected admin APIs for policies and logs.
FR-16	OpenAPI Documentation	Provide interactive API documentation.

Table 4. Functional requirements

3.4 Non-Functional Requirements

ID	Quality Attribute	Requirement
NFR-01	Security	Secrets, keys, tokens, and credentials must not be logged or stored in plaintext.
NFR-02	Performance	Gateway decision overhead should remain low enough for normal API traffic.
NFR-03	Scalability	Multiple gateway instances should share Redis-backed operational state.
NFR-04	Availability	Downstream failure should produce controlled responses.
NFR-05	Observability	Metrics and audit records must explain important behavior.
NFR-06	Maintainability	Modules should have clear boundaries and tests.
NFR-07	Configurability	Profiles and environment variables should separate environments.
NFR-08	Auditability	Administrative changes and decisions should be traceable.
NFR-09	Compatibility	Standard HTTP clients and auth header formats should be supported.
NFR-10	Testability	Security and policy behavior must be

ID	Quality Attribute	Requirement
		automatically tested.

Table 5. Non-functional requirements

3.5 Constraints and Assumptions

The design assumes downstream services are reachable through internal network addresses and that production deployments use TLS. In Sentra Gateway, this topic is important because the gateway is the first component that receives untrusted traffic. A weak design at this layer can expose every internal service, while a strong design can centralize control and reduce repeated security code across the system.

The design decision for this section is to make behavior explicit, measurable, and testable. Every important action should either produce a route decision, an audit event, a metric, or a safe error response. This gives developers and evaluators evidence that the system is behaving according to the documented requirements.

A production-style gateway must balance strict security with usability. If rules are too loose, the system becomes unsafe. If rules are too strict or poorly documented, legitimate clients will fail unexpectedly. Sentra Gateway therefore uses route-level policies, subject-specific limits, and documented response codes.

The module should be implemented through small services rather than one large filter. The filter extracts request data and delegates to services; the services perform validation, policy checks, logging, and decision making. This structure allows unit testing and reduces the chance of hidden coupling.

The design favors explicit policy definitions over hidden behavior. A gateway that silently permits or denies traffic without clear policy ownership becomes difficult to operate. For that reason, Sentra Gateway models routes, permissions, API keys, rate limits, IP rules, and audit event types as first-class concepts.

Observability is treated as part of the architecture instead of an afterthought. A security gateway must explain its decisions, reveal latency patterns, expose upstream failures, and support investigation during incidents. Metrics and logs are therefore aligned with request identifiers and route identifiers.

- Define the policy in a persistent or external configuration model.
- Expose the decision through audit events and metrics.
- Add positive and negative tests for the behavior.
- Document the expected error response and operational impact.

3.6 Success Criteria

Success is measured by correct policy enforcement, successful routing, audit completeness, dashboards, and reproducible tests. In Sentra Gateway, this topic is important because the gateway is the first component that receives untrusted traffic. A weak design at this layer can expose every internal service, while a strong design can centralize control and reduce repeated security code across the system.

The design decision for this section is to make behavior explicit, measurable, and testable. Every important action should either produce a route decision, an audit event, a metric, or a safe error response. This gives developers and evaluators evidence that the system is behaving according to the documented requirements.

A production-style gateway must balance strict security with usability. If rules are too loose, the system becomes unsafe. If rules are too strict or poorly documented, legitimate clients will fail unexpectedly. Sentra Gateway therefore uses route-level policies, subject-specific limits, and documented response codes.

The module should be implemented through small services rather than one large filter. The filter extracts request data and delegates to services; the services perform validation, policy checks, logging, and decision making. This structure allows unit testing and reduces the chance of hidden coupling.

The design favors explicit policy definitions over hidden behavior. A gateway that silently permits or denies traffic without clear policy ownership becomes difficult to operate. For that reason, Sentra Gateway models routes, permissions, API keys, rate limits, IP rules, and audit event types as first-class concepts.

Observability is treated as part of the architecture instead of an afterthought. A security gateway must explain its decisions, reveal latency patterns, expose upstream failures, and support investigation during incidents. Metrics and logs are therefore aligned with request identifiers and route identifiers.

- Define the policy in a persistent or external configuration model.
- Expose the decision through audit events and metrics.

- Add positive and negative tests for the behavior.
- Document the expected error response and operational impact.

3.7 Requirements Traceability

Every major requirement should map to implementation modules and test evidence. In Sentra Gateway, this topic is important because the gateway is the first component that receives untrusted traffic. A weak design at this layer can expose every internal service, while a strong design can centralize control and reduce repeated security code across the system.

The design decision for this section is to make behavior explicit, measurable, and testable. Every important action should either produce a route decision, an audit event, a metric, or a safe error response. This gives developers and evaluators evidence that the system is behaving according to the documented requirements.

A production-style gateway must balance strict security with usability. If rules are too loose, the system becomes unsafe. If rules are too strict or poorly documented, legitimate clients will fail unexpectedly. Sentra Gateway therefore uses route-level policies, subject-specific limits, and documented response codes.

The module should be implemented through small services rather than one large filter. The filter extracts request data and delegates to services; the services perform validation, policy checks, logging, and decision making. This structure allows unit testing and reduces the chance of hidden coupling.

The design favors explicit policy definitions over hidden behavior. A gateway that silently permits or denies traffic without clear policy ownership becomes difficult to operate. For that reason, Sentra Gateway models routes, permissions, API keys, rate limits, IP rules, and audit event types as first-class concepts.

Observability is treated as part of the architecture instead of an afterthought. A security gateway must explain its decisions, reveal latency patterns, expose upstream failures, and support investigation during incidents. Metrics and logs are therefore aligned with request identifiers and route identifiers.

- Define the policy in a persistent or external configuration model.
- Expose the decision through audit events and metrics.
- Add positive and negative tests for the behavior.
- Document the expected error response and operational impact.

Chapter 4: System Analysis

4.1 System Context

Sentra Gateway sits between external clients and internal services. In Sentra Gateway, this topic is important because the gateway is the first component that receives untrusted traffic. A weak design at this layer can expose every internal service, while a strong design can centralize control and reduce repeated security code across the system.

The design decision for this section is to make behavior explicit, measurable, and testable. Every important action should either produce a route decision, an audit event, a metric, or a safe error response. This gives developers and evaluators evidence that the system is behaving according to the documented requirements.

A production-style gateway must balance strict security with usability. If rules are too loose, the system becomes unsafe. If rules are too strict or poorly documented, legitimate clients will fail unexpectedly. Sentra Gateway therefore uses route-level policies, subject-specific limits, and documented response codes.

The module should be implemented through small services rather than one large filter. The filter extracts request data and delegates to services; the services perform validation, policy checks, logging, and decision making. This structure allows unit testing and reduces the chance of hidden coupling.

The design favors explicit policy definitions over hidden behavior. A gateway that silently permits or denies traffic without clear policy ownership becomes difficult to operate. For that reason, Sentra Gateway models routes, permissions, API keys, rate limits, IP rules, and audit event types as first-class concepts.

Observability is treated as part of the architecture instead of an afterthought. A security gateway must explain its decisions, reveal latency patterns, expose upstream failures, and support investigation during incidents. Metrics and logs are therefore aligned with request identifiers and route identifiers.

- Define the policy in a persistent or external configuration model.
- Expose the decision through audit events and metrics.
- Add positive and negative tests for the behavior.
- Document the expected error response and operational impact.

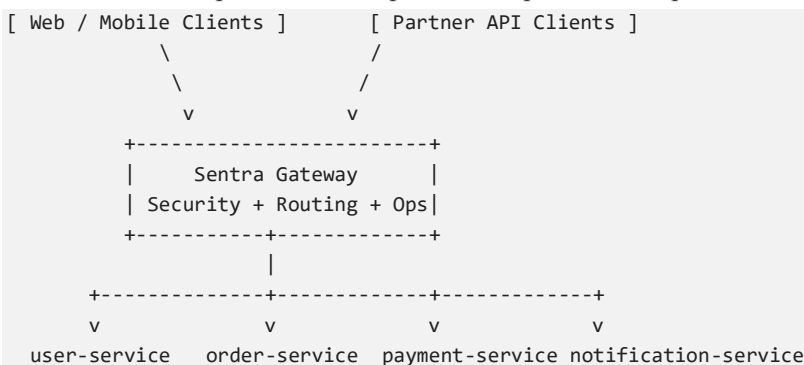


Figure 1. High-level Sentra Gateway system context

4.2 Actors

Actor	Description	Main Interaction
Anonymous Client	Client without verified identity.	Can access public routes only under strict rate limits.
Authenticated User	Client presenting valid JWT.	Accesses user routes according to roles and permissions.
API Client	Machine integration using API key or signed request.	Accesses scoped partner routes.
Administrator	Privileged operator.	Configures routes, keys, IP rules, rate policies, and audits.
Downstream Service	Internal backend service.	Receives validated requests with trusted headers.
Monitoring System	Prometheus/Grafana.	Scrapes and visualizes metrics.

Table 6. Primary system actors

4.3 Use Cases

ID	Use Case	Description
UC-01	Forward Public Request	Client calls a public route and gateway forwards it.
UC-02	Access Protected Route with JWT	Gateway validates JWT and route permissions.
UC-03	Access Partner Route with API Key	Gateway validates API key, scopes, and limits.
UC-04	Reject Unauthorized Request	Gateway rejects invalid or missing credentials.
UC-05	Apply Rate Limit	Gateway rejects traffic exceeding policy.
UC-06	Validate Signed Request	Gateway verifies signature, timestamp, body hash, and nonce.
UC-07	Block Suspicious IP	Gateway checks IP policy before routing.
UC-08	Record Audit Event	Gateway records important decisions.
UC-09	Monitor Gateway Health	Operator views metrics and dashboards.
UC-10	Configure Route Policy	Admin creates or updates route security requirements.

Table 7. Use case summary

4.4 Request Lifecycle

A request passes through deterministic gateway stages before reaching a downstream service. In Sentra Gateway, this topic is important because the gateway is the first component that receives untrusted traffic. A weak design at this layer can expose every internal service, while a strong design can centralize control and reduce repeated security code across the system.

The design decision for this section is to make behavior explicit, measurable, and testable. Every important action should either produce a route decision, an audit event, a metric, or a safe error response. This gives developers and evaluators evidence that the system is behaving according to the documented requirements.

A production-style gateway must balance strict security with usability. If rules are too loose, the system becomes unsafe. If rules are too strict or poorly documented, legitimate clients will fail unexpectedly. Sentra Gateway therefore uses route-level policies, subject-specific limits, and documented response codes.

The module should be implemented through small services rather than one large filter. The filter extracts request data and delegates to services; the services perform validation, policy checks, logging, and decision making. This structure allows unit testing and reduces the chance of hidden coupling.

The design favors explicit policy definitions over hidden behavior. A gateway that silently permits or denies traffic without clear policy ownership becomes difficult to operate. For that reason, Sentra Gateway models routes, permissions, API keys, rate limits, IP rules, and audit event types as first-class concepts.

Observability is treated as part of the architecture instead of an afterthought. A security gateway must explain its decisions, reveal latency patterns, expose upstream failures, and support investigation during incidents. Metrics and logs are therefore aligned with request identifiers and route identifiers.

- Define the policy in a persistent or external configuration model.
- Expose the decision through audit events and metrics.
- Add positive and negative tests for the behavior.
- Document the expected error response and operational impact.

Client Request

```
|
Request ID -> IP Policy -> Authentication -> Authorization
|
```

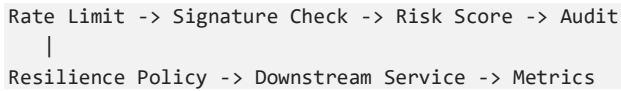


Figure 2. Gateway request processing pipeline

4.5 Without vs With Sentra Gateway

Centralizing controls reduces duplication, improves visibility, and decreases direct exposure of internal services. In Sentra Gateway, this topic is important because the gateway is the first component that receives untrusted traffic. A weak design at this layer can expose every internal service, while a strong design can centralize control and reduce repeated security code across the system.

The design decision for this section is to make behavior explicit, measurable, and testable. Every important action should either produce a route decision, an audit event, a metric, or a safe error response. This gives developers and evaluators evidence that the system is behaving according to the documented requirements.

A production-style gateway must balance strict security with usability. If rules are too loose, the system becomes unsafe. If rules are too strict or poorly documented, legitimate clients will fail unexpectedly. Sentra Gateway therefore uses route-level policies, subject-specific limits, and documented response codes.

The module should be implemented through small services rather than one large filter. The filter extracts request data and delegates to services; the services perform validation, policy checks, logging, and decision making. This structure allows unit testing and reduces the chance of hidden coupling.

The design favors explicit policy definitions over hidden behavior. A gateway that silently permits or denies traffic without clear policy ownership becomes difficult to operate. For that reason, Sentra Gateway models routes, permissions, API keys, rate limits, IP rules, and audit event types as first-class concepts.

Observability is treated as part of the architecture instead of an afterthought. A security gateway must explain its decisions, reveal latency patterns, expose upstream failures, and support investigation during incidents. Metrics and logs are therefore aligned with request identifiers and route identifiers.

- Define the policy in a persistent or external configuration model.
- Expose the decision through audit events and metrics.
- Add positive and negative tests for the behavior.
- Document the expected error response and operational impact.

4.6 Sequence: JWT User Request

An authenticated user request demonstrates token validation, permission checking, rate limiting, forwarding, auditing, and metrics. In Sentra Gateway, this topic is important because the gateway is the first component that receives untrusted traffic. A weak design at this layer can expose every internal service, while a strong design can centralize control and reduce repeated security code across the system.

The design decision for this section is to make behavior explicit, measurable, and testable. Every important action should either produce a route decision, an audit event, a metric, or a safe error response. This gives developers and evaluators evidence that the system is behaving according to the documented requirements.

A production-style gateway must balance strict security with usability. If rules are too loose, the system becomes unsafe. If rules are too strict or poorly documented, legitimate clients will fail unexpectedly. Sentra Gateway therefore uses route-level policies, subject-specific limits, and documented response codes.

The module should be implemented through small services rather than one large filter. The filter extracts request data and delegates to services; the services perform validation, policy checks, logging, and decision making. This structure allows unit testing and reduces the chance of hidden coupling.

The design favors explicit policy definitions over hidden behavior. A gateway that silently permits or denies traffic without clear policy ownership becomes difficult to operate. For that reason, Sentra Gateway models routes, permissions, API keys, rate limits, IP rules, and audit event types as first-class concepts.

Observability is treated as part of the architecture instead of an afterthought. A security gateway must explain its decisions, reveal latency patterns, expose upstream failures, and support investigation during incidents. Metrics and logs are therefore aligned with request identifiers and route identifiers.

- Define the policy in a persistent or external configuration model.
- Expose the decision through audit events and metrics.
- Add positive and negative tests for the behavior.

- Document the expected error response and operational impact.

4.7 Sequence: API Key Partner Request

A partner request demonstrates API key validation, request signing, nonce storage, scope enforcement, and route forwarding. In Sentra Gateway, this topic is important because the gateway is the first component that receives untrusted traffic. A weak design at this layer can expose every internal service, while a strong design can centralize control and reduce repeated security code across the system.

The design decision for this section is to make behavior explicit, measurable, and testable. Every important action should either produce a route decision, an audit event, a metric, or a safe error response. This gives developers and evaluators evidence that the system is behaving according to the documented requirements.

A production-style gateway must balance strict security with usability. If rules are too loose, the system becomes unsafe. If rules are too strict or poorly documented, legitimate clients will fail unexpectedly. Sentra Gateway therefore uses route-level policies, subject-specific limits, and documented response codes.

The module should be implemented through small services rather than one large filter. The filter extracts request data and delegates to services; the services perform validation, policy checks, logging, and decision making. This structure allows unit testing and reduces the chance of hidden coupling.

The design favors explicit policy definitions over hidden behavior. A gateway that silently permits or denies traffic without clear policy ownership becomes difficult to operate. For that reason, Sentra Gateway models routes, permissions, API keys, rate limits, IP rules, and audit event types as first-class concepts.

Observability is treated as part of the architecture instead of an afterthought. A security gateway must explain its decisions, reveal latency patterns, expose upstream failures, and support investigation during incidents. Metrics and logs are therefore aligned with request identifiers and route identifiers.

- Define the policy in a persistent or external configuration model.
- Expose the decision through audit events and metrics.
- Add positive and negative tests for the behavior.
- Document the expected error response and operational impact.

Chapter 5: Architecture and System Design

5.1 Architectural Style

Sentra Gateway uses a modular layered architecture inside a gateway-centered deployment topology. In Sentra Gateway, this topic is important because the gateway is the first component that receives untrusted traffic. A weak design at this layer can expose every internal service, while a strong design can centralize control and reduce repeated security code across the system.

The design decision for this section is to make behavior explicit, measurable, and testable. Every important action should either produce a route decision, an audit event, a metric, or a safe error response. This gives developers and evaluators evidence that the system is behaving according to the documented requirements.

A production-style gateway must balance strict security with usability. If rules are too loose, the system becomes unsafe. If rules are too strict or poorly documented, legitimate clients will fail unexpectedly. Sentra Gateway therefore uses route-level policies, subject-specific limits, and documented response codes.

The module should be implemented through small services rather than one large filter. The filter extracts request data and delegates to services; the services perform validation, policy checks, logging, and decision making. This structure allows unit testing and reduces the chance of hidden coupling.

The design favors explicit policy definitions over hidden behavior. A gateway that silently permits or denies traffic without clear policy ownership becomes difficult to operate. For that reason, Sentra Gateway models routes, permissions, API keys, rate limits, IP rules, and audit event types as first-class concepts.

Observability is treated as part of the architecture instead of an afterthought. A security gateway must explain its decisions, reveal latency patterns, expose upstream failures, and support investigation during incidents. Metrics and logs are therefore aligned with request identifiers and route identifiers.

- Define the policy in a persistent or external configuration model.
- Expose the decision through audit events and metrics.
- Add positive and negative tests for the behavior.
- Document the expected error response and operational impact.

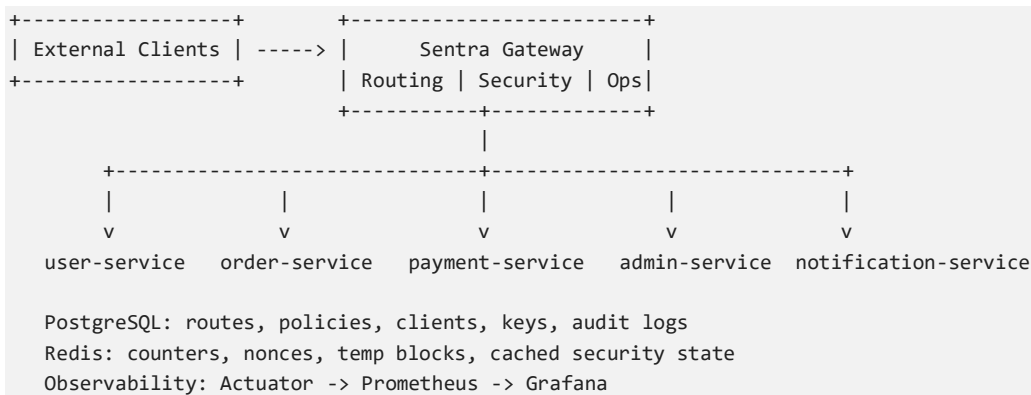


Figure 3. High-level Sentra Gateway architecture

5.2 Module Decomposition

Module	Responsibility
routing	Route definitions, predicates, target URIs, route metadata, and enable/disable state.
security.jwt	JWT validation, claims extraction, key rotation support, trusted headers.
security.apikey	API client registration, key hashing, scopes, rotation, usage metadata.
security.signing	HMAC signing, timestamp validation, body hash verification, nonce replay protection.
security.ip	Allowlist, blocklist, CIDR matching, temporary blocks.
security.risk	Rule-based bot detection and action decisions.

Module	Responsibility
ratelimit	Redis token bucket policies by subject and route.
authorization	Route-level roles and scopes.
audit	Structured audit event creation, persistence, and search.
observability	Micrometer metrics, request timers, counters, health indicators.
resilience	Timeout, retry, circuit breaker, and fallback behavior.
admin	Protected APIs for route, policy, client, and audit management.

Table 8. Module decomposition

5.3 Layered Design

The internal codebase separates REST/admin APIs, services, policy components, repositories, external clients, and infrastructure. In Sentra Gateway, this topic is important because the gateway is the first component that receives untrusted traffic. A weak design at this layer can expose every internal service, while a strong design can centralize control and reduce repeated security code across the system.

The design decision for this section is to make behavior explicit, measurable, and testable. Every important action should either produce a route decision, an audit event, a metric, or a safe error response. This gives developers and evaluators evidence that the system is behaving according to the documented requirements.

A production-style gateway must balance strict security with usability. If rules are too loose, the system becomes unsafe. If rules are too strict or poorly documented, legitimate clients will fail unexpectedly. Sentra Gateway therefore uses route-level policies, subject-specific limits, and documented response codes.

The module should be implemented through small services rather than one large filter. The filter extracts request data and delegates to services; the services perform validation, policy checks, logging, and decision making. This structure allows unit testing and reduces the chance of hidden coupling.

The design favors explicit policy definitions over hidden behavior. A gateway that silently permits or denies traffic without clear policy ownership becomes difficult to operate. For that reason, Sentra Gateway models routes, permissions, API keys, rate limits, IP rules, and audit event types as first-class concepts.

Observability is treated as part of the architecture instead of an afterthought. A security gateway must explain its decisions, reveal latency patterns, expose upstream failures, and support investigation during incidents. Metrics and logs are therefore aligned with request identifiers and route identifiers.

- Define the policy in a persistent or external configuration model.
- Expose the decision through audit events and metrics.
- Add positive and negative tests for the behavior.
- Document the expected error response and operational impact.

5.4 Filter Ordering

Security decisions depend on running checks in a deterministic and safe order. In Sentra Gateway, this topic is important because the gateway is the first component that receives untrusted traffic. A weak design at this layer can expose every internal service, while a strong design can centralize control and reduce repeated security code across the system.

The design decision for this section is to make behavior explicit, measurable, and testable. Every important action should either produce a route decision, an audit event, a metric, or a safe error response. This gives developers and evaluators evidence that the system is behaving according to the documented requirements.

A production-style gateway must balance strict security with usability. If rules are too loose, the system becomes unsafe. If rules are too strict or poorly documented, legitimate clients will fail unexpectedly. Sentra Gateway therefore uses route-level policies, subject-specific limits, and documented response codes.

The module should be implemented through small services rather than one large filter. The filter extracts request data and delegates to services; the services perform validation, policy checks, logging, and decision making. This structure allows unit testing and reduces the chance of hidden coupling.

The design favors explicit policy definitions over hidden behavior. A gateway that silently permits or denies traffic without clear policy ownership becomes difficult to operate. For that reason, Sentra Gateway models routes, permissions, API keys, rate limits, IP rules, and audit event types as first-class concepts.

Observability is treated as part of the architecture instead of an afterthought. A security gateway must explain its decisions, reveal latency patterns, expose upstream failures, and support investigation during incidents. Metrics and logs are therefore aligned with request identifiers and route identifiers.

- Define the policy in a persistent or external configuration model.
- Expose the decision through audit events and metrics.
- Add positive and negative tests for the behavior.
- Document the expected error response and operational impact.

5.5 Route Model

A route is more than a path: it carries target URI, method restrictions, authentication requirements, and resilience settings. In Sentra Gateway, this topic is important because the gateway is the first component that receives untrusted traffic. A weak design at this layer can expose every internal service, while a strong design can centralize control and reduce repeated security code across the system.

The design decision for this section is to make behavior explicit, measurable, and testable. Every important action should either produce a route decision, an audit event, a metric, or a safe error response. This gives developers and evaluators evidence that the system is behaving according to the documented requirements.

A production-style gateway must balance strict security with usability. If rules are too loose, the system becomes unsafe. If rules are too strict or poorly documented, legitimate clients will fail unexpectedly. Sentra Gateway therefore uses route-level policies, subject-specific limits, and documented response codes.

The module should be implemented through small services rather than one large filter. The filter extracts request data and delegates to services; the services perform validation, policy checks, logging, and decision making. This structure allows unit testing and reduces the chance of hidden coupling.

The design favors explicit policy definitions over hidden behavior. A gateway that silently permits or denies traffic without clear policy ownership becomes difficult to operate. For that reason, Sentra Gateway models routes, permissions, API keys, rate limits, IP rules, and audit event types as first-class concepts.

Observability is treated as part of the architecture instead of an afterthought. A security gateway must explain its decisions, reveal latency patterns, expose upstream failures, and support investigation during incidents. Metrics and logs are therefore aligned with request identifiers and route identifiers.

- Define the policy in a persistent or external configuration model.
- Expose the decision through audit events and metrics.
- Add positive and negative tests for the behavior.
- Document the expected error response and operational impact.

5.6 Trusted Header Propagation

Validated identities can be forwarded using gateway-controlled internal headers. In Sentra Gateway, this topic is important because the gateway is the first component that receives untrusted traffic. A weak design at this layer can expose every internal service, while a strong design can centralize control and reduce repeated security code across the system.

The design decision for this section is to make behavior explicit, measurable, and testable. Every important action should either produce a route decision, an audit event, a metric, or a safe error response. This gives developers and evaluators evidence that the system is behaving according to the documented requirements.

A production-style gateway must balance strict security with usability. If rules are too loose, the system becomes unsafe. If rules are too strict or poorly documented, legitimate clients will fail unexpectedly. Sentra Gateway therefore uses route-level policies, subject-specific limits, and documented response codes.

The module should be implemented through small services rather than one large filter. The filter extracts request data and delegates to services; the services perform validation, policy checks, logging, and decision making. This structure allows unit testing and reduces the chance of hidden coupling.

The design favors explicit policy definitions over hidden behavior. A gateway that silently permits or denies traffic without clear policy ownership becomes difficult to operate. For that reason, Sentra Gateway models routes, permissions, API keys, rate limits, IP rules, and audit event types as first-class concepts.

Observability is treated as part of the architecture instead of an afterthought. A security gateway must explain its decisions, reveal latency patterns, expose upstream failures, and support investigation during incidents. Metrics and logs are therefore aligned with request identifiers and route identifiers.

- Define the policy in a persistent or external configuration model.
- Expose the decision through audit events and metrics.
- Add positive and negative tests for the behavior.
- Document the expected error response and operational impact.

5.7 Docker Compose Topology

The local system includes the gateway, mock services, PostgreSQL, Redis, Prometheus, and Grafana. In Sentra Gateway, this topic is important because the gateway is the first component that receives untrusted traffic. A weak design at this layer can expose every internal service, while a strong design can centralize control and reduce repeated security code across the system.

The design decision for this section is to make behavior explicit, measurable, and testable. Every important action should either produce a route decision, an audit event, a metric, or a safe error response. This gives developers and evaluators evidence that the system is behaving according to the documented requirements.

A production-style gateway must balance strict security with usability. If rules are too loose, the system becomes unsafe. If rules are too strict or poorly documented, legitimate clients will fail unexpectedly. Sentra Gateway therefore uses route-level policies, subject-specific limits, and documented response codes.

The module should be implemented through small services rather than one large filter. The filter extracts request data and delegates to services; the services perform validation, policy checks, logging, and decision making. This structure allows unit testing and reduces the chance of hidden coupling.

The design favors explicit policy definitions over hidden behavior. A gateway that silently permits or denies traffic without clear policy ownership becomes difficult to operate. For that reason, Sentra Gateway models routes, permissions, API keys, rate limits, IP rules, and audit event types as first-class concepts.

Observability is treated as part of the architecture instead of an afterthought. A security gateway must explain its decisions, reveal latency patterns, expose upstream failures, and support investigation during incidents. Metrics and logs are therefore aligned with request identifiers and route identifiers.

- Define the policy in a persistent or external configuration model.
- Expose the decision through audit events and metrics.
- Add positive and negative tests for the behavior.
- Document the expected error response and operational impact.

5.8 Architecture Quality Attributes

Security, maintainability, observability, reliability, scalability, and testability are explicitly designed. In Sentra Gateway, this topic is important because the gateway is the first component that receives untrusted traffic. A weak design at this layer can expose every internal service, while a strong design can centralize control and reduce repeated security code across the system.

The design decision for this section is to make behavior explicit, measurable, and testable. Every important action should either produce a route decision, an audit event, a metric, or a safe error response. This gives developers and evaluators evidence that the system is behaving according to the documented requirements.

A production-style gateway must balance strict security with usability. If rules are too loose, the system becomes unsafe. If rules are too strict or poorly documented, legitimate clients will fail unexpectedly. Sentra Gateway therefore uses route-level policies, subject-specific limits, and documented response codes.

The module should be implemented through small services rather than one large filter. The filter extracts request data and delegates to services; the services perform validation, policy checks, logging, and decision making. This structure allows unit testing and reduces the chance of hidden coupling.

The design favors explicit policy definitions over hidden behavior. A gateway that silently permits or denies traffic without clear policy ownership becomes difficult to operate. For that reason, Sentra Gateway models routes, permissions, API keys, rate limits, IP rules, and audit event types as first-class concepts.

Observability is treated as part of the architecture instead of an afterthought. A security gateway must explain its decisions, reveal latency patterns, expose upstream failures, and support investigation during incidents. Metrics and logs are therefore aligned with request identifiers and route identifiers.

- Define the policy in a persistent or external configuration model.
- Expose the decision through audit events and metrics.
- Add positive and negative tests for the behavior.
- Document the expected error response and operational impact.

Chapter 6: Security Design

6.1 Security Goals

Security goals define what the gateway must protect and how it should behave under unsafe input. In Sentra Gateway, this topic is important because the gateway is the first component that receives untrusted traffic. A weak design at this layer can expose every internal service, while a strong design can centralize control and reduce repeated security code across the system.

The design decision for this section is to make behavior explicit, measurable, and testable. Every important action should either produce a route decision, an audit event, a metric, or a safe error response. This gives developers and evaluators evidence that the system is behaving according to the documented requirements.

A production-style gateway must balance strict security with usability. If rules are too loose, the system becomes unsafe. If rules are too strict or poorly documented, legitimate clients will fail unexpectedly. Sentra Gateway therefore uses route-level policies, subject-specific limits, and documented response codes.

The module should be implemented through small services rather than one large filter. The filter extracts request data and delegates to services; the services perform validation, policy checks, logging, and decision making. This structure allows unit testing and reduces the chance of hidden coupling.

The design favors explicit policy definitions over hidden behavior. A gateway that silently permits or denies traffic without clear policy ownership becomes difficult to operate. For that reason, Sentra Gateway models routes, permissions, API keys, rate limits, IP rules, and audit event types as first-class concepts.

Observability is treated as part of the architecture instead of an afterthought. A security gateway must explain its decisions, reveal latency patterns, expose upstream failures, and support investigation during incidents. Metrics and logs are therefore aligned with request identifiers and route identifiers.

- Reject invalid credentials.
- Prevent replay attacks.
- Avoid logging secrets.
- Keep admin APIs protected.

6.2 JWT Validation

JWT validation is used for user-facing protected routes. In Sentra Gateway, this topic is important because the gateway is the first component that receives untrusted traffic. A weak design at this layer can expose every internal service, while a strong design can centralize control and reduce repeated security code across the system.

The design decision for this section is to make behavior explicit, measurable, and testable. Every important action should either produce a route decision, an audit event, a metric, or a safe error response. This gives developers and evaluators evidence that the system is behaving according to the documented requirements.

A production-style gateway must balance strict security with usability. If rules are too loose, the system becomes unsafe. If rules are too strict or poorly documented, legitimate clients will fail unexpectedly. Sentra Gateway therefore uses route-level policies, subject-specific limits, and documented response codes.

The module should be implemented through small services rather than one large filter. The filter extracts request data and delegates to services; the services perform validation, policy checks, logging, and decision making. This structure allows unit testing and reduces the chance of hidden coupling.

The design favors explicit policy definitions over hidden behavior. A gateway that silently permits or denies traffic without clear policy ownership becomes difficult to operate. For that reason, Sentra Gateway models routes, permissions, API keys, rate limits, IP rules, and audit event types as first-class concepts.

Observability is treated as part of the architecture instead of an afterthought. A security gateway must explain its decisions, reveal latency patterns, expose upstream failures, and support investigation during incidents. Metrics and logs are therefore aligned with request identifiers and route identifiers.

- Verify signature.
- Validate issuer and audience.
- Check expiration.
- Extract roles and scopes.

6.3 API Key Security

API keys are used for machine-to-machine access and must be managed as sensitive secrets. In Sentra Gateway, this topic is important because the gateway is the first component that receives untrusted traffic. A weak design at this layer can expose every internal service, while a strong design can centralize control and reduce repeated security code across the system.

The design decision for this section is to make behavior explicit, measurable, and testable. Every important action should either produce a route decision, an audit event, a metric, or a safe error response. This gives developers and evaluators evidence that the system is behaving according to the documented requirements.

A production-style gateway must balance strict security with usability. If rules are too loose, the system becomes unsafe. If rules are too strict or poorly documented, legitimate clients will fail unexpectedly. Sentra Gateway therefore uses route-level policies, subject-specific limits, and documented response codes.

The module should be implemented through small services rather than one large filter. The filter extracts request data and delegates to services; the services perform validation, policy checks, logging, and decision making. This structure allows unit testing and reduces the chance of hidden coupling.

The design favors explicit policy definitions over hidden behavior. A gateway that silently permits or denies traffic without clear policy ownership becomes difficult to operate. For that reason, Sentra Gateway models routes, permissions, API keys, rate limits, IP rules, and audit event types as first-class concepts.

Observability is treated as part of the architecture instead of an afterthought. A security gateway must explain its decisions, reveal latency patterns, expose upstream failures, and support investigation during incidents. Metrics and logs are therefore aligned with request identifiers and route identifiers.

- Hash key material.
- Show raw key once.
- Support expiration and rotation.
- Track usage safely.

6.4 Request Signing

Request signing protects high-risk partner operations from tampering and replay. In Sentra Gateway, this topic is important because the gateway is the first component that receives untrusted traffic. A weak design at this layer can expose every internal service, while a strong design can centralize control and reduce repeated security code across the system.

The design decision for this section is to make behavior explicit, measurable, and testable. Every important action should either produce a route decision, an audit event, a metric, or a safe error response. This gives developers and evaluators evidence that the system is behaving according to the documented requirements.

A production-style gateway must balance strict security with usability. If rules are too loose, the system becomes unsafe. If rules are too strict or poorly documented, legitimate clients will fail unexpectedly. Sentra Gateway therefore uses route-level policies, subject-specific limits, and documented response codes.

The module should be implemented through small services rather than one large filter. The filter extracts request data and delegates to services; the services perform validation, policy checks, logging, and decision making. This structure allows unit testing and reduces the chance of hidden coupling.

The design favors explicit policy definitions over hidden behavior. A gateway that silently permits or denies traffic without clear policy ownership becomes difficult to operate. For that reason, Sentra Gateway models routes, permissions, API keys, rate limits, IP rules, and audit event types as first-class concepts.

Observability is treated as part of the architecture instead of an afterthought. A security gateway must explain its decisions, reveal latency patterns, expose upstream failures, and support investigation during incidents. Metrics and logs are therefore aligned with request identifiers and route identifiers.

- Validate timestamp.
- Verify body hash.
- Store nonce in Redis.
- Reject repeated nonces.

6.5 Rate Limiting

Rate limiting protects services from abuse, mistakes, scraping, brute force, and traffic spikes. In Sentra Gateway, this topic is important because the gateway is the first component that receives untrusted traffic. A weak design at this layer can

expose every internal service, while a strong design can centralize control and reduce repeated security code across the system.

The design decision for this section is to make behavior explicit, measurable, and testable. Every important action should either produce a route decision, an audit event, a metric, or a safe error response. This gives developers and evaluators evidence that the system is behaving according to the documented requirements.

A production-style gateway must balance strict security with usability. If rules are too loose, the system becomes unsafe. If rules are too strict or poorly documented, legitimate clients will fail unexpectedly. Sentra Gateway therefore uses route-level policies, subject-specific limits, and documented response codes.

The module should be implemented through small services rather than one large filter. The filter extracts request data and delegates to services; the services perform validation, policy checks, logging, and decision making. This structure allows unit testing and reduces the chance of hidden coupling.

The design favors explicit policy definitions over hidden behavior. A gateway that silently permits or denies traffic without clear policy ownership becomes difficult to operate. For that reason, Sentra Gateway models routes, permissions, API keys, rate limits, IP rules, and audit event types as first-class concepts.

Observability is treated as part of the architecture instead of an afterthought. A security gateway must explain its decisions, reveal latency patterns, expose upstream failures, and support investigation during incidents. Metrics and logs are therefore aligned with request identifiers and route identifiers.

- Limit by IP.
- Limit by user.
- Limit by API key.
- Limit by route.

6.6 IP Filtering

IP allow and block policies provide early rejection before expensive security checks. In Sentra Gateway, this topic is important because the gateway is the first component that receives untrusted traffic. A weak design at this layer can expose every internal service, while a strong design can centralize control and reduce repeated security code across the system.

The design decision for this section is to make behavior explicit, measurable, and testable. Every important action should either produce a route decision, an audit event, a metric, or a safe error response. This gives developers and evaluators evidence that the system is behaving according to the documented requirements.

A production-style gateway must balance strict security with usability. If rules are too loose, the system becomes unsafe. If rules are too strict or poorly documented, legitimate clients will fail unexpectedly. Sentra Gateway therefore uses route-level policies, subject-specific limits, and documented response codes.

The module should be implemented through small services rather than one large filter. The filter extracts request data and delegates to services; the services perform validation, policy checks, logging, and decision making. This structure allows unit testing and reduces the chance of hidden coupling.

The design favors explicit policy definitions over hidden behavior. A gateway that silently permits or denies traffic without clear policy ownership becomes difficult to operate. For that reason, Sentra Gateway models routes, permissions, API keys, rate limits, IP rules, and audit event types as first-class concepts.

Observability is treated as part of the architecture instead of an afterthought. A security gateway must explain its decisions, reveal latency patterns, expose upstream failures, and support investigation during incidents. Metrics and logs are therefore aligned with request identifiers and route identifiers.

- Support CIDR.
- Support temporary blocks.
- Protect admin routes.
- Audit blocked requests.

6.7 Bot Detection and Risk Scoring

Risk scoring uses transparent rules to detect suspicious patterns. In Sentra Gateway, this topic is important because the gateway is the first component that receives untrusted traffic. A weak design at this layer can expose every internal service, while a strong design can centralize control and reduce repeated security code across the system.

The design decision for this section is to make behavior explicit, measurable, and testable. Every important action should either produce a route decision, an audit event, a metric, or a safe error response. This gives developers and evaluators evidence that the system is behaving according to the documented requirements.

A production-style gateway must balance strict security with usability. If rules are too loose, the system becomes unsafe. If rules are too strict or poorly documented, legitimate clients will fail unexpectedly. Sentra Gateway therefore uses route-level policies, subject-specific limits, and documented response codes.

The module should be implemented through small services rather than one large filter. The filter extracts request data and delegates to services; the services perform validation, policy checks, logging, and decision making. This structure allows unit testing and reduces the chance of hidden coupling.

The design favors explicit policy definitions over hidden behavior. A gateway that silently permits or denies traffic without clear policy ownership becomes difficult to operate. For that reason, Sentra Gateway models routes, permissions, API keys, rate limits, IP rules, and audit event types as first-class concepts.

Observability is treated as part of the architecture instead of an afterthought. A security gateway must explain its decisions, reveal latency patterns, expose upstream failures, and support investigation during incidents. Metrics and logs are therefore aligned with request identifiers and route identifiers.

- Missing User-Agent.
- High request velocity.
- Repeated invalid credentials.
- Many 404s.

6.8 Secure Error Handling

Error responses must help legitimate clients without leaking details to attackers. In Sentra Gateway, this topic is important because the gateway is the first component that receives untrusted traffic. A weak design at this layer can expose every internal service, while a strong design can centralize control and reduce repeated security code across the system.

The design decision for this section is to make behavior explicit, measurable, and testable. Every important action should either produce a route decision, an audit event, a metric, or a safe error response. This gives developers and evaluators evidence that the system is behaving according to the documented requirements.

A production-style gateway must balance strict security with usability. If rules are too loose, the system becomes unsafe. If rules are too strict or poorly documented, legitimate clients will fail unexpectedly. Sentra Gateway therefore uses route-level policies, subject-specific limits, and documented response codes.

The module should be implemented through small services rather than one large filter. The filter extracts request data and delegates to services; the services perform validation, policy checks, logging, and decision making. This structure allows unit testing and reduces the chance of hidden coupling.

The design favors explicit policy definitions over hidden behavior. A gateway that silently permits or denies traffic without clear policy ownership becomes difficult to operate. For that reason, Sentra Gateway models routes, permissions, API keys, rate limits, IP rules, and audit event types as first-class concepts.

Observability is treated as part of the architecture instead of an afterthought. A security gateway must explain its decisions, reveal latency patterns, expose upstream failures, and support investigation during incidents. Metrics and logs are therefore aligned with request identifiers and route identifiers.

- Use consistent codes.
- Avoid detailed token failure leakage.
- Log internal reason codes.
- Return safe messages.

6.9 Secret Management

Secrets must be externalized, redacted, hashed, rotated, and never committed. In Sentra Gateway, this topic is important because the gateway is the first component that receives untrusted traffic. A weak design at this layer can expose every internal service, while a strong design can centralize control and reduce repeated security code across the system.

The design decision for this section is to make behavior explicit, measurable, and testable. Every important action should either produce a route decision, an audit event, a metric, or a safe error response. This gives developers and evaluators evidence that the system is behaving according to the documented requirements.

A production-style gateway must balance strict security with usability. If rules are too loose, the system becomes unsafe. If rules are too strict or poorly documented, legitimate clients will fail unexpectedly. Sentra Gateway therefore uses route-level policies, subject-specific limits, and documented response codes.

The module should be implemented through small services rather than one large filter. The filter extracts request data and delegates to services; the services perform validation, policy checks, logging, and decision making. This structure allows unit testing and reduces the chance of hidden coupling.

The design favors explicit policy definitions over hidden behavior. A gateway that silently permits or denies traffic without clear policy ownership becomes difficult to operate. For that reason, Sentra Gateway models routes, permissions, API keys, rate limits, IP rules, and audit event types as first-class concepts.

Observability is treated as part of the architecture instead of an afterthought. A security gateway must explain its decisions, reveal latency patterns, expose upstream failures, and support investigation during incidents. Metrics and logs are therefore aligned with request identifiers and route identifiers.

- No secrets in Git.
- No secrets in logs.
- Use env variables or secret manager.
- Rotate keys.

6.10 Threat Model Summary

Threat	Scenario	Mitigation
Token forgery	Attacker sends fake JWT.	Verify signature, issuer, audience, expiration, and key identifier.
Expired token reuse	Old access token is reused.	Reject expired tokens and use short lifetime.
API key leakage	Attacker obtains key.	Hash keys, rotate, disable, scope, and rate-limit.
Replay attack	Captured signed request sent again.	Timestamp + nonce with Redis TTL.
Header spoofing	Client sends fake X-User-Id.	Strip external trusted headers and regenerate after validation.
Route misconfiguration	Sensitive service exposed.	Protected admin APIs, audit changes, safe defaults.
Sensitive log leakage	Tokens or keys appear in logs.	Redaction and safe structured fields.
Cascading failure	Downstream failure harms whole system.	Timeouts, retries, circuit breakers, fallbacks.

Table 9. Threat model summary

Chapter 7: Database and Data Model Design

7.1 Persistence Strategy

PostgreSQL stores durable policy and audit data, while Redis stores fast-changing security and traffic-control state. In Sentra Gateway, this topic is important because the gateway is the first component that receives untrusted traffic. A weak design at this layer can expose every internal service, while a strong design can centralize control and reduce repeated security code across the system.

The design decision for this section is to make behavior explicit, measurable, and testable. Every important action should either produce a route decision, an audit event, a metric, or a safe error response. This gives developers and evaluators evidence that the system is behaving according to the documented requirements.

A production-style gateway must balance strict security with usability. If rules are too loose, the system becomes unsafe. If rules are too strict or poorly documented, legitimate clients will fail unexpectedly. Sentra Gateway therefore uses route-level policies, subject-specific limits, and documented response codes.

The module should be implemented through small services rather than one large filter. The filter extracts request data and delegates to services; the services perform validation, policy checks, logging, and decision making. This structure allows unit testing and reduces the chance of hidden coupling.

The design favors explicit policy definitions over hidden behavior. A gateway that silently permits or denies traffic without clear policy ownership becomes difficult to operate. For that reason, Sentra Gateway models routes, permissions, API keys, rate limits, IP rules, and audit event types as first-class concepts.

Observability is treated as part of the architecture instead of an afterthought. A security gateway must explain its decisions, reveal latency patterns, expose upstream failures, and support investigation during incidents. Metrics and logs are therefore aligned with request identifiers and route identifiers.

- Define the policy in a persistent or external configuration model.
- Expose the decision through audit events and metrics.
- Add positive and negative tests for the behavior.
- Document the expected error response and operational impact.

```
GatewayRoute 1---* RoutePermission
GatewayRoute 1---* RateLimitPolicy
ApiClient    1---* ApiKey
ApiKey       *---* ApiScope
GatewayRoute 1---* AuditEvent
IpRule       *---1 GatewayRoute (optional)
RiskRule     *---1 GatewayRoute (optional)
```

Figure 4. Logical database relationship model

7.2 Database Table Catalog

Table	Purpose
gateway_routes	Route definitions and target service URIs.
route_permissions	Role and scope requirements for routes.
api_clients	Machine-to-machine client records.
api_keys	Hashed API key records and lifecycle metadata.
api_scopes	Named scopes attached to keys and routes.
rate_limit_policies	Subject-based limit configuration.
ip_rules	Allow, block, CIDR, temporary, and route-specific IP rules.
risk_rules	Bot detection and suspicious behavior rules.
audit_events	Structured gateway decisions and security events.
admin_action_logs	Administrative changes to routes and policies.

Table 10. Database table catalog

7.3 gateway_routes Table

Route records define how paths map to downstream services and what security policies apply. In Sentra Gateway, this topic is important because the gateway is the first component that receives untrusted traffic. A weak design at this layer can expose every internal service, while a strong design can centralize control and reduce repeated security code across the system.

The design decision for this section is to make behavior explicit, measurable, and testable. Every important action should either produce a route decision, an audit event, a metric, or a safe error response. This gives developers and evaluators evidence that the system is behaving according to the documented requirements.

A production-style gateway must balance strict security with usability. If rules are too loose, the system becomes unsafe. If rules are too strict or poorly documented, legitimate clients will fail unexpectedly. Sentra Gateway therefore uses route-level policies, subject-specific limits, and documented response codes.

The module should be implemented through small services rather than one large filter. The filter extracts request data and delegates to services; the services perform validation, policy checks, logging, and decision making. This structure allows unit testing and reduces the chance of hidden coupling.

The design favors explicit policy definitions over hidden behavior. A gateway that silently permits or denies traffic without clear policy ownership becomes difficult to operate. For that reason, Sentra Gateway models routes, permissions, API keys, rate limits, IP rules, and audit event types as first-class concepts.

Observability is treated as part of the architecture instead of an afterthought. A security gateway must explain its decisions, reveal latency patterns, expose upstream failures, and support investigation during incidents. Metrics and logs are therefore aligned with request identifiers and route identifiers.

- Define the policy in a persistent or external configuration model.
- Expose the decision through audit events and metrics.
- Add positive and negative tests for the behavior.
- Document the expected error response and operational impact.

7.4 api_clients and api_keys Tables

API client tables control machine-to-machine access and key lifecycle management. In Sentra Gateway, this topic is important because the gateway is the first component that receives untrusted traffic. A weak design at this layer can expose every internal service, while a strong design can centralize control and reduce repeated security code across the system.

The design decision for this section is to make behavior explicit, measurable, and testable. Every important action should either produce a route decision, an audit event, a metric, or a safe error response. This gives developers and evaluators evidence that the system is behaving according to the documented requirements.

A production-style gateway must balance strict security with usability. If rules are too loose, the system becomes unsafe. If rules are too strict or poorly documented, legitimate clients will fail unexpectedly. Sentra Gateway therefore uses route-level policies, subject-specific limits, and documented response codes.

The module should be implemented through small services rather than one large filter. The filter extracts request data and delegates to services; the services perform validation, policy checks, logging, and decision making. This structure allows unit testing and reduces the chance of hidden coupling.

The design favors explicit policy definitions over hidden behavior. A gateway that silently permits or denies traffic without clear policy ownership becomes difficult to operate. For that reason, Sentra Gateway models routes, permissions, API keys, rate limits, IP rules, and audit event types as first-class concepts.

Observability is treated as part of the architecture instead of an afterthought. A security gateway must explain its decisions, reveal latency patterns, expose upstream failures, and support investigation during incidents. Metrics and logs are therefore aligned with request identifiers and route identifiers.

- Define the policy in a persistent or external configuration model.
- Expose the decision through audit events and metrics.
- Add positive and negative tests for the behavior.
- Document the expected error response and operational impact.

7.5 rate_limit_policies Table

Rate limit policies describe who is limited, where the limit applies, and how tokens refill. In Sentra Gateway, this topic is important because the gateway is the first component that receives untrusted traffic. A weak design at this layer can expose every internal service, while a strong design can centralize control and reduce repeated security code across the system.

The design decision for this section is to make behavior explicit, measurable, and testable. Every important action should either produce a route decision, an audit event, a metric, or a safe error response. This gives developers and evaluators evidence that the system is behaving according to the documented requirements.

A production-style gateway must balance strict security with usability. If rules are too loose, the system becomes unsafe. If rules are too strict or poorly documented, legitimate clients will fail unexpectedly. Sentra Gateway therefore uses route-level policies, subject-specific limits, and documented response codes.

The module should be implemented through small services rather than one large filter. The filter extracts request data and delegates to services; the services perform validation, policy checks, logging, and decision making. This structure allows unit testing and reduces the chance of hidden coupling.

The design favors explicit policy definitions over hidden behavior. A gateway that silently permits or denies traffic without clear policy ownership becomes difficult to operate. For that reason, Sentra Gateway models routes, permissions, API keys, rate limits, IP rules, and audit event types as first-class concepts.

Observability is treated as part of the architecture instead of an afterthought. A security gateway must explain its decisions, reveal latency patterns, expose upstream failures, and support investigation during incidents. Metrics and logs are therefore aligned with request identifiers and route identifiers.

- Define the policy in a persistent or external configuration model.
- Expose the decision through audit events and metrics.
- Add positive and negative tests for the behavior.
- Document the expected error response and operational impact.

7.6 audit_events Table

Audit events preserve security and routing decisions for investigation and evaluation. In Sentra Gateway, this topic is important because the gateway is the first component that receives untrusted traffic. A weak design at this layer can expose every internal service, while a strong design can centralize control and reduce repeated security code across the system.

The design decision for this section is to make behavior explicit, measurable, and testable. Every important action should either produce a route decision, an audit event, a metric, or a safe error response. This gives developers and evaluators evidence that the system is behaving according to the documented requirements.

A production-style gateway must balance strict security with usability. If rules are too loose, the system becomes unsafe. If rules are too strict or poorly documented, legitimate clients will fail unexpectedly. Sentra Gateway therefore uses route-level policies, subject-specific limits, and documented response codes.

The module should be implemented through small services rather than one large filter. The filter extracts request data and delegates to services; the services perform validation, policy checks, logging, and decision making. This structure allows unit testing and reduces the chance of hidden coupling.

The design favors explicit policy definitions over hidden behavior. A gateway that silently permits or denies traffic without clear policy ownership becomes difficult to operate. For that reason, Sentra Gateway models routes, permissions, API keys, rate limits, IP rules, and audit event types as first-class concepts.

Observability is treated as part of the architecture instead of an afterthought. A security gateway must explain its decisions, reveal latency patterns, expose upstream failures, and support investigation during incidents. Metrics and logs are therefore aligned with request identifiers and route identifiers.

- Define the policy in a persistent or external configuration model.
- Expose the decision through audit events and metrics.
- Add positive and negative tests for the behavior.
- Document the expected error response and operational impact.

7.7 Flyway Migration Strategy

Database schema changes should be versioned, reviewable, and reproducible. In Sentra Gateway, this topic is important because the gateway is the first component that receives untrusted traffic. A weak design at this layer can expose every internal service, while a strong design can centralize control and reduce repeated security code across the system.

The design decision for this section is to make behavior explicit, measurable, and testable. Every important action should either produce a route decision, an audit event, a metric, or a safe error response. This gives developers and evaluators evidence that the system is behaving according to the documented requirements.

A production-style gateway must balance strict security with usability. If rules are too loose, the system becomes unsafe. If rules are too strict or poorly documented, legitimate clients will fail unexpectedly. Sentra Gateway therefore uses route-level policies, subject-specific limits, and documented response codes.

The module should be implemented through small services rather than one large filter. The filter extracts request data and delegates to services; the services perform validation, policy checks, logging, and decision making. This structure allows unit testing and reduces the chance of hidden coupling.

The design favors explicit policy definitions over hidden behavior. A gateway that silently permits or denies traffic without clear policy ownership becomes difficult to operate. For that reason, Sentra Gateway models routes, permissions, API keys, rate limits, IP rules, and audit event types as first-class concepts.

Observability is treated as part of the architecture instead of an afterthought. A security gateway must explain its decisions, reveal latency patterns, expose upstream failures, and support investigation during incidents. Metrics and logs are therefore aligned with request identifiers and route identifiers.

- Define the policy in a persistent or external configuration model.
- Expose the decision through audit events and metrics.
- Add positive and negative tests for the behavior.
- Document the expected error response and operational impact.

7.8 Indexing Strategy

Indexes improve route lookups, key validation, audit search, and dashboard queries. In Sentra Gateway, this topic is important because the gateway is the first component that receives untrusted traffic. A weak design at this layer can expose every internal service, while a strong design can centralize control and reduce repeated security code across the system.

The design decision for this section is to make behavior explicit, measurable, and testable. Every important action should either produce a route decision, an audit event, a metric, or a safe error response. This gives developers and evaluators evidence that the system is behaving according to the documented requirements.

A production-style gateway must balance strict security with usability. If rules are too loose, the system becomes unsafe. If rules are too strict or poorly documented, legitimate clients will fail unexpectedly. Sentra Gateway therefore uses route-level policies, subject-specific limits, and documented response codes.

The module should be implemented through small services rather than one large filter. The filter extracts request data and delegates to services; the services perform validation, policy checks, logging, and decision making. This structure allows unit testing and reduces the chance of hidden coupling.

The design favors explicit policy definitions over hidden behavior. A gateway that silently permits or denies traffic without clear policy ownership becomes difficult to operate. For that reason, Sentra Gateway models routes, permissions, API keys, rate limits, IP rules, and audit event types as first-class concepts.

Observability is treated as part of the architecture instead of an afterthought. A security gateway must explain its decisions, reveal latency patterns, expose upstream failures, and support investigation during incidents. Metrics and logs are therefore aligned with request identifiers and route identifiers.

- Define the policy in a persistent or external configuration model.
- Expose the decision through audit events and metrics.
- Add positive and negative tests for the behavior.
- Document the expected error response and operational impact.

7.9 Retention Strategy

Audit and operational data should have retention rules to balance evidence, cost, and privacy. In Sentra Gateway, this topic is important because the gateway is the first component that receives untrusted traffic. A weak design at this layer can expose every internal service, while a strong design can centralize control and reduce repeated security code across the system.

The design decision for this section is to make behavior explicit, measurable, and testable. Every important action should either produce a route decision, an audit event, a metric, or a safe error response. This gives developers and evaluators evidence that the system is behaving according to the documented requirements.

A production-style gateway must balance strict security with usability. If rules are too loose, the system becomes unsafe. If rules are too strict or poorly documented, legitimate clients will fail unexpectedly. Sentra Gateway therefore uses route-level policies, subject-specific limits, and documented response codes.

The module should be implemented through small services rather than one large filter. The filter extracts request data and delegates to services; the services perform validation, policy checks, logging, and decision making. This structure allows unit testing and reduces the chance of hidden coupling.

The design favors explicit policy definitions over hidden behavior. A gateway that silently permits or denies traffic without clear policy ownership becomes difficult to operate. For that reason, Sentra Gateway models routes, permissions, API keys, rate limits, IP rules, and audit event types as first-class concepts.

Observability is treated as part of the architecture instead of an afterthought. A security gateway must explain its decisions, reveal latency patterns, expose upstream failures, and support investigation during incidents. Metrics and logs are therefore aligned with request identifiers and route identifiers.

- Define the policy in a persistent or external configuration model.
- Expose the decision through audit events and metrics.
- Add positive and negative tests for the behavior.
- Document the expected error response and operational impact.

Chapter 8: API Design

8.1 API Design Principles

Admin APIs use clear resource-based paths under `/api/v1` and return consistent error shapes. In Sentra Gateway, this topic is important because the gateway is the first component that receives untrusted traffic. A weak design at this layer can expose every internal service, while a strong design can centralize control and reduce repeated security code across the system.

The design decision for this section is to make behavior explicit, measurable, and testable. Every important action should either produce a route decision, an audit event, a metric, or a safe error response. This gives developers and evaluators evidence that the system is behaving according to the documented requirements.

A production-style gateway must balance strict security with usability. If rules are too loose, the system becomes unsafe. If rules are too strict or poorly documented, legitimate clients will fail unexpectedly. Sentra Gateway therefore uses route-level policies, subject-specific limits, and documented response codes.

The module should be implemented through small services rather than one large filter. The filter extracts request data and delegates to services; the services perform validation, policy checks, logging, and decision making. This structure allows unit testing and reduces the chance of hidden coupling.

The design favors explicit policy definitions over hidden behavior. A gateway that silently permits or denies traffic without clear policy ownership becomes difficult to operate. For that reason, Sentra Gateway models routes, permissions, API keys, rate limits, IP rules, and audit event types as first-class concepts.

Observability is treated as part of the architecture instead of an afterthought. A security gateway must explain its decisions, reveal latency patterns, expose upstream failures, and support investigation during incidents. Metrics and logs are therefore aligned with request identifiers and route identifiers.

- Define the policy in a persistent or external configuration model.
- Expose the decision through audit events and metrics.
- Add positive and negative tests for the behavior.
- Document the expected error response and operational impact.

8.2 Route Categories

Public, user, partner, admin, and internal routes have different security needs. In Sentra Gateway, this topic is important because the gateway is the first component that receives untrusted traffic. A weak design at this layer can expose every internal service, while a strong design can centralize control and reduce repeated security code across the system.

The design decision for this section is to make behavior explicit, measurable, and testable. Every important action should either produce a route decision, an audit event, a metric, or a safe error response. This gives developers and evaluators evidence that the system is behaving according to the documented requirements.

A production-style gateway must balance strict security with usability. If rules are too loose, the system becomes unsafe. If rules are too strict or poorly documented, legitimate clients will fail unexpectedly. Sentra Gateway therefore uses route-level policies, subject-specific limits, and documented response codes.

The module should be implemented through small services rather than one large filter. The filter extracts request data and delegates to services; the services perform validation, policy checks, logging, and decision making. This structure allows unit testing and reduces the chance of hidden coupling.

The design favors explicit policy definitions over hidden behavior. A gateway that silently permits or denies traffic without clear policy ownership becomes difficult to operate. For that reason, Sentra Gateway models routes, permissions, API keys, rate limits, IP rules, and audit event types as first-class concepts.

Observability is treated as part of the architecture instead of an afterthought. A security gateway must explain its decisions, reveal latency patterns, expose upstream failures, and support investigation during incidents. Metrics and logs are therefore aligned with request identifiers and route identifiers.

- Define the policy in a persistent or external configuration model.
- Expose the decision through audit events and metrics.
- Add positive and negative tests for the behavior.
- Document the expected error response and operational impact.

8.3 Route Management APIs

Administrators create, update, enable, disable, and delete route definitions. In Sentra Gateway, this topic is important because the gateway is the first component that receives untrusted traffic. A weak design at this layer can expose every internal service, while a strong design can centralize control and reduce repeated security code across the system.

The design decision for this section is to make behavior explicit, measurable, and testable. Every important action should either produce a route decision, an audit event, a metric, or a safe error response. This gives developers and evaluators evidence that the system is behaving according to the documented requirements.

A production-style gateway must balance strict security with usability. If rules are too loose, the system becomes unsafe. If rules are too strict or poorly documented, legitimate clients will fail unexpectedly. Sentra Gateway therefore uses route-level policies, subject-specific limits, and documented response codes.

The module should be implemented through small services rather than one large filter. The filter extracts request data and delegates to services; the services perform validation, policy checks, logging, and decision making. This structure allows unit testing and reduces the chance of hidden coupling.

The design favors explicit policy definitions over hidden behavior. A gateway that silently permits or denies traffic without clear policy ownership becomes difficult to operate. For that reason, Sentra Gateway models routes, permissions, API keys, rate limits, IP rules, and audit event types as first-class concepts.

Observability is treated as part of the architecture instead of an afterthought. A security gateway must explain its decisions, reveal latency patterns, expose upstream failures, and support investigation during incidents. Metrics and logs are therefore aligned with request identifiers and route identifiers.

- Define the policy in a persistent or external configuration model.
- Expose the decision through audit events and metrics.
- Add positive and negative tests for the behavior.
- Document the expected error response and operational impact.

8.4 API Client APIs

Administrators create clients, generate keys, rotate keys, disable keys, and inspect usage. In Sentra Gateway, this topic is important because the gateway is the first component that receives untrusted traffic. A weak design at this layer can expose every internal service, while a strong design can centralize control and reduce repeated security code across the system.

The design decision for this section is to make behavior explicit, measurable, and testable. Every important action should either produce a route decision, an audit event, a metric, or a safe error response. This gives developers and evaluators evidence that the system is behaving according to the documented requirements.

A production-style gateway must balance strict security with usability. If rules are too loose, the system becomes unsafe. If rules are too strict or poorly documented, legitimate clients will fail unexpectedly. Sentra Gateway therefore uses route-level policies, subject-specific limits, and documented response codes.

The module should be implemented through small services rather than one large filter. The filter extracts request data and delegates to services; the services perform validation, policy checks, logging, and decision making. This structure allows unit testing and reduces the chance of hidden coupling.

The design favors explicit policy definitions over hidden behavior. A gateway that silently permits or denies traffic without clear policy ownership becomes difficult to operate. For that reason, Sentra Gateway models routes, permissions, API keys, rate limits, IP rules, and audit event types as first-class concepts.

Observability is treated as part of the architecture instead of an afterthought. A security gateway must explain its decisions, reveal latency patterns, expose upstream failures, and support investigation during incidents. Metrics and logs are therefore aligned with request identifiers and route identifiers.

- Define the policy in a persistent or external configuration model.
- Expose the decision through audit events and metrics.
- Add positive and negative tests for the behavior.
- Document the expected error response and operational impact.

8.5 Security Policy APIs

Administrators manage rate limits, IP rules, risk rules, route permissions, and signing requirements. In Sentra Gateway, this topic is important because the gateway is the first component that receives untrusted traffic. A weak design at this

layer can expose every internal service, while a strong design can centralize control and reduce repeated security code across the system.

The design decision for this section is to make behavior explicit, measurable, and testable. Every important action should either produce a route decision, an audit event, a metric, or a safe error response. This gives developers and evaluators evidence that the system is behaving according to the documented requirements.

A production-style gateway must balance strict security with usability. If rules are too loose, the system becomes unsafe. If rules are too strict or poorly documented, legitimate clients will fail unexpectedly. Sentra Gateway therefore uses route-level policies, subject-specific limits, and documented response codes.

The module should be implemented through small services rather than one large filter. The filter extracts request data and delegates to services; the services perform validation, policy checks, logging, and decision making. This structure allows unit testing and reduces the chance of hidden coupling.

The design favors explicit policy definitions over hidden behavior. A gateway that silently permits or denies traffic without clear policy ownership becomes difficult to operate. For that reason, Sentra Gateway models routes, permissions, API keys, rate limits, IP rules, and audit event types as first-class concepts.

Observability is treated as part of the architecture instead of an afterthought. A security gateway must explain its decisions, reveal latency patterns, expose upstream failures, and support investigation during incidents. Metrics and logs are therefore aligned with request identifiers and route identifiers.

- Define the policy in a persistent or external configuration model.
- Expose the decision through audit events and metrics.
- Add positive and negative tests for the behavior.
- Document the expected error response and operational impact.

8.6 Audit APIs

Audit APIs allow searching by time, route, event type, decision, status, subject, and request ID. In Sentra Gateway, this topic is important because the gateway is the first component that receives untrusted traffic. A weak design at this layer can expose every internal service, while a strong design can centralize control and reduce repeated security code across the system.

The design decision for this section is to make behavior explicit, measurable, and testable. Every important action should either produce a route decision, an audit event, a metric, or a safe error response. This gives developers and evaluators evidence that the system is behaving according to the documented requirements.

A production-style gateway must balance strict security with usability. If rules are too loose, the system becomes unsafe. If rules are too strict or poorly documented, legitimate clients will fail unexpectedly. Sentra Gateway therefore uses route-level policies, subject-specific limits, and documented response codes.

The module should be implemented through small services rather than one large filter. The filter extracts request data and delegates to services; the services perform validation, policy checks, logging, and decision making. This structure allows unit testing and reduces the chance of hidden coupling.

The design favors explicit policy definitions over hidden behavior. A gateway that silently permits or denies traffic without clear policy ownership becomes difficult to operate. For that reason, Sentra Gateway models routes, permissions, API keys, rate limits, IP rules, and audit event types as first-class concepts.

Observability is treated as part of the architecture instead of an afterthought. A security gateway must explain its decisions, reveal latency patterns, expose upstream failures, and support investigation during incidents. Metrics and logs are therefore aligned with request identifiers and route identifiers.

- Define the policy in a persistent or external configuration model.
- Expose the decision through audit events and metrics.
- Add positive and negative tests for the behavior.
- Document the expected error response and operational impact.

8.7 Metrics APIs

Operational metrics are exposed through Actuator and Prometheus endpoints. In Sentra Gateway, this topic is important because the gateway is the first component that receives untrusted traffic. A weak design at this layer can expose every internal service, while a strong design can centralize control and reduce repeated security code across the system.

The design decision for this section is to make behavior explicit, measurable, and testable. Every important action should either produce a route decision, an audit event, a metric, or a safe error response. This gives developers and evaluators evidence that the system is behaving according to the documented requirements.

A production-style gateway must balance strict security with usability. If rules are too loose, the system becomes unsafe. If rules are too strict or poorly documented, legitimate clients will fail unexpectedly. Sentra Gateway therefore uses route-level policies, subject-specific limits, and documented response codes.

The module should be implemented through small services rather than one large filter. The filter extracts request data and delegates to services; the services perform validation, policy checks, logging, and decision making. This structure allows unit testing and reduces the chance of hidden coupling.

The design favors explicit policy definitions over hidden behavior. A gateway that silently permits or denies traffic without clear policy ownership becomes difficult to operate. For that reason, Sentra Gateway models routes, permissions, API keys, rate limits, IP rules, and audit event types as first-class concepts.

Observability is treated as part of the architecture instead of an afterthought. A security gateway must explain its decisions, reveal latency patterns, expose upstream failures, and support investigation during incidents. Metrics and logs are therefore aligned with request identifiers and route identifiers.

- Define the policy in a persistent or external configuration model.
- Expose the decision through audit events and metrics.
- Add positive and negative tests for the behavior.
- Document the expected error response and operational impact.

8.8 Error Response Contract

All errors should contain timestamp, request ID, status, code, message, path, and route ID when available. In Sentra Gateway, this topic is important because the gateway is the first component that receives untrusted traffic. A weak design at this layer can expose every internal service, while a strong design can centralize control and reduce repeated security code across the system.

The design decision for this section is to make behavior explicit, measurable, and testable. Every important action should either produce a route decision, an audit event, a metric, or a safe error response. This gives developers and evaluators evidence that the system is behaving according to the documented requirements.

A production-style gateway must balance strict security with usability. If rules are too loose, the system becomes unsafe. If rules are too strict or poorly documented, legitimate clients will fail unexpectedly. Sentra Gateway therefore uses route-level policies, subject-specific limits, and documented response codes.

The module should be implemented through small services rather than one large filter. The filter extracts request data and delegates to services; the services perform validation, policy checks, logging, and decision making. This structure allows unit testing and reduces the chance of hidden coupling.

The design favors explicit policy definitions over hidden behavior. A gateway that silently permits or denies traffic without clear policy ownership becomes difficult to operate. For that reason, Sentra Gateway models routes, permissions, API keys, rate limits, IP rules, and audit event types as first-class concepts.

Observability is treated as part of the architecture instead of an afterthought. A security gateway must explain its decisions, reveal latency patterns, expose upstream failures, and support investigation during incidents. Metrics and logs are therefore aligned with request identifiers and route identifiers.

- Define the policy in a persistent or external configuration model.
- Expose the decision through audit events and metrics.
- Add positive and negative tests for the behavior.
- Document the expected error response and operational impact.

8.9 API Versioning

The first version uses `/api/v1` and future changes should preserve compatibility or introduce new versions. In Sentra Gateway, this topic is important because the gateway is the first component that receives untrusted traffic. A weak design at this layer can expose every internal service, while a strong design can centralize control and reduce repeated security code across the system.

The design decision for this section is to make behavior explicit, measurable, and testable. Every important action should either produce a route decision, an audit event, a metric, or a safe error response. This gives developers and evaluators evidence that the system is behaving according to the documented requirements.

A production-style gateway must balance strict security with usability. If rules are too loose, the system becomes unsafe. If rules are too strict or poorly documented, legitimate clients will fail unexpectedly. Sentra Gateway therefore uses route-level policies, subject-specific limits, and documented response codes.

The module should be implemented through small services rather than one large filter. The filter extracts request data and delegates to services; the services perform validation, policy checks, logging, and decision making. This structure allows unit testing and reduces the chance of hidden coupling.

The design favors explicit policy definitions over hidden behavior. A gateway that silently permits or denies traffic without clear policy ownership becomes difficult to operate. For that reason, Sentra Gateway models routes, permissions, API keys, rate limits, IP rules, and audit event types as first-class concepts.

Observability is treated as part of the architecture instead of an afterthought. A security gateway must explain its decisions, reveal latency patterns, expose upstream failures, and support investigation during incidents. Metrics and logs are therefore aligned with request identifiers and route identifiers.

- Define the policy in a persistent or external configuration model.
- Expose the decision through audit events and metrics.
- Add positive and negative tests for the behavior.
- Document the expected error response and operational impact.

8.10 Endpoint Catalog

Method	Path	Purpose
GET	/api/v1/admin/routes	List routes.
POST	/api/v1/admin/routes	Create route.
PATCH	/api/v1/admin/routes/{id}	Update route.
POST	/api/v1/admin/api-clients	Create API client.
POST	/api/v1/admin/api-clients/{id}/keys	Create API key.
POST	/api/v1/admin/api-keys/{id}/rotate	Rotate API key.
GET	/api/v1/admin/rate-limits	List rate policies.
POST	/api/v1/admin/ip-rules	Create IP rule.
POST	/api/v1/admin/risk-rules	Create risk rule.
GET	/api/v1/admin/audit-events	Search audit events.
GET	/actuator/health	Health endpoint.
GET	/actuator/prometheus	Prometheus metrics endpoint.

Table 11. Admin API catalog

Chapter 9: Implementation Strategy

```

sentra-gateway/
  gateway-service/
    src/main/java/com/omar/sentra/gateway/
      admin/ audit/ authorization/ common/ config/ metrics/
      ratelimit/ resilience/ routing/
      security/apikey/ security/ip/ security/jwt/ security/risk/ security/signing/
    src/main/resources/db/migration/
  mock-user-service/
  mock-order-service/
  mock-payment-service/
  mock-notification-service/
  observability/
  performance/k6/
  docs/
  docker-compose.yml

```

Figure 5. Suggested project structure

9.1 Implementation Phases

The project should be implemented in phases so that the core gateway works before advanced features are added. In Sentra Gateway, this topic is important because the gateway is the first component that receives untrusted traffic. A weak design at this layer can expose every internal service, while a strong design can centralize control and reduce repeated security code across the system.

The design decision for this section is to make behavior explicit, measurable, and testable. Every important action should either produce a route decision, an audit event, a metric, or a safe error response. This gives developers and evaluators evidence that the system is behaving according to the documented requirements.

A production-style gateway must balance strict security with usability. If rules are too loose, the system becomes unsafe. If rules are too strict or poorly documented, legitimate clients will fail unexpectedly. Sentra Gateway therefore uses route-level policies, subject-specific limits, and documented response codes.

The module should be implemented through small services rather than one large filter. The filter extracts request data and delegates to services; the services perform validation, policy checks, logging, and decision making. This structure allows unit testing and reduces the chance of hidden coupling.

The design favors explicit policy definitions over hidden behavior. A gateway that silently permits or denies traffic without clear policy ownership becomes difficult to operate. For that reason, Sentra Gateway models routes, permissions, API keys, rate limits, IP rules, and audit event types as first-class concepts.

Observability is treated as part of the architecture instead of an afterthought. A security gateway must explain its decisions, reveal latency patterns, expose upstream failures, and support investigation during incidents. Metrics and logs are therefore aligned with request identifiers and route identifiers.

- Define the policy in a persistent or external configuration model.
- Expose the decision through audit events and metrics.
- Add positive and negative tests for the behavior.
- Document the expected error response and operational impact.

9.2 Configuration Profiles

Profiles separate local, test, performance, and production-like behavior. In Sentra Gateway, this topic is important because the gateway is the first component that receives untrusted traffic. A weak design at this layer can expose every internal service, while a strong design can centralize control and reduce repeated security code across the system.

The design decision for this section is to make behavior explicit, measurable, and testable. Every important action should either produce a route decision, an audit event, a metric, or a safe error response. This gives developers and evaluators evidence that the system is behaving according to the documented requirements.

A production-style gateway must balance strict security with usability. If rules are too loose, the system becomes unsafe. If rules are too strict or poorly documented, legitimate clients will fail unexpectedly. Sentra Gateway therefore uses route-level policies, subject-specific limits, and documented response codes.

The module should be implemented through small services rather than one large filter. The filter extracts request data and delegates to services; the services perform validation, policy checks, logging, and decision making. This structure allows unit testing and reduces the chance of hidden coupling.

The design favors explicit policy definitions over hidden behavior. A gateway that silently permits or denies traffic without clear policy ownership becomes difficult to operate. For that reason, Sentra Gateway models routes, permissions, API keys, rate limits, IP rules, and audit event types as first-class concepts.

Observability is treated as part of the architecture instead of an afterthought. A security gateway must explain its decisions, reveal latency patterns, expose upstream failures, and support investigation during incidents. Metrics and logs are therefore aligned with request identifiers and route identifiers.

- Define the policy in a persistent or external configuration model.
- Expose the decision through audit events and metrics.
- Add positive and negative tests for the behavior.
- Document the expected error response and operational impact.

9.3 Filter Implementation Pattern

Filters should be thin and delegate logic to services. In Sentra Gateway, this topic is important because the gateway is the first component that receives untrusted traffic. A weak design at this layer can expose every internal service, while a strong design can centralize control and reduce repeated security code across the system.

The design decision for this section is to make behavior explicit, measurable, and testable. Every important action should either produce a route decision, an audit event, a metric, or a safe error response. This gives developers and evaluators evidence that the system is behaving according to the documented requirements.

A production-style gateway must balance strict security with usability. If rules are too loose, the system becomes unsafe. If rules are too strict or poorly documented, legitimate clients will fail unexpectedly. Sentra Gateway therefore uses route-level policies, subject-specific limits, and documented response codes.

The module should be implemented through small services rather than one large filter. The filter extracts request data and delegates to services; the services perform validation, policy checks, logging, and decision making. This structure allows unit testing and reduces the chance of hidden coupling.

The design favors explicit policy definitions over hidden behavior. A gateway that silently permits or denies traffic without clear policy ownership becomes difficult to operate. For that reason, Sentra Gateway models routes, permissions, API keys, rate limits, IP rules, and audit event types as first-class concepts.

Observability is treated as part of the architecture instead of an afterthought. A security gateway must explain its decisions, reveal latency patterns, expose upstream failures, and support investigation during incidents. Metrics and logs are therefore aligned with request identifiers and route identifiers.

- Define the policy in a persistent or external configuration model.
- Expose the decision through audit events and metrics.
- Add positive and negative tests for the behavior.
- Document the expected error response and operational impact.

9.4 Service Components

Services encapsulate JWT validation, API key checks, rate limiting, signing, IP policy, risk scoring, audit, and metrics. In Sentra Gateway, this topic is important because the gateway is the first component that receives untrusted traffic. A weak design at this layer can expose every internal service, while a strong design can centralize control and reduce repeated security code across the system.

The design decision for this section is to make behavior explicit, measurable, and testable. Every important action should either produce a route decision, an audit event, a metric, or a safe error response. This gives developers and evaluators evidence that the system is behaving according to the documented requirements.

A production-style gateway must balance strict security with usability. If rules are too loose, the system becomes unsafe. If rules are too strict or poorly documented, legitimate clients will fail unexpectedly. Sentra Gateway therefore uses route-level policies, subject-specific limits, and documented response codes.

The module should be implemented through small services rather than one large filter. The filter extracts request data and delegates to services; the services perform validation, policy checks, logging, and decision making. This structure allows unit testing and reduces the chance of hidden coupling.

The design favors explicit policy definitions over hidden behavior. A gateway that silently permits or denies traffic without clear policy ownership becomes difficult to operate. For that reason, Sentra Gateway models routes, permissions, API keys, rate limits, IP rules, and audit event types as first-class concepts.

Observability is treated as part of the architecture instead of an afterthought. A security gateway must explain its decisions, reveal latency patterns, expose upstream failures, and support investigation during incidents. Metrics and logs are therefore aligned with request identifiers and route identifiers.

- Define the policy in a persistent or external configuration model.
- Expose the decision through audit events and metrics.
- Add positive and negative tests for the behavior.
- Document the expected error response and operational impact.

9.5 Repository Design

Repositories persist policy data and audit evidence. In Sentra Gateway, this topic is important because the gateway is the first component that receives untrusted traffic. A weak design at this layer can expose every internal service, while a strong design can centralize control and reduce repeated security code across the system.

The design decision for this section is to make behavior explicit, measurable, and testable. Every important action should either produce a route decision, an audit event, a metric, or a safe error response. This gives developers and evaluators evidence that the system is behaving according to the documented requirements.

A production-style gateway must balance strict security with usability. If rules are too loose, the system becomes unsafe. If rules are too strict or poorly documented, legitimate clients will fail unexpectedly. Sentra Gateway therefore uses route-level policies, subject-specific limits, and documented response codes.

The module should be implemented through small services rather than one large filter. The filter extracts request data and delegates to services; the services perform validation, policy checks, logging, and decision making. This structure allows unit testing and reduces the chance of hidden coupling.

The design favors explicit policy definitions over hidden behavior. A gateway that silently permits or denies traffic without clear policy ownership becomes difficult to operate. For that reason, Sentra Gateway models routes, permissions, API keys, rate limits, IP rules, and audit event types as first-class concepts.

Observability is treated as part of the architecture instead of an afterthought. A security gateway must explain its decisions, reveal latency patterns, expose upstream failures, and support investigation during incidents. Metrics and logs are therefore aligned with request identifiers and route identifiers.

- Define the policy in a persistent or external configuration model.
- Expose the decision through audit events and metrics.
- Add positive and negative tests for the behavior.
- Document the expected error response and operational impact.

9.6 Redis Key Design

Redis key naming must be predictable, collision-resistant, and compatible with TTL usage. In Sentra Gateway, this topic is important because the gateway is the first component that receives untrusted traffic. A weak design at this layer can expose every internal service, while a strong design can centralize control and reduce repeated security code across the system.

The design decision for this section is to make behavior explicit, measurable, and testable. Every important action should either produce a route decision, an audit event, a metric, or a safe error response. This gives developers and evaluators evidence that the system is behaving according to the documented requirements.

A production-style gateway must balance strict security with usability. If rules are too loose, the system becomes unsafe. If rules are too strict or poorly documented, legitimate clients will fail unexpectedly. Sentra Gateway therefore uses route-level policies, subject-specific limits, and documented response codes.

The module should be implemented through small services rather than one large filter. The filter extracts request data and delegates to services; the services perform validation, policy checks, logging, and decision making. This structure allows unit testing and reduces the chance of hidden coupling.

The design favors explicit policy definitions over hidden behavior. A gateway that silently permits or denies traffic without clear policy ownership becomes difficult to operate. For that reason, Sentra Gateway models routes, permissions, API keys, rate limits, IP rules, and audit event types as first-class concepts.

Observability is treated as part of the architecture instead of an afterthought. A security gateway must explain its decisions, reveal latency patterns, expose upstream failures, and support investigation during incidents. Metrics and logs are therefore aligned with request identifiers and route identifiers.

- Define the policy in a persistent or external configuration model.
- Expose the decision through audit events and metrics.
- Add positive and negative tests for the behavior.
- Document the expected error response and operational impact.

9.7 DTO and Validation Design

Admin API DTOs should use validation annotations and meaningful enum types. In Sentra Gateway, this topic is important because the gateway is the first component that receives untrusted traffic. A weak design at this layer can expose every internal service, while a strong design can centralize control and reduce repeated security code across the system.

The design decision for this section is to make behavior explicit, measurable, and testable. Every important action should either produce a route decision, an audit event, a metric, or a safe error response. This gives developers and evaluators evidence that the system is behaving according to the documented requirements.

A production-style gateway must balance strict security with usability. If rules are too loose, the system becomes unsafe. If rules are too strict or poorly documented, legitimate clients will fail unexpectedly. Sentra Gateway therefore uses route-level policies, subject-specific limits, and documented response codes.

The module should be implemented through small services rather than one large filter. The filter extracts request data and delegates to services; the services perform validation, policy checks, logging, and decision making. This structure allows unit testing and reduces the chance of hidden coupling.

The design favors explicit policy definitions over hidden behavior. A gateway that silently permits or denies traffic without clear policy ownership becomes difficult to operate. For that reason, Sentra Gateway models routes, permissions, API keys, rate limits, IP rules, and audit event types as first-class concepts.

Observability is treated as part of the architecture instead of an afterthought. A security gateway must explain its decisions, reveal latency patterns, expose upstream failures, and support investigation during incidents. Metrics and logs are therefore aligned with request identifiers and route identifiers.

- Define the policy in a persistent or external configuration model.
- Expose the decision through audit events and metrics.
- Add positive and negative tests for the behavior.
- Document the expected error response and operational impact.

9.8 Development Workflow

Development should follow tests, implementation, metrics, audit behavior, documentation, and performance checks. In Sentra Gateway, this topic is important because the gateway is the first component that receives untrusted traffic. A weak design at this layer can expose every internal service, while a strong design can centralize control and reduce repeated security code across the system.

The design decision for this section is to make behavior explicit, measurable, and testable. Every important action should either produce a route decision, an audit event, a metric, or a safe error response. This gives developers and evaluators evidence that the system is behaving according to the documented requirements.

A production-style gateway must balance strict security with usability. If rules are too loose, the system becomes unsafe. If rules are too strict or poorly documented, legitimate clients will fail unexpectedly. Sentra Gateway therefore uses route-level policies, subject-specific limits, and documented response codes.

The module should be implemented through small services rather than one large filter. The filter extracts request data and delegates to services; the services perform validation, policy checks, logging, and decision making. This structure allows unit testing and reduces the chance of hidden coupling.

The design favors explicit policy definitions over hidden behavior. A gateway that silently permits or denies traffic without clear policy ownership becomes difficult to operate. For that reason, Sentra Gateway models routes, permissions, API keys, rate limits, IP rules, and audit event types as first-class concepts.

Observability is treated as part of the architecture instead of an afterthought. A security gateway must explain its decisions, reveal latency patterns, expose upstream failures, and support investigation during incidents. Metrics and logs are therefore aligned with request identifiers and route identifiers.

- Define the policy in a persistent or external configuration model.
- Expose the decision through audit events and metrics.
- Add positive and negative tests for the behavior.
- Document the expected error response and operational impact.

9.9 Implementation Risks

Risks include filter order mistakes, logging secrets, policy cache staleness, route pattern errors, and retry storms. In Sentra Gateway, this topic is important because the gateway is the first component that receives untrusted traffic. A weak design at this layer can expose every internal service, while a strong design can centralize control and reduce repeated security code across the system.

The design decision for this section is to make behavior explicit, measurable, and testable. Every important action should either produce a route decision, an audit event, a metric, or a safe error response. This gives developers and evaluators evidence that the system is behaving according to the documented requirements.

A production-style gateway must balance strict security with usability. If rules are too loose, the system becomes unsafe. If rules are too strict or poorly documented, legitimate clients will fail unexpectedly. Sentra Gateway therefore uses route-level policies, subject-specific limits, and documented response codes.

The module should be implemented through small services rather than one large filter. The filter extracts request data and delegates to services; the services perform validation, policy checks, logging, and decision making. This structure allows unit testing and reduces the chance of hidden coupling.

The design favors explicit policy definitions over hidden behavior. A gateway that silently permits or denies traffic without clear policy ownership becomes difficult to operate. For that reason, Sentra Gateway models routes, permissions, API keys, rate limits, IP rules, and audit event types as first-class concepts.

Observability is treated as part of the architecture instead of an afterthought. A security gateway must explain its decisions, reveal latency patterns, expose upstream failures, and support investigation during incidents. Metrics and logs are therefore aligned with request identifiers and route identifiers.

- Define the policy in a persistent or external configuration model.
- Expose the decision through audit events and metrics.
- Add positive and negative tests for the behavior.
- Document the expected error response and operational impact.

Chapter 10: Observability and Reliability

10.1 Observability Objectives

Gateway observability must show what traffic was received, what decisions were made, and how upstream services behaved. In Sentra Gateway, this topic is important because the gateway is the first component that receives untrusted traffic. A weak design at this layer can expose every internal service, while a strong design can centralize control and reduce repeated security code across the system.

The design decision for this section is to make behavior explicit, measurable, and testable. Every important action should either produce a route decision, an audit event, a metric, or a safe error response. This gives developers and evaluators evidence that the system is behaving according to the documented requirements.

A production-style gateway must balance strict security with usability. If rules are too loose, the system becomes unsafe. If rules are too strict or poorly documented, legitimate clients will fail unexpectedly. Sentra Gateway therefore uses route-level policies, subject-specific limits, and documented response codes.

The module should be implemented through small services rather than one large filter. The filter extracts request data and delegates to services; the services perform validation, policy checks, logging, and decision making. This structure allows unit testing and reduces the chance of hidden coupling.

The design favors explicit policy definitions over hidden behavior. A gateway that silently permits or denies traffic without clear policy ownership becomes difficult to operate. For that reason, Sentra Gateway models routes, permissions, API keys, rate limits, IP rules, and audit event types as first-class concepts.

Observability is treated as part of the architecture instead of an afterthought. A security gateway must explain its decisions, reveal latency patterns, expose upstream failures, and support investigation during incidents. Metrics and logs are therefore aligned with request identifiers and route identifiers.

- Define the policy in a persistent or external configuration model.
- Expose the decision through audit events and metrics.
- Add positive and negative tests for the behavior.
- Document the expected error response and operational impact.

10.2 Metrics Design

Metrics should include traffic, latency, security failures, rate limits, retries, circuit breaker states, and upstream errors. In Sentra Gateway, this topic is important because the gateway is the first component that receives untrusted traffic. A weak design at this layer can expose every internal service, while a strong design can centralize control and reduce repeated security code across the system.

The design decision for this section is to make behavior explicit, measurable, and testable. Every important action should either produce a route decision, an audit event, a metric, or a safe error response. This gives developers and evaluators evidence that the system is behaving according to the documented requirements.

A production-style gateway must balance strict security with usability. If rules are too loose, the system becomes unsafe. If rules are too strict or poorly documented, legitimate clients will fail unexpectedly. Sentra Gateway therefore uses route-level policies, subject-specific limits, and documented response codes.

The module should be implemented through small services rather than one large filter. The filter extracts request data and delegates to services; the services perform validation, policy checks, logging, and decision making. This structure allows unit testing and reduces the chance of hidden coupling.

The design favors explicit policy definitions over hidden behavior. A gateway that silently permits or denies traffic without clear policy ownership becomes difficult to operate. For that reason, Sentra Gateway models routes, permissions, API keys, rate limits, IP rules, and audit event types as first-class concepts.

Observability is treated as part of the architecture instead of an afterthought. A security gateway must explain its decisions, reveal latency patterns, expose upstream failures, and support investigation during incidents. Metrics and logs are therefore aligned with request identifiers and route identifiers.

- Define the policy in a persistent or external configuration model.
- Expose the decision through audit events and metrics.
- Add positive and negative tests for the behavior.
- Document the expected error response and operational impact.

10.3 Prometheus and Grafana Pipeline

Sentra Gateway exposes metrics to Prometheus, and Grafana visualizes them through dashboards. In Sentra Gateway, this topic is important because the gateway is the first component that receives untrusted traffic. A weak design at this layer can expose every internal service, while a strong design can centralize control and reduce repeated security code across the system.

The design decision for this section is to make behavior explicit, measurable, and testable. Every important action should either produce a route decision, an audit event, a metric, or a safe error response. This gives developers and evaluators evidence that the system is behaving according to the documented requirements.

A production-style gateway must balance strict security with usability. If rules are too loose, the system becomes unsafe. If rules are too strict or poorly documented, legitimate clients will fail unexpectedly. Sentra Gateway therefore uses route-level policies, subject-specific limits, and documented response codes.

The module should be implemented through small services rather than one large filter. The filter extracts request data and delegates to services; the services perform validation, policy checks, logging, and decision making. This structure allows unit testing and reduces the chance of hidden coupling.

The design favors explicit policy definitions over hidden behavior. A gateway that silently permits or denies traffic without clear policy ownership becomes difficult to operate. For that reason, Sentra Gateway models routes, permissions, API keys, rate limits, IP rules, and audit event types as first-class concepts.

Observability is treated as part of the architecture instead of an afterthought. A security gateway must explain its decisions, reveal latency patterns, expose upstream failures, and support investigation during incidents. Metrics and logs are therefore aligned with request identifiers and route identifiers.

- Define the policy in a persistent or external configuration model.
- Expose the decision through audit events and metrics.
- Add positive and negative tests for the behavior.
- Document the expected error response and operational impact.

10.4 Structured Logging

Structured logs should use request IDs and safe fields without exposing secrets. In Sentra Gateway, this topic is important because the gateway is the first component that receives untrusted traffic. A weak design at this layer can expose every internal service, while a strong design can centralize control and reduce repeated security code across the system.

The design decision for this section is to make behavior explicit, measurable, and testable. Every important action should either produce a route decision, an audit event, a metric, or a safe error response. This gives developers and evaluators evidence that the system is behaving according to the documented requirements.

A production-style gateway must balance strict security with usability. If rules are too loose, the system becomes unsafe. If rules are too strict or poorly documented, legitimate clients will fail unexpectedly. Sentra Gateway therefore uses route-level policies, subject-specific limits, and documented response codes.

The module should be implemented through small services rather than one large filter. The filter extracts request data and delegates to services; the services perform validation, policy checks, logging, and decision making. This structure allows unit testing and reduces the chance of hidden coupling.

The design favors explicit policy definitions over hidden behavior. A gateway that silently permits or denies traffic without clear policy ownership becomes difficult to operate. For that reason, Sentra Gateway models routes, permissions, API keys, rate limits, IP rules, and audit event types as first-class concepts.

Observability is treated as part of the architecture instead of an afterthought. A security gateway must explain its decisions, reveal latency patterns, expose upstream failures, and support investigation during incidents. Metrics and logs are therefore aligned with request identifiers and route identifiers.

- Define the policy in a persistent or external configuration model.
- Expose the decision through audit events and metrics.
- Add positive and negative tests for the behavior.
- Document the expected error response and operational impact.

10.5 Audit Event Taxonomy

Audit events should use stable event names and reason codes. In Sentra Gateway, this topic is important because the gateway is the first component that receives untrusted traffic. A weak design at this layer can expose every internal service, while a strong design can centralize control and reduce repeated security code across the system.

The design decision for this section is to make behavior explicit, measurable, and testable. Every important action should either produce a route decision, an audit event, a metric, or a safe error response. This gives developers and evaluators evidence that the system is behaving according to the documented requirements.

A production-style gateway must balance strict security with usability. If rules are too loose, the system becomes unsafe. If rules are too strict or poorly documented, legitimate clients will fail unexpectedly. Sentra Gateway therefore uses route-level policies, subject-specific limits, and documented response codes.

The module should be implemented through small services rather than one large filter. The filter extracts request data and delegates to services; the services perform validation, policy checks, logging, and decision making. This structure allows unit testing and reduces the chance of hidden coupling.

The design favors explicit policy definitions over hidden behavior. A gateway that silently permits or denies traffic without clear policy ownership becomes difficult to operate. For that reason, Sentra Gateway models routes, permissions, API keys, rate limits, IP rules, and audit event types as first-class concepts.

Observability is treated as part of the architecture instead of an afterthought. A security gateway must explain its decisions, reveal latency patterns, expose upstream failures, and support investigation during incidents. Metrics and logs are therefore aligned with request identifiers and route identifiers.

- Define the policy in a persistent or external configuration model.
- Expose the decision through audit events and metrics.
- Add positive and negative tests for the behavior.
- Document the expected error response and operational impact.

10.6 Reliability Patterns

Timeouts, retries, circuit breakers, and fallbacks protect clients and the gateway from downstream failures. In Sentra Gateway, this topic is important because the gateway is the first component that receives untrusted traffic. A weak design at this layer can expose every internal service, while a strong design can centralize control and reduce repeated security code across the system.

The design decision for this section is to make behavior explicit, measurable, and testable. Every important action should either produce a route decision, an audit event, a metric, or a safe error response. This gives developers and evaluators evidence that the system is behaving according to the documented requirements.

A production-style gateway must balance strict security with usability. If rules are too loose, the system becomes unsafe. If rules are too strict or poorly documented, legitimate clients will fail unexpectedly. Sentra Gateway therefore uses route-level policies, subject-specific limits, and documented response codes.

The module should be implemented through small services rather than one large filter. The filter extracts request data and delegates to services; the services perform validation, policy checks, logging, and decision making. This structure allows unit testing and reduces the chance of hidden coupling.

The design favors explicit policy definitions over hidden behavior. A gateway that silently permits or denies traffic without clear policy ownership becomes difficult to operate. For that reason, Sentra Gateway models routes, permissions, API keys, rate limits, IP rules, and audit event types as first-class concepts.

Observability is treated as part of the architecture instead of an afterthought. A security gateway must explain its decisions, reveal latency patterns, expose upstream failures, and support investigation during incidents. Metrics and logs are therefore aligned with request identifiers and route identifiers.

- Define the policy in a persistent or external configuration model.
- Expose the decision through audit events and metrics.
- Add positive and negative tests for the behavior.
- Document the expected error response and operational impact.

10.7 Alerting Strategy

Alerts should detect high 5xx rate, high 429 rate, invalid token spikes, Redis outage, audit insert failure, and open circuit breakers. In Sentra Gateway, this topic is important because the gateway is the first component that receives untrusted

traffic. A weak design at this layer can expose every internal service, while a strong design can centralize control and reduce repeated security code across the system.

The design decision for this section is to make behavior explicit, measurable, and testable. Every important action should either produce a route decision, an audit event, a metric, or a safe error response. This gives developers and evaluators evidence that the system is behaving according to the documented requirements.

A production-style gateway must balance strict security with usability. If rules are too loose, the system becomes unsafe. If rules are too strict or poorly documented, legitimate clients will fail unexpectedly. Sentra Gateway therefore uses route-level policies, subject-specific limits, and documented response codes.

The module should be implemented through small services rather than one large filter. The filter extracts request data and delegates to services; the services perform validation, policy checks, logging, and decision making. This structure allows unit testing and reduces the chance of hidden coupling.

The design favors explicit policy definitions over hidden behavior. A gateway that silently permits or denies traffic without clear policy ownership becomes difficult to operate. For that reason, Sentra Gateway models routes, permissions, API keys, rate limits, IP rules, and audit event types as first-class concepts.

Observability is treated as part of the architecture instead of an afterthought. A security gateway must explain its decisions, reveal latency patterns, expose upstream failures, and support investigation during incidents. Metrics and logs are therefore aligned with request identifiers and route identifiers.

- Define the policy in a persistent or external configuration model.
- Expose the decision through audit events and metrics.
- Add positive and negative tests for the behavior.
- Document the expected error response and operational impact.

10.8 Operational Trade-offs

Security, performance, and availability trade-offs must be documented for each policy. In Sentra Gateway, this topic is important because the gateway is the first component that receives untrusted traffic. A weak design at this layer can expose every internal service, while a strong design can centralize control and reduce repeated security code across the system.

The design decision for this section is to make behavior explicit, measurable, and testable. Every important action should either produce a route decision, an audit event, a metric, or a safe error response. This gives developers and evaluators evidence that the system is behaving according to the documented requirements.

A production-style gateway must balance strict security with usability. If rules are too loose, the system becomes unsafe. If rules are too strict or poorly documented, legitimate clients will fail unexpectedly. Sentra Gateway therefore uses route-level policies, subject-specific limits, and documented response codes.

The module should be implemented through small services rather than one large filter. The filter extracts request data and delegates to services; the services perform validation, policy checks, logging, and decision making. This structure allows unit testing and reduces the chance of hidden coupling.

The design favors explicit policy definitions over hidden behavior. A gateway that silently permits or denies traffic without clear policy ownership becomes difficult to operate. For that reason, Sentra Gateway models routes, permissions, API keys, rate limits, IP rules, and audit event types as first-class concepts.

Observability is treated as part of the architecture instead of an afterthought. A security gateway must explain its decisions, reveal latency patterns, expose upstream failures, and support investigation during incidents. Metrics and logs are therefore aligned with request identifiers and route identifiers.

- Define the policy in a persistent or external configuration model.
- Expose the decision through audit events and metrics.
- Add positive and negative tests for the behavior.
- Document the expected error response and operational impact.

10.9 Metrics Catalog

Metric	Type	Meaning
sentra_gateway_requests_total	Counter	Total requests by route, method, status, and decision.
sentra_gateway_request_duration_seconds	Timer	End-to-end request latency.

Metric	Type	Meaning
sentra_gateway_upstream_duration_seconds	Timer	Downstream service latency.
sentra_gateway_blocked_requests_total	Counter	Blocked requests by reason.
sentra_gateway_rate_limited_total	Counter	429 responses by policy.
sentra_gateway_invalid_jwt_total	Counter	JWT validation failures.
sentra_gateway_invalid_api_key_total	Counter	API key failures.
sentra_gateway_signature_failures_total	Counter	Request signing failures.
sentra_gateway_circuit_breaker_state	Gauge	Circuit breaker state by route.

Table 12. Metric catalog

Chapter 11: Testing and Evaluation Plan

11.1 Testing Philosophy

Security-sensitive gateways require negative tests as much as happy-path tests. In Sentra Gateway, this topic is important because the gateway is the first component that receives untrusted traffic. A weak design at this layer can expose every internal service, while a strong design can centralize control and reduce repeated security code across the system.

The design decision for this section is to make behavior explicit, measurable, and testable. Every important action should either produce a route decision, an audit event, a metric, or a safe error response. This gives developers and evaluators evidence that the system is behaving according to the documented requirements.

A production-style gateway must balance strict security with usability. If rules are too loose, the system becomes unsafe. If rules are too strict or poorly documented, legitimate clients will fail unexpectedly. Sentra Gateway therefore uses route-level policies, subject-specific limits, and documented response codes.

The module should be implemented through small services rather than one large filter. The filter extracts request data and delegates to services; the services perform validation, policy checks, logging, and decision making. This structure allows unit testing and reduces the chance of hidden coupling.

The design favors explicit policy definitions over hidden behavior. A gateway that silently permits or denies traffic without clear policy ownership becomes difficult to operate. For that reason, Sentra Gateway models routes, permissions, API keys, rate limits, IP rules, and audit event types as first-class concepts.

Observability is treated as part of the architecture instead of an afterthought. A security gateway must explain its decisions, reveal latency patterns, expose upstream failures, and support investigation during incidents. Metrics and logs are therefore aligned with request identifiers and route identifiers.

- Define the policy in a persistent or external configuration model.
- Expose the decision through audit events and metrics.
- Add positive and negative tests for the behavior.
- Document the expected error response and operational impact.

11.2 Unit Testing

Unit tests verify service logic such as token validation, CIDR matching, token bucket math, and signature canonicalization. In Sentra Gateway, this topic is important because the gateway is the first component that receives untrusted traffic. A weak design at this layer can expose every internal service, while a strong design can centralize control and reduce repeated security code across the system.

The design decision for this section is to make behavior explicit, measurable, and testable. Every important action should either produce a route decision, an audit event, a metric, or a safe error response. This gives developers and evaluators evidence that the system is behaving according to the documented requirements.

A production-style gateway must balance strict security with usability. If rules are too loose, the system becomes unsafe. If rules are too strict or poorly documented, legitimate clients will fail unexpectedly. Sentra Gateway therefore uses route-level policies, subject-specific limits, and documented response codes.

The module should be implemented through small services rather than one large filter. The filter extracts request data and delegates to services; the services perform validation, policy checks, logging, and decision making. This structure allows unit testing and reduces the chance of hidden coupling.

The design favors explicit policy definitions over hidden behavior. A gateway that silently permits or denies traffic without clear policy ownership becomes difficult to operate. For that reason, Sentra Gateway models routes, permissions, API keys, rate limits, IP rules, and audit event types as first-class concepts.

Observability is treated as part of the architecture instead of an afterthought. A security gateway must explain its decisions, reveal latency patterns, expose upstream failures, and support investigation during incidents. Metrics and logs are therefore aligned with request identifiers and route identifiers.

- Define the policy in a persistent or external configuration model.
- Expose the decision through audit events and metrics.
- Add positive and negative tests for the behavior.
- Document the expected error response and operational impact.

11.3 Integration Testing

Integration tests verify gateway behavior with PostgreSQL, Redis, and mock downstream services. In Sentra Gateway, this topic is important because the gateway is the first component that receives untrusted traffic. A weak design at this layer can expose every internal service, while a strong design can centralize control and reduce repeated security code across the system.

The design decision for this section is to make behavior explicit, measurable, and testable. Every important action should either produce a route decision, an audit event, a metric, or a safe error response. This gives developers and evaluators evidence that the system is behaving according to the documented requirements.

A production-style gateway must balance strict security with usability. If rules are too loose, the system becomes unsafe. If rules are too strict or poorly documented, legitimate clients will fail unexpectedly. Sentra Gateway therefore uses route-level policies, subject-specific limits, and documented response codes.

The module should be implemented through small services rather than one large filter. The filter extracts request data and delegates to services; the services perform validation, policy checks, logging, and decision making. This structure allows unit testing and reduces the chance of hidden coupling.

The design favors explicit policy definitions over hidden behavior. A gateway that silently permits or denies traffic without clear policy ownership becomes difficult to operate. For that reason, Sentra Gateway models routes, permissions, API keys, rate limits, IP rules, and audit event types as first-class concepts.

Observability is treated as part of the architecture instead of an afterthought. A security gateway must explain its decisions, reveal latency patterns, expose upstream failures, and support investigation during incidents. Metrics and logs are therefore aligned with request identifiers and route identifiers.

- Define the policy in a persistent or external configuration model.
- Expose the decision through audit events and metrics.
- Add positive and negative tests for the behavior.
- Document the expected error response and operational impact.

11.4 Security Regression Testing

Security tests verify invalid tokens, missing scopes, blocked IPs, replayed nonces, spoofed headers, and redacted logs. In Sentra Gateway, this topic is important because the gateway is the first component that receives untrusted traffic. A weak design at this layer can expose every internal service, while a strong design can centralize control and reduce repeated security code across the system.

The design decision for this section is to make behavior explicit, measurable, and testable. Every important action should either produce a route decision, an audit event, a metric, or a safe error response. This gives developers and evaluators evidence that the system is behaving according to the documented requirements.

A production-style gateway must balance strict security with usability. If rules are too loose, the system becomes unsafe. If rules are too strict or poorly documented, legitimate clients will fail unexpectedly. Sentra Gateway therefore uses route-level policies, subject-specific limits, and documented response codes.

The module should be implemented through small services rather than one large filter. The filter extracts request data and delegates to services; the services perform validation, policy checks, logging, and decision making. This structure allows unit testing and reduces the chance of hidden coupling.

The design favors explicit policy definitions over hidden behavior. A gateway that silently permits or denies traffic without clear policy ownership becomes difficult to operate. For that reason, Sentra Gateway models routes, permissions, API keys, rate limits, IP rules, and audit event types as first-class concepts.

Observability is treated as part of the architecture instead of an afterthought. A security gateway must explain its decisions, reveal latency patterns, expose upstream failures, and support investigation during incidents. Metrics and logs are therefore aligned with request identifiers and route identifiers.

- Define the policy in a persistent or external configuration model.
- Expose the decision through audit events and metrics.
- Add positive and negative tests for the behavior.
- Document the expected error response and operational impact.

11.5 Contract Testing

Contract tests verify stable request and response shapes for admin APIs. In Sentra Gateway, this topic is important because the gateway is the first component that receives untrusted traffic. A weak design at this layer can expose every internal service, while a strong design can centralize control and reduce repeated security code across the system.

The design decision for this section is to make behavior explicit, measurable, and testable. Every important action should either produce a route decision, an audit event, a metric, or a safe error response. This gives developers and evaluators evidence that the system is behaving according to the documented requirements.

A production-style gateway must balance strict security with usability. If rules are too loose, the system becomes unsafe. If rules are too strict or poorly documented, legitimate clients will fail unexpectedly. Sentra Gateway therefore uses route-level policies, subject-specific limits, and documented response codes.

The module should be implemented through small services rather than one large filter. The filter extracts request data and delegates to services; the services perform validation, policy checks, logging, and decision making. This structure allows unit testing and reduces the chance of hidden coupling.

The design favors explicit policy definitions over hidden behavior. A gateway that silently permits or denies traffic without clear policy ownership becomes difficult to operate. For that reason, Sentra Gateway models routes, permissions, API keys, rate limits, IP rules, and audit event types as first-class concepts.

Observability is treated as part of the architecture instead of an afterthought. A security gateway must explain its decisions, reveal latency patterns, expose upstream failures, and support investigation during incidents. Metrics and logs are therefore aligned with request identifiers and route identifiers.

- Define the policy in a persistent or external configuration model.
- Expose the decision through audit events and metrics.
- Add positive and negative tests for the behavior.
- Document the expected error response and operational impact.

11.6 Performance Testing with k6

k6 scenarios measure latency, throughput, limits, and behavior under bursts. In Sentra Gateway, this topic is important because the gateway is the first component that receives untrusted traffic. A weak design at this layer can expose every internal service, while a strong design can centralize control and reduce repeated security code across the system.

The design decision for this section is to make behavior explicit, measurable, and testable. Every important action should either produce a route decision, an audit event, a metric, or a safe error response. This gives developers and evaluators evidence that the system is behaving according to the documented requirements.

A production-style gateway must balance strict security with usability. If rules are too loose, the system becomes unsafe. If rules are too strict or poorly documented, legitimate clients will fail unexpectedly. Sentra Gateway therefore uses route-level policies, subject-specific limits, and documented response codes.

The module should be implemented through small services rather than one large filter. The filter extracts request data and delegates to services; the services perform validation, policy checks, logging, and decision making. This structure allows unit testing and reduces the chance of hidden coupling.

The design favors explicit policy definitions over hidden behavior. A gateway that silently permits or denies traffic without clear policy ownership becomes difficult to operate. For that reason, Sentra Gateway models routes, permissions, API keys, rate limits, IP rules, and audit event types as first-class concepts.

Observability is treated as part of the architecture instead of an afterthought. A security gateway must explain its decisions, reveal latency patterns, expose upstream failures, and support investigation during incidents. Metrics and logs are therefore aligned with request identifiers and route identifiers.

- Define the policy in a persistent or external configuration model.
- Expose the decision through audit events and metrics.
- Add positive and negative tests for the behavior.
- Document the expected error response and operational impact.

11.7 Resilience Testing

Failure tests verify timeout, retry, circuit breaker, and fallback behavior. In Sentra Gateway, this topic is important because the gateway is the first component that receives untrusted traffic. A weak design at this layer can expose every internal service, while a strong design can centralize control and reduce repeated security code across the system.

The design decision for this section is to make behavior explicit, measurable, and testable. Every important action should either produce a route decision, an audit event, a metric, or a safe error response. This gives developers and evaluators evidence that the system is behaving according to the documented requirements.

A production-style gateway must balance strict security with usability. If rules are too loose, the system becomes unsafe. If rules are too strict or poorly documented, legitimate clients will fail unexpectedly. Sentra Gateway therefore uses route-level policies, subject-specific limits, and documented response codes.

The module should be implemented through small services rather than one large filter. The filter extracts request data and delegates to services; the services perform validation, policy checks, logging, and decision making. This structure allows unit testing and reduces the chance of hidden coupling.

The design favors explicit policy definitions over hidden behavior. A gateway that silently permits or denies traffic without clear policy ownership becomes difficult to operate. For that reason, Sentra Gateway models routes, permissions, API keys, rate limits, IP rules, and audit event types as first-class concepts.

Observability is treated as part of the architecture instead of an afterthought. A security gateway must explain its decisions, reveal latency patterns, expose upstream failures, and support investigation during incidents. Metrics and logs are therefore aligned with request identifiers and route identifiers.

- Define the policy in a persistent or external configuration model.
- Expose the decision through audit events and metrics.
- Add positive and negative tests for the behavior.
- Document the expected error response and operational impact.

11.8 Evaluation Reporting

Final results should include environment details, commit hash, configuration profile, metrics, screenshots, and limitations. In Sentra Gateway, this topic is important because the gateway is the first component that receives untrusted traffic. A weak design at this layer can expose every internal service, while a strong design can centralize control and reduce repeated security code across the system.

The design decision for this section is to make behavior explicit, measurable, and testable. Every important action should either produce a route decision, an audit event, a metric, or a safe error response. This gives developers and evaluators evidence that the system is behaving according to the documented requirements.

A production-style gateway must balance strict security with usability. If rules are too loose, the system becomes unsafe. If rules are too strict or poorly documented, legitimate clients will fail unexpectedly. Sentra Gateway therefore uses route-level policies, subject-specific limits, and documented response codes.

The module should be implemented through small services rather than one large filter. The filter extracts request data and delegates to services; the services perform validation, policy checks, logging, and decision making. This structure allows unit testing and reduces the chance of hidden coupling.

The design favors explicit policy definitions over hidden behavior. A gateway that silently permits or denies traffic without clear policy ownership becomes difficult to operate. For that reason, Sentra Gateway models routes, permissions, API keys, rate limits, IP rules, and audit event types as first-class concepts.

Observability is treated as part of the architecture instead of an afterthought. A security gateway must explain its decisions, reveal latency patterns, expose upstream failures, and support investigation during incidents. Metrics and logs are therefore aligned with request identifiers and route identifiers.

- Define the policy in a persistent or external configuration model.
- Expose the decision through audit events and metrics.
- Add positive and negative tests for the behavior.
- Document the expected error response and operational impact.

11.9 Security Test Matrix

ID	Scenario	Expected Result
SEC-01	Missing token on protected route.	401 and audit event.
SEC-02	Expired JWT.	401 and JWT_EXPIRED reason.
SEC-03	JWT missing role.	403 and permission denial.
SEC-04	Invalid API key.	401 and API_KEY_INVALID reason.

ID	Scenario	Expected Result
SEC-05	API key missing scope.	403 and scope denial.
SEC-06	Changed body in signed request.	401 and body hash mismatch.
SEC-07	Reused nonce.	401 and replay reason.
SEC-08	Blocked IP.	403 and IP_BLOCKED event.
SEC-09	Spoofed trusted header.	Header stripped and regenerated only after auth.
SEC-10	Rate limit exceeded.	429 and rate-limit metric.

Table 13. Security test matrix

Chapter 12: Deployment and Operations

12.1 Local Deployment

The local environment should run through Docker Compose and be easy to demonstrate. In Sentra Gateway, this topic is important because the gateway is the first component that receives untrusted traffic. A weak design at this layer can expose every internal service, while a strong design can centralize control and reduce repeated security code across the system.

The design decision for this section is to make behavior explicit, measurable, and testable. Every important action should either produce a route decision, an audit event, a metric, or a safe error response. This gives developers and evaluators evidence that the system is behaving according to the documented requirements.

A production-style gateway must balance strict security with usability. If rules are too loose, the system becomes unsafe. If rules are too strict or poorly documented, legitimate clients will fail unexpectedly. Sentra Gateway therefore uses route-level policies, subject-specific limits, and documented response codes.

The module should be implemented through small services rather than one large filter. The filter extracts request data and delegates to services; the services perform validation, policy checks, logging, and decision making. This structure allows unit testing and reduces the chance of hidden coupling.

The design favors explicit policy definitions over hidden behavior. A gateway that silently permits or denies traffic without clear policy ownership becomes difficult to operate. For that reason, Sentra Gateway models routes, permissions, API keys, rate limits, IP rules, and audit event types as first-class concepts.

Observability is treated as part of the architecture instead of an afterthought. A security gateway must explain its decisions, reveal latency patterns, expose upstream failures, and support investigation during incidents. Metrics and logs are therefore aligned with request identifiers and route identifiers.

- Define the policy in a persistent or external configuration model.
- Expose the decision through audit events and metrics.
- Add positive and negative tests for the behavior.
- Document the expected error response and operational impact.

12.2 Production-Like Deployment

Production-like deployment requires TLS, private networks, restricted actuator exposure, managed data services, and externalized secrets. In Sentra Gateway, this topic is important because the gateway is the first component that receives untrusted traffic. A weak design at this layer can expose every internal service, while a strong design can centralize control and reduce repeated security code across the system.

The design decision for this section is to make behavior explicit, measurable, and testable. Every important action should either produce a route decision, an audit event, a metric, or a safe error response. This gives developers and evaluators evidence that the system is behaving according to the documented requirements.

A production-style gateway must balance strict security with usability. If rules are too loose, the system becomes unsafe. If rules are too strict or poorly documented, legitimate clients will fail unexpectedly. Sentra Gateway therefore uses route-level policies, subject-specific limits, and documented response codes.

The module should be implemented through small services rather than one large filter. The filter extracts request data and delegates to services; the services perform validation, policy checks, logging, and decision making. This structure allows unit testing and reduces the chance of hidden coupling.

The design favors explicit policy definitions over hidden behavior. A gateway that silently permits or denies traffic without clear policy ownership becomes difficult to operate. For that reason, Sentra Gateway models routes, permissions, API keys, rate limits, IP rules, and audit event types as first-class concepts.

Observability is treated as part of the architecture instead of an afterthought. A security gateway must explain its decisions, reveal latency patterns, expose upstream failures, and support investigation during incidents. Metrics and logs are therefore aligned with request identifiers and route identifiers.

- Define the policy in a persistent or external configuration model.
- Expose the decision through audit events and metrics.
- Add positive and negative tests for the behavior.
- Document the expected error response and operational impact.

12.3 Environment Variables

Environment variables externalize configuration and keep secrets out of source control. In Sentra Gateway, this topic is important because the gateway is the first component that receives untrusted traffic. A weak design at this layer can expose every internal service, while a strong design can centralize control and reduce repeated security code across the system.

The design decision for this section is to make behavior explicit, measurable, and testable. Every important action should either produce a route decision, an audit event, a metric, or a safe error response. This gives developers and evaluators evidence that the system is behaving according to the documented requirements.

A production-style gateway must balance strict security with usability. If rules are too loose, the system becomes unsafe. If rules are too strict or poorly documented, legitimate clients will fail unexpectedly. Sentra Gateway therefore uses route-level policies, subject-specific limits, and documented response codes.

The module should be implemented through small services rather than one large filter. The filter extracts request data and delegates to services; the services perform validation, policy checks, logging, and decision making. This structure allows unit testing and reduces the chance of hidden coupling.

The design favors explicit policy definitions over hidden behavior. A gateway that silently permits or denies traffic without clear policy ownership becomes difficult to operate. For that reason, Sentra Gateway models routes, permissions, API keys, rate limits, IP rules, and audit event types as first-class concepts.

Observability is treated as part of the architecture instead of an afterthought. A security gateway must explain its decisions, reveal latency patterns, expose upstream failures, and support investigation during incidents. Metrics and logs are therefore aligned with request identifiers and route identifiers.

- Define the policy in a persistent or external configuration model.
- Expose the decision through audit events and metrics.
- Add positive and negative tests for the behavior.
- Document the expected error response and operational impact.

12.4 Operational Runbooks

Runbooks describe how to respond to Redis outage, PostgreSQL outage, key leakage, invalid token spikes, unexpected 429s, and open circuit breakers. In Sentra Gateway, this topic is important because the gateway is the first component that receives untrusted traffic. A weak design at this layer can expose every internal service, while a strong design can centralize control and reduce repeated security code across the system.

The design decision for this section is to make behavior explicit, measurable, and testable. Every important action should either produce a route decision, an audit event, a metric, or a safe error response. This gives developers and evaluators evidence that the system is behaving according to the documented requirements.

A production-style gateway must balance strict security with usability. If rules are too loose, the system becomes unsafe. If rules are too strict or poorly documented, legitimate clients will fail unexpectedly. Sentra Gateway therefore uses route-level policies, subject-specific limits, and documented response codes.

The module should be implemented through small services rather than one large filter. The filter extracts request data and delegates to services; the services perform validation, policy checks, logging, and decision making. This structure allows unit testing and reduces the chance of hidden coupling.

The design favors explicit policy definitions over hidden behavior. A gateway that silently permits or denies traffic without clear policy ownership becomes difficult to operate. For that reason, Sentra Gateway models routes, permissions, API keys, rate limits, IP rules, and audit event types as first-class concepts.

Observability is treated as part of the architecture instead of an afterthought. A security gateway must explain its decisions, reveal latency patterns, expose upstream failures, and support investigation during incidents. Metrics and logs are therefore aligned with request identifiers and route identifiers.

- Define the policy in a persistent or external configuration model.
- Expose the decision through audit events and metrics.
- Add positive and negative tests for the behavior.
- Document the expected error response and operational impact.

12.5 Backup and Recovery

PostgreSQL needs backups and restore tests, while Redis is mostly temporary operational state. In Sentra Gateway, this topic is important because the gateway is the first component that receives untrusted traffic. A weak design at this layer

can expose every internal service, while a strong design can centralize control and reduce repeated security code across the system.

The design decision for this section is to make behavior explicit, measurable, and testable. Every important action should either produce a route decision, an audit event, a metric, or a safe error response. This gives developers and evaluators evidence that the system is behaving according to the documented requirements.

A production-style gateway must balance strict security with usability. If rules are too loose, the system becomes unsafe. If rules are too strict or poorly documented, legitimate clients will fail unexpectedly. Sentra Gateway therefore uses route-level policies, subject-specific limits, and documented response codes.

The module should be implemented through small services rather than one large filter. The filter extracts request data and delegates to services; the services perform validation, policy checks, logging, and decision making. This structure allows unit testing and reduces the chance of hidden coupling.

The design favors explicit policy definitions over hidden behavior. A gateway that silently permits or denies traffic without clear policy ownership becomes difficult to operate. For that reason, Sentra Gateway models routes, permissions, API keys, rate limits, IP rules, and audit event types as first-class concepts.

Observability is treated as part of the architecture instead of an afterthought. A security gateway must explain its decisions, reveal latency patterns, expose upstream failures, and support investigation during incidents. Metrics and logs are therefore aligned with request identifiers and route identifiers.

- Define the policy in a persistent or external configuration model.
- Expose the decision through audit events and metrics.
- Add positive and negative tests for the behavior.
- Document the expected error response and operational impact.

12.6 Release Governance

A safe release pipeline includes tests, security checks, staging, approval, monitoring, and rollback. In Sentra Gateway, this topic is important because the gateway is the first component that receives untrusted traffic. A weak design at this layer can expose every internal service, while a strong design can centralize control and reduce repeated security code across the system.

The design decision for this section is to make behavior explicit, measurable, and testable. Every important action should either produce a route decision, an audit event, a metric, or a safe error response. This gives developers and evaluators evidence that the system is behaving according to the documented requirements.

A production-style gateway must balance strict security with usability. If rules are too loose, the system becomes unsafe. If rules are too strict or poorly documented, legitimate clients will fail unexpectedly. Sentra Gateway therefore uses route-level policies, subject-specific limits, and documented response codes.

The module should be implemented through small services rather than one large filter. The filter extracts request data and delegates to services; the services perform validation, policy checks, logging, and decision making. This structure allows unit testing and reduces the chance of hidden coupling.

The design favors explicit policy definitions over hidden behavior. A gateway that silently permits or denies traffic without clear policy ownership becomes difficult to operate. For that reason, Sentra Gateway models routes, permissions, API keys, rate limits, IP rules, and audit event types as first-class concepts.

Observability is treated as part of the architecture instead of an afterthought. A security gateway must explain its decisions, reveal latency patterns, expose upstream failures, and support investigation during incidents. Metrics and logs are therefore aligned with request identifiers and route identifiers.

- Define the policy in a persistent or external configuration model.
- Expose the decision through audit events and metrics.
- Add positive and negative tests for the behavior.
- Document the expected error response and operational impact.

12.7 Production Hardening

Hardening includes network isolation, mTLS, policy review, log redaction, secret rotation, and alerting. In Sentra Gateway, this topic is important because the gateway is the first component that receives untrusted traffic. A weak design at this layer can expose every internal service, while a strong design can centralize control and reduce repeated security code across the system.

The design decision for this section is to make behavior explicit, measurable, and testable. Every important action should either produce a route decision, an audit event, a metric, or a safe error response. This gives developers and evaluators evidence that the system is behaving according to the documented requirements.

A production-style gateway must balance strict security with usability. If rules are too loose, the system becomes unsafe. If rules are too strict or poorly documented, legitimate clients will fail unexpectedly. Sentra Gateway therefore uses route-level policies, subject-specific limits, and documented response codes.

The module should be implemented through small services rather than one large filter. The filter extracts request data and delegates to services; the services perform validation, policy checks, logging, and decision making. This structure allows unit testing and reduces the chance of hidden coupling.

The design favors explicit policy definitions over hidden behavior. A gateway that silently permits or denies traffic without clear policy ownership becomes difficult to operate. For that reason, Sentra Gateway models routes, permissions, API keys, rate limits, IP rules, and audit event types as first-class concepts.

Observability is treated as part of the architecture instead of an afterthought. A security gateway must explain its decisions, reveal latency patterns, expose upstream failures, and support investigation during incidents. Metrics and logs are therefore aligned with request identifiers and route identifiers.

- Define the policy in a persistent or external configuration model.
- Expose the decision through audit events and metrics.
- Add positive and negative tests for the behavior.
- Document the expected error response and operational impact.

12.8 Environment Variable Catalog

Variable	Purpose
SPRING_PROFILES_ACTIVE	Select local, test, perf, or prod profile.
SERVER_PORT	Gateway HTTP port.
DATABASE_URL	PostgreSQL JDBC URL.
DATABASE_USERNAME	PostgreSQL username.
DATABASE_PASSWORD	PostgreSQL password.
REDIS_HOST	Redis host.
REDIS_PORT	Redis port.
REDIS_PASSWORD	Redis password when configured.
JWT_ISSUER	Expected JWT issuer.
JWT_AUDIENCE	Expected JWT audience.
JWT_JWKS_URI	OIDC JWKS URI.
JWT_PUBLIC_KEY_PATH	Local public key path.
API_KEY_PEPPER	Secret pepper for API key hashing.
SIGNING_SECRET_PEPPER	Secret used for signing derivation.
CORS_ALLOWED_ORIGINS	Allowed browser origins.
MANAGEMENT_ENDPOINTS_WEB_EXPOSURE_INCLUDE	Actuator exposure settings.

Table 14. Environment variable catalog

Chapter 13: Project Management, Roadmap, and Future Work

13.1 Work Breakdown Structure

The project is divided into foundation, routing, identity, API clients, traffic control, request integrity, audit, observability, resilience, testing, and documentation work packages. In Sentra Gateway, this topic is important because the gateway is the first component that receives untrusted traffic. A weak design at this layer can expose every internal service, while a strong design can centralize control and reduce repeated security code across the system.

The design decision for this section is to make behavior explicit, measurable, and testable. Every important action should either produce a route decision, an audit event, a metric, or a safe error response. This gives developers and evaluators evidence that the system is behaving according to the documented requirements.

A production-style gateway must balance strict security with usability. If rules are too loose, the system becomes unsafe. If rules are too strict or poorly documented, legitimate clients will fail unexpectedly. Sentra Gateway therefore uses route-level policies, subject-specific limits, and documented response codes.

The module should be implemented through small services rather than one large filter. The filter extracts request data and delegates to services; the services perform validation, policy checks, logging, and decision making. This structure allows unit testing and reduces the chance of hidden coupling.

The design favors explicit policy definitions over hidden behavior. A gateway that silently permits or denies traffic without clear policy ownership becomes difficult to operate. For that reason, Sentra Gateway models routes, permissions, API keys, rate limits, IP rules, and audit event types as first-class concepts.

Observability is treated as part of the architecture instead of an afterthought. A security gateway must explain its decisions, reveal latency patterns, expose upstream failures, and support investigation during incidents. Metrics and logs are therefore aligned with request identifiers and route identifiers.

- Define the policy in a persistent or external configuration model.
- Expose the decision through audit events and metrics.
- Add positive and negative tests for the behavior.
- Document the expected error response and operational impact.

13.2 Suggested Timeline

A phased timeline reduces risk by delivering a working gateway before advanced features. In Sentra Gateway, this topic is important because the gateway is the first component that receives untrusted traffic. A weak design at this layer can expose every internal service, while a strong design can centralize control and reduce repeated security code across the system.

The design decision for this section is to make behavior explicit, measurable, and testable. Every important action should either produce a route decision, an audit event, a metric, or a safe error response. This gives developers and evaluators evidence that the system is behaving according to the documented requirements.

A production-style gateway must balance strict security with usability. If rules are too loose, the system becomes unsafe. If rules are too strict or poorly documented, legitimate clients will fail unexpectedly. Sentra Gateway therefore uses route-level policies, subject-specific limits, and documented response codes.

The module should be implemented through small services rather than one large filter. The filter extracts request data and delegates to services; the services perform validation, policy checks, logging, and decision making. This structure allows unit testing and reduces the chance of hidden coupling.

The design favors explicit policy definitions over hidden behavior. A gateway that silently permits or denies traffic without clear policy ownership becomes difficult to operate. For that reason, Sentra Gateway models routes, permissions, API keys, rate limits, IP rules, and audit event types as first-class concepts.

Observability is treated as part of the architecture instead of an afterthought. A security gateway must explain its decisions, reveal latency patterns, expose upstream failures, and support investigation during incidents. Metrics and logs are therefore aligned with request identifiers and route identifiers.

- Define the policy in a persistent or external configuration model.
- Expose the decision through audit events and metrics.
- Add positive and negative tests for the behavior.
- Document the expected error response and operational impact.

13.3 Risk Management

The main risks are scope creep, filter complexity, insufficient negative tests, weak metrics, and incorrect request signing canonicalization. In Sentra Gateway, this topic is important because the gateway is the first component that receives untrusted traffic. A weak design at this layer can expose every internal service, while a strong design can centralize control and reduce repeated security code across the system.

The design decision for this section is to make behavior explicit, measurable, and testable. Every important action should either produce a route decision, an audit event, a metric, or a safe error response. This gives developers and evaluators evidence that the system is behaving according to the documented requirements.

A production-style gateway must balance strict security with usability. If rules are too loose, the system becomes unsafe. If rules are too strict or poorly documented, legitimate clients will fail unexpectedly. Sentra Gateway therefore uses route-level policies, subject-specific limits, and documented response codes.

The module should be implemented through small services rather than one large filter. The filter extracts request data and delegates to services; the services perform validation, policy checks, logging, and decision making. This structure allows unit testing and reduces the chance of hidden coupling.

The design favors explicit policy definitions over hidden behavior. A gateway that silently permits or denies traffic without clear policy ownership becomes difficult to operate. For that reason, Sentra Gateway models routes, permissions, API keys, rate limits, IP rules, and audit event types as first-class concepts.

Observability is treated as part of the architecture instead of an afterthought. A security gateway must explain its decisions, reveal latency patterns, expose upstream failures, and support investigation during incidents. Metrics and logs are therefore aligned with request identifiers and route identifiers.

- Define the policy in a persistent or external configuration model.
- Expose the decision through audit events and metrics.
- Add positive and negative tests for the behavior.
- Document the expected error response and operational impact.

13.4 Minimum Viable Demonstration

A strong demonstration includes routing, JWT, API keys, permissions, Redis rate limits, audit logs, metrics, a dashboard, and k6 tests. In Sentra Gateway, this topic is important because the gateway is the first component that receives untrusted traffic. A weak design at this layer can expose every internal service, while a strong design can centralize control and reduce repeated security code across the system.

The design decision for this section is to make behavior explicit, measurable, and testable. Every important action should either produce a route decision, an audit event, a metric, or a safe error response. This gives developers and evaluators evidence that the system is behaving according to the documented requirements.

A production-style gateway must balance strict security with usability. If rules are too loose, the system becomes unsafe. If rules are too strict or poorly documented, legitimate clients will fail unexpectedly. Sentra Gateway therefore uses route-level policies, subject-specific limits, and documented response codes.

The module should be implemented through small services rather than one large filter. The filter extracts request data and delegates to services; the services perform validation, policy checks, logging, and decision making. This structure allows unit testing and reduces the chance of hidden coupling.

The design favors explicit policy definitions over hidden behavior. A gateway that silently permits or denies traffic without clear policy ownership becomes difficult to operate. For that reason, Sentra Gateway models routes, permissions, API keys, rate limits, IP rules, and audit event types as first-class concepts.

Observability is treated as part of the architecture instead of an afterthought. A security gateway must explain its decisions, reveal latency patterns, expose upstream failures, and support investigation during incidents. Metrics and logs are therefore aligned with request identifiers and route identifiers.

- Define the policy in a persistent or external configuration model.
- Expose the decision through audit events and metrics.
- Add positive and negative tests for the behavior.
- Document the expected error response and operational impact.

13.5 Future Work

Future work can add Kubernetes, distributed tracing, policy-as-code, mTLS, adaptive risk scoring, canary routing, WAF-like rules, and an admin dashboard. In Sentra Gateway, this topic is important because the gateway is the first component that receives untrusted traffic. A weak design at this layer can expose every internal service, while a strong design can centralize control and reduce repeated security code across the system.

The design decision for this section is to make behavior explicit, measurable, and testable. Every important action should either produce a route decision, an audit event, a metric, or a safe error response. This gives developers and evaluators evidence that the system is behaving according to the documented requirements.

A production-style gateway must balance strict security with usability. If rules are too loose, the system becomes unsafe. If rules are too strict or poorly documented, legitimate clients will fail unexpectedly. Sentra Gateway therefore uses route-level policies, subject-specific limits, and documented response codes.

The module should be implemented through small services rather than one large filter. The filter extracts request data and delegates to services; the services perform validation, policy checks, logging, and decision making. This structure allows unit testing and reduces the chance of hidden coupling.

The design favors explicit policy definitions over hidden behavior. A gateway that silently permits or denies traffic without clear policy ownership becomes difficult to operate. For that reason, Sentra Gateway models routes, permissions, API keys, rate limits, IP rules, and audit event types as first-class concepts.

Observability is treated as part of the architecture instead of an afterthought. A security gateway must explain its decisions, reveal latency patterns, expose upstream failures, and support investigation during incidents. Metrics and logs are therefore aligned with request identifiers and route identifiers.

- Define the policy in a persistent or external configuration model.
- Expose the decision through audit events and metrics.
- Add positive and negative tests for the behavior.
- Document the expected error response and operational impact.

13.6 Future Work Roadmap

Feature	Description
Kubernetes Deployment	Helm charts, readiness probes, horizontal scaling, and ingress.
Distributed Tracing	OpenTelemetry traces across gateway and services.
Policy-as-Code	Versioned policy bundles and approval workflows.
Advanced OIDC	Multiple providers and tenant-specific identity mapping.
mTLS	Mutual TLS between gateway and internal services.
Adaptive Risk Scoring	Historical baselines and anomaly detection.
Admin Dashboard UI	Frontend for policies, clients, logs, and metrics summaries.
Canary Routing	Route percentages of traffic to new service versions.
WAF-Like Rules	Pattern rules for common injection and scanning behavior.

Table 15. Future work roadmap

Chapter 14: Conclusion

Sentra Gateway is designed as an enterprise-style backend infrastructure project that solves a real problem in microservice-based systems: duplicated and inconsistent edge security. By centralizing authentication, authorization, rate limiting, request signing, IP filtering, audit logging, observability, and resilience policies, the gateway reduces repeated code and improves operational visibility.

The project demonstrates advanced backend engineering skills beyond typical CRUD development. It combines Java, Spring Boot, Spring Cloud Gateway, PostgreSQL, Redis, Resilience4j, Prometheus, Grafana, Docker Compose, OpenAPI, and k6 into one coherent platform. The design emphasizes secure defaults, explainable policies, structured audit evidence, and measurable evaluation.

The project shows that an API gateway can be more than a router. It can act as a security enforcement point, traffic governor, reliability shield, and observability source. When implemented with clear module boundaries and thorough tests, Sentra Gateway can serve as a strong portfolio project and a realistic foundation for understanding production backend infrastructure.

Future work can extend the system with Kubernetes deployment, distributed tracing, policy-as-code, adaptive risk scoring, mTLS, canary routing, and an administrative dashboard. These extensions can turn Sentra Gateway from a portfolio-level backend project into a broader platform engineering project.

Appendix A: Extended Requirements Traceability Matrix

This appendix maps requirements to implementation evidence and tests.

Requirement	Description	Implementation Evidence	Test Evidence
FR-01	Dynamic route registration and controlled request forwarding.	Route entity, route service, gateway route locator, admin route API.	Route matching, path rewrite, disabled route, and upstream forwarding tests.
FR-02	JWT validation for protected user-facing routes.	JWT verifier, JWKS/key-rotation support, authentication filter.	Valid token, expired token, invalid signature, wrong issuer, and missing token tests.
FR-03	API key authentication for machine-to-machine clients.	API client/key model, hashed key storage, key lookup service, key rotation API.	Valid key, revoked key, expired key, missing scope, and rotation tests.
FR-04	Route-level authorization using roles and scopes.	Route permission model, policy evaluator, trusted header propagation.	Role allowed/denied, scope allowed/denied, and mixed JWT/API-key tests.
FR-05	Redis-backed rate limiting by IP, user, API key, tenant, route, and method.	Rate-limit policy table, token-bucket service, Redis counters.	Limit boundary, refill behavior, route override, and distributed-counter tests.
FR-06	Request signing validation with replay protection.	HMAC signing service, timestamp/nonce validator, Redis nonce cache.	Valid signature, tampered body, stale timestamp, and replayed nonce tests.
FR-07	IP allowlist/blocklist enforcement with optional expiry.	IP rule model, CIDR matcher, security policy filter.	Exact IP, CIDR range, expired rule, admin-route allowlist, and block tests.
FR-08	Risk scoring and bot-detection rules.	Risk signal collector, score calculator, policy action resolver.	Missing user-agent, velocity spike, repeated failure, and temporary block tests.
FR-09	Structured audit events for security and routing decisions.	Audit event entity, event writer, async persistence, event taxonomy.	Allowed request, blocked request, rate limit, auth failure, and upstream error tests.
FR-10	Prometheus-ready metrics and Grafana dashboard support.	Micrometer counters/timers, actuator metrics endpoint, dashboard JSON.	Metric presence, label correctness, latency histogram, and dashboard import checks.
FR-11	Resilience policies for upstream services.	Resilience4j timeout, retry, circuit breaker, fallback response mapping.	Timeout, retry success, open circuit, half-open recovery, and fallback tests.
FR-12	Admin APIs for routes, keys, policies, and audit queries.	Admin controllers, validation DTOs, service layer, OpenAPI contracts.	CRUD, validation failure, pagination, sorting, filtering, and authorization tests.
FR-13	OpenAPI documentation for public and admin endpoints.	Springdoc configuration, grouped API docs, response schemas.	Swagger UI availability, schema validation, and contract review checks.
FR-14	Docker Compose local environment for gateway and dependencies.	Compose file, PostgreSQL, Redis, Prometheus, Grafana, mock services.	Startup health, dependency connectivity, profile loading, and smoke tests.
FR-15	Performance evaluation using k6 load profiles.	k6 scripts, seeded policies, route scenarios, summary reports.	Baseline latency, rate-limit stress, burst traffic, and upstream failure scenarios.
FR-16	Operational runbooks and release readiness checklist.	Operations appendix, environment catalog, deployment notes, incident workflow.	Runbook review, configuration validation, rollback checklist, and monitoring checks.

Table 16. Extended requirements traceability matrix

Appendix B: Detailed Endpoint Catalog

This appendix expands the admin API and route catalog.

Method	Path	Required Access	Description
GET	/api/v1/admin/routes	ADMIN	List routes.
POST	/api/v1/admin/routes	ADMIN	Create route.
PATCH	/api/v1/admin/routes/{id}	ADMIN	Update route.
POST	/api/v1/admin/api-clients	ADMIN	Create API client.
POST	/api/v1/admin/api-clients/{id}/keys	ADMIN	Create API key.
POST	/api/v1/admin/api-keys/{id}/rotate	ADMIN	Rotate API key.
GET	/api/v1/admin/audit-events	ADMIN	Search audit events.
GET	/api/users/**	ROUTE POLICY	Forward to user service.
POST	/api/orders/**	ROUTE POLICY	Forward to order service.
POST	/api/payments/**	ROUTE POLICY	Forward to payment service.
POST	/api/notifications/**	ROUTE POLICY	Forward to notification service.

Table 17. Detailed endpoint catalog

Appendix C: SQL Migration Examples

This appendix gives initial migration examples.

```
CREATE TABLE gateway_routes (  
  id UUID PRIMARY KEY,  
  route_id VARCHAR(100) NOT NULL UNIQUE,  
  path_pattern VARCHAR(255) NOT NULL,  
  methods VARCHAR(255) NOT NULL,  
  target_uri VARCHAR(500) NOT NULL,  
  requires_auth BOOLEAN NOT NULL DEFAULT TRUE,  
  requires_signature BOOLEAN NOT NULL DEFAULT FALSE,  
  enabled BOOLEAN NOT NULL DEFAULT TRUE,  
  timeout_ms INTEGER NOT NULL DEFAULT 3000,  
  retry_enabled BOOLEAN NOT NULL DEFAULT FALSE,  
  circuit_breaker_enabled BOOLEAN NOT NULL DEFAULT FALSE,  
  created_at TIMESTAMP NOT NULL,  
  updated_at TIMESTAMP NOT NULL  
);  
  
CREATE TABLE api_clients (  
  id UUID PRIMARY KEY,  
  client_id VARCHAR(120) NOT NULL UNIQUE,  
  name VARCHAR(255) NOT NULL,  
  status VARCHAR(40) NOT NULL,  
  owner_email VARCHAR(255),  
  created_at TIMESTAMP NOT NULL,  
  updated_at TIMESTAMP NOT NULL  
);  
  
CREATE TABLE audit_events (  
  id UUID PRIMARY KEY,  
  request_id VARCHAR(120) NOT NULL,  
  event_type VARCHAR(80) NOT NULL,  
  decision VARCHAR(40) NOT NULL,  
  reason_code VARCHAR(120),  
  subject_type VARCHAR(50),  
  subject_id VARCHAR(255),  
  route_id VARCHAR(120),  
  method VARCHAR(20),  
  path VARCHAR(500),  
  status_code INTEGER,  
  ip_address VARCHAR(80),  
  latency_ms INTEGER,  
  created_at TIMESTAMP NOT NULL  
);
```

Appendix D: Security Checklist

This appendix provides a checklist for implementation and review.

- ☐ JWT signature validation enabled for protected routes.
- ☐ JWT issuer and audience are configured and tested.
- ☐ Expired tokens are rejected.
- ☐ Unknown key identifiers are rejected.
- ☐ API keys are hashed, never stored plaintext.
- ☐ Raw API key is displayed only once.
- ☐ Route roles and scopes are tested.
- ☐ Rate limits exist for anonymous, authenticated, API key, and admin traffic.
- ☐ Request signing canonicalization is documented.
- ☐ Nonce replay protection uses Redis TTL.
- ☐ Blocked IP rules run before expensive authentication.
- ☐ Incoming trusted headers are stripped before gateway adds its own.
- ☐ Authorization headers and API keys are redacted from logs.
- ☐ Audit events are created for deny decisions.
- ☐ Admin APIs require administrator access.
- ☐ Actuator endpoints are restricted in production.
- ☐ Secrets are externalized and not committed.
- ☐ Production deployment uses TLS at ingress.

Appendix E: k6 Performance Test Plan

This appendix defines performance scenarios and expected evidence.

ID	Scenario	Notes
PERF-01	baseline_public	Define VUs, duration, thresholds, expected status mix, and required metrics screenshots.
PERF-02	jwt_user_route	Define VUs, duration, thresholds, expected status mix, and required metrics screenshots.
PERF-03	api_key_route	Define VUs, duration, thresholds, expected status mix, and required metrics screenshots.
PERF-04	rate_limit_burst	Define VUs, duration, thresholds, expected status mix, and required metrics screenshots.
PERF-05	signed_request	Define VUs, duration, thresholds, expected status mix, and required metrics screenshots.
PERF-06	upstream_timeout	Define VUs, duration, thresholds, expected status mix, and required metrics screenshots.
PERF-07	circuit_breaker_open	Define VUs, duration, thresholds, expected status mix, and required metrics screenshots.
PERF-08	mixed_traffic	Define VUs, duration, thresholds, expected status mix, and required metrics screenshots.

Table 18. k6 scenario plan

```
import http from 'k6/http';
import { check, sleep } from 'k6';

export const options = {
  vus: 20,
  duration: '1m',
  thresholds: {
    http_req_failed: ['rate<0.05'],
    http_req_duration: ['p(95)<500']
  }
};

export default function () {
  const res = http.get('http://localhost:8080/api/public/health');
  check(res, { 'status is 200': (r) => r.status === 200 });
  sleep(1);
}
```

Appendix F: README

This appendix provides a GitHub-ready README structure.

```
# Sentra Gateway
```

```
Enterprise API Security and Observability Gateway.
```

```
Sentra Gateway is a production-style Spring Boot API Gateway that centralizes routing, JWT validation, API key management, rate limiting, request signing, IP filtering, audit logging, Prometheus metrics, Grafana dashboards, and Resilience4j reliability controls for microservice-based systems.
```

```
## Key Features
```

- Dynamic route management
- JWT validation with trusted header propagation
- API key lifecycle management and scopes
- Redis-backed rate limiting
- HMAC request signing and nonce replay protection
- IP allow/block rules
- Rule-based bot and risk scoring
- Structured audit events
- Micrometer, Prometheus, and Grafana observability
- Circuit breaker, retry, timeout, and fallback policies
- Docker Compose local environment
- k6 performance testing

Appendix G: Glossary

This appendix defines key project terms.

Term	Definition
Gateway Filter	A component that runs before or after routing to enforce policies.
Route Predicate	A matching condition that determines whether a route applies.
Principal	The authenticated identity represented after validation.
Scope	A permission string used to limit access.
Trusted Header	A header set by the gateway after validation and trusted internally only.
Replay Attack	An attack where a valid captured request is sent again.
Nonce	A unique value used once to prevent replay.
Token Bucket	A rate limiting algorithm that consumes and refills tokens.
Circuit Breaker	A resilience mechanism that stops calls to failing dependencies.
Audit Event	A structured record of a gateway decision.
Observability	Understanding behavior through logs, metrics, traces, and events.

Table 19. Glossary

Appendix H: References and Standards

This appendix lists the key references and standards that inform the engineering design.

1. Spring Boot official documentation.
2. Spring Cloud Gateway official documentation.
3. Spring Security official documentation.
4. PostgreSQL official documentation.
5. Redis official documentation.
6. Resilience4j official documentation.
7. Micrometer official documentation.
8. Prometheus official documentation.
9. Grafana official documentation.
10. k6 official documentation.
11. RFC 7519: JSON Web Token (JWT).
12. RFC 6749: The OAuth 2.0 Authorization Framework.
13. OpenID Connect Core specification.
14. OWASP API Security Top 10.
15. NIST guidance on digital identity and secure systems, where applicable.

Appendix I: Extended Engineering Notes

This appendix adds detailed guidance for project execution, portfolio evaluation, and operational thinking.

I.1 Route Governance

Route Governance is important for Sentra Gateway because the project is designed to look and behave like a real backend infrastructure platform rather than a small demo. The gateway should make security policies explicit, support review, and produce evidence through tests, logs, metrics, and documentation.

A strong implementation should show not only that requests can be forwarded, but also why each request was allowed or denied. This is the difference between a simple proxy and a trustworthy gateway. The documentation should connect every feature to a problem, a design decision, an implementation artifact, and an evaluation method.

Route Governance is important for Sentra Gateway because the project is designed to look and behave like a real backend infrastructure platform rather than a small demo. The gateway should make security policies explicit, support review, and produce evidence through tests, logs, metrics, and documentation.

A strong implementation should show not only that requests can be forwarded, but also why each request was allowed or denied. This is the difference between a simple proxy and a trustworthy gateway. The documentation should connect every feature to a problem, a design decision, an implementation artifact, and an evaluation method.

Route Governance is important for Sentra Gateway because the project is designed to look and behave like a real backend infrastructure platform rather than a small demo. The gateway should make security policies explicit, support review, and produce evidence through tests, logs, metrics, and documentation.

A strong implementation should show not only that requests can be forwarded, but also why each request was allowed or denied. This is the difference between a simple proxy and a trustworthy gateway. The documentation should connect every feature to a problem, a design decision, an implementation artifact, and an evaluation method.

Route Governance is important for Sentra Gateway because the project is designed to look and behave like a real backend infrastructure platform rather than a small demo. The gateway should make security policies explicit, support review, and produce evidence through tests, logs, metrics, and documentation.

A strong implementation should show not only that requests can be forwarded, but also why each request was allowed or denied. This is the difference between a simple proxy and a trustworthy gateway. The documentation should connect every feature to a problem, a design decision, an implementation artifact, and an evaluation method.

I.2 Security Event Taxonomy

Security Event Taxonomy is important for Sentra Gateway because the project is designed to look and behave like a real backend infrastructure platform rather than a small demo. The gateway should make security policies explicit, support review, and produce evidence through tests, logs, metrics, and documentation.

A strong implementation should show not only that requests can be forwarded, but also why each request was allowed or denied. This is the difference between a simple proxy and a trustworthy gateway. The documentation should connect every feature to a problem, a design decision, an implementation artifact, and an evaluation method.

Security Event Taxonomy is important for Sentra Gateway because the project is designed to look and behave like a real backend infrastructure platform rather than a small demo. The gateway should make security policies explicit, support review, and produce evidence through tests, logs, metrics, and documentation.

A strong implementation should show not only that requests can be forwarded, but also why each request was allowed or denied. This is the difference between a simple proxy and a trustworthy gateway. The documentation should connect every feature to a problem, a design decision, an implementation artifact, and an evaluation method.

Security Event Taxonomy is important for Sentra Gateway because the project is designed to look and behave like a real backend infrastructure platform rather than a small demo. The gateway should make security policies explicit, support review, and produce evidence through tests, logs, metrics, and documentation.

A strong implementation should show not only that requests can be forwarded, but also why each request was allowed or denied. This is the difference between a simple proxy and a trustworthy gateway. The documentation should connect every feature to a problem, a design decision, an implementation artifact, and an evaluation method.

Security Event Taxonomy is important for Sentra Gateway because the project is designed to look and behave like a real backend infrastructure platform rather than a small demo. The gateway should make security policies explicit, support review, and produce evidence through tests, logs, metrics, and documentation.

A strong implementation should show not only that requests can be forwarded, but also why each request was allowed or denied. This is the difference between a simple proxy and a trustworthy gateway. The documentation should connect every feature to a problem, a design decision, an implementation artifact, and an evaluation method.

1.3 Safe Defaults

Safe Defaults is important for Sentra Gateway because the project is designed to look and behave like a real backend infrastructure platform rather than a small demo. The gateway should make security policies explicit, support review, and produce evidence through tests, logs, metrics, and documentation.

A strong implementation should show not only that requests can be forwarded, but also why each request was allowed or denied. This is the difference between a simple proxy and a trustworthy gateway. The documentation should connect every feature to a problem, a design decision, an implementation artifact, and an evaluation method.

Safe Defaults is important for Sentra Gateway because the project is designed to look and behave like a real backend infrastructure platform rather than a small demo. The gateway should make security policies explicit, support review, and produce evidence through tests, logs, metrics, and documentation.

A strong implementation should show not only that requests can be forwarded, but also why each request was allowed or denied. This is the difference between a simple proxy and a trustworthy gateway. The documentation should connect every feature to a problem, a design decision, an implementation artifact, and an evaluation method.

Safe Defaults is important for Sentra Gateway because the project is designed to look and behave like a real backend infrastructure platform rather than a small demo. The gateway should make security policies explicit, support review, and produce evidence through tests, logs, metrics, and documentation.

A strong implementation should show not only that requests can be forwarded, but also why each request was allowed or denied. This is the difference between a simple proxy and a trustworthy gateway. The documentation should connect every feature to a problem, a design decision, an implementation artifact, and an evaluation method.

Safe Defaults is important for Sentra Gateway because the project is designed to look and behave like a real backend infrastructure platform rather than a small demo. The gateway should make security policies explicit, support review, and produce evidence through tests, logs, metrics, and documentation.

A strong implementation should show not only that requests can be forwarded, but also why each request was allowed or denied. This is the difference between a simple proxy and a trustworthy gateway. The documentation should connect every feature to a problem, a design decision, an implementation artifact, and an evaluation method.

1.4 Policy Caching

Policy Caching is important for Sentra Gateway because the project is designed to look and behave like a real backend infrastructure platform rather than a small demo. The gateway should make security policies explicit, support review, and produce evidence through tests, logs, metrics, and documentation.

A strong implementation should show not only that requests can be forwarded, but also why each request was allowed or denied. This is the difference between a simple proxy and a trustworthy gateway. The documentation should connect every feature to a problem, a design decision, an implementation artifact, and an evaluation method.

Policy Caching is important for Sentra Gateway because the project is designed to look and behave like a real backend infrastructure platform rather than a small demo. The gateway should make security policies explicit, support review, and produce evidence through tests, logs, metrics, and documentation.

A strong implementation should show not only that requests can be forwarded, but also why each request was allowed or denied. This is the difference between a simple proxy and a trustworthy gateway. The documentation should connect every feature to a problem, a design decision, an implementation artifact, and an evaluation method.

Policy Caching is important for Sentra Gateway because the project is designed to look and behave like a real backend infrastructure platform rather than a small demo. The gateway should make security policies explicit, support review, and produce evidence through tests, logs, metrics, and documentation.

A strong implementation should show not only that requests can be forwarded, but also why each request was allowed or denied. This is the difference between a simple proxy and a trustworthy gateway. The documentation should connect every feature to a problem, a design decision, an implementation artifact, and an evaluation method.

Policy Caching is important for Sentra Gateway because the project is designed to look and behave like a real backend infrastructure platform rather than a small demo. The gateway should make security policies explicit, support review, and produce evidence through tests, logs, metrics, and documentation.

A strong implementation should show not only that requests can be forwarded, but also why each request was allowed or denied. This is the difference between a simple proxy and a trustworthy gateway. The documentation should connect every feature to a problem, a design decision, an implementation artifact, and an evaluation method.

I.5 Downstream Trust Boundary

Downstream Trust Boundary is important for Sentra Gateway because the project is designed to look and behave like a real backend infrastructure platform rather than a small demo. The gateway should make security policies explicit, support review, and produce evidence through tests, logs, metrics, and documentation.

A strong implementation should show not only that requests can be forwarded, but also why each request was allowed or denied. This is the difference between a simple proxy and a trustworthy gateway. The documentation should connect every feature to a problem, a design decision, an implementation artifact, and an evaluation method.

Downstream Trust Boundary is important for Sentra Gateway because the project is designed to look and behave like a real backend infrastructure platform rather than a small demo. The gateway should make security policies explicit, support review, and produce evidence through tests, logs, metrics, and documentation.

A strong implementation should show not only that requests can be forwarded, but also why each request was allowed or denied. This is the difference between a simple proxy and a trustworthy gateway. The documentation should connect every feature to a problem, a design decision, an implementation artifact, and an evaluation method.

Downstream Trust Boundary is important for Sentra Gateway because the project is designed to look and behave like a real backend infrastructure platform rather than a small demo. The gateway should make security policies explicit, support review, and produce evidence through tests, logs, metrics, and documentation.

A strong implementation should show not only that requests can be forwarded, but also why each request was allowed or denied. This is the difference between a simple proxy and a trustworthy gateway. The documentation should connect every feature to a problem, a design decision, an implementation artifact, and an evaluation method.

Downstream Trust Boundary is important for Sentra Gateway because the project is designed to look and behave like a real backend infrastructure platform rather than a small demo. The gateway should make security policies explicit, support review, and produce evidence through tests, logs, metrics, and documentation.

A strong implementation should show not only that requests can be forwarded, but also why each request was allowed or denied. This is the difference between a simple proxy and a trustworthy gateway. The documentation should connect every feature to a problem, a design decision, an implementation artifact, and an evaluation method.

I.6 Performance Boundaries

Performance Boundaries is important for Sentra Gateway because the project is designed to look and behave like a real backend infrastructure platform rather than a small demo. The gateway should make security policies explicit, support review, and produce evidence through tests, logs, metrics, and documentation.

A strong implementation should show not only that requests can be forwarded, but also why each request was allowed or denied. This is the difference between a simple proxy and a trustworthy gateway. The documentation should connect every feature to a problem, a design decision, an implementation artifact, and an evaluation method.

Performance Boundaries is important for Sentra Gateway because the project is designed to look and behave like a real backend infrastructure platform rather than a small demo. The gateway should make security policies explicit, support review, and produce evidence through tests, logs, metrics, and documentation.

A strong implementation should show not only that requests can be forwarded, but also why each request was allowed or denied. This is the difference between a simple proxy and a trustworthy gateway. The documentation should connect every feature to a problem, a design decision, an implementation artifact, and an evaluation method.

Performance Boundaries is important for Sentra Gateway because the project is designed to look and behave like a real backend infrastructure platform rather than a small demo. The gateway should make security policies explicit, support review, and produce evidence through tests, logs, metrics, and documentation.

A strong implementation should show not only that requests can be forwarded, but also why each request was allowed or denied. This is the difference between a simple proxy and a trustworthy gateway. The documentation should connect every feature to a problem, a design decision, an implementation artifact, and an evaluation method.

Performance Boundaries is important for Sentra Gateway because the project is designed to look and behave like a real backend infrastructure platform rather than a small demo. The gateway should make security policies explicit, support review, and produce evidence through tests, logs, metrics, and documentation.

A strong implementation should show not only that requests can be forwarded, but also why each request was allowed or denied. This is the difference between a simple proxy and a trustworthy gateway. The documentation should connect every feature to a problem, a design decision, an implementation artifact, and an evaluation method.

I.7 Incident Review

Incident Review is important for Sentra Gateway because the project is designed to look and behave like a real backend infrastructure platform rather than a small demo. The gateway should make security policies explicit, support review, and produce evidence through tests, logs, metrics, and documentation.

A strong implementation should show not only that requests can be forwarded, but also why each request was allowed or denied. This is the difference between a simple proxy and a trustworthy gateway. The documentation should connect every feature to a problem, a design decision, an implementation artifact, and an evaluation method.

Incident Review is important for Sentra Gateway because the project is designed to look and behave like a real backend infrastructure platform rather than a small demo. The gateway should make security policies explicit, support review, and produce evidence through tests, logs, metrics, and documentation.

A strong implementation should show not only that requests can be forwarded, but also why each request was allowed or denied. This is the difference between a simple proxy and a trustworthy gateway. The documentation should connect every feature to a problem, a design decision, an implementation artifact, and an evaluation method.

Incident Review is important for Sentra Gateway because the project is designed to look and behave like a real backend infrastructure platform rather than a small demo. The gateway should make security policies explicit, support review, and produce evidence through tests, logs, metrics, and documentation.

A strong implementation should show not only that requests can be forwarded, but also why each request was allowed or denied. This is the difference between a simple proxy and a trustworthy gateway. The documentation should connect every feature to a problem, a design decision, an implementation artifact, and an evaluation method.

Incident Review is important for Sentra Gateway because the project is designed to look and behave like a real backend infrastructure platform rather than a small demo. The gateway should make security policies explicit, support review, and produce evidence through tests, logs, metrics, and documentation.

A strong implementation should show not only that requests can be forwarded, but also why each request was allowed or denied. This is the difference between a simple proxy and a trustworthy gateway. The

documentation should connect every feature to a problem, a design decision, an implementation artifact, and an evaluation method.

I.8 Portfolio Evaluation

Portfolio Evaluation is important for Sentra Gateway because the project is designed to look and behave like a real backend infrastructure platform rather than a small demo. The gateway should make security policies explicit, support review, and produce evidence through tests, logs, metrics, and documentation.

A strong implementation should show not only that requests can be forwarded, but also why each request was allowed or denied. This is the difference between a simple proxy and a trustworthy gateway. The documentation should connect every feature to a problem, a design decision, an implementation artifact, and an evaluation method.

Portfolio Evaluation is important for Sentra Gateway because the project is designed to look and behave like a real backend infrastructure platform rather than a small demo. The gateway should make security policies explicit, support review, and produce evidence through tests, logs, metrics, and documentation.

A strong implementation should show not only that requests can be forwarded, but also why each request was allowed or denied. This is the difference between a simple proxy and a trustworthy gateway. The documentation should connect every feature to a problem, a design decision, an implementation artifact, and an evaluation method.

Portfolio Evaluation is important for Sentra Gateway because the project is designed to look and behave like a real backend infrastructure platform rather than a small demo. The gateway should make security policies explicit, support review, and produce evidence through tests, logs, metrics, and documentation.

A strong implementation should show not only that requests can be forwarded, but also why each request was allowed or denied. This is the difference between a simple proxy and a trustworthy gateway. The documentation should connect every feature to a problem, a design decision, an implementation artifact, and an evaluation method.

Portfolio Evaluation is important for Sentra Gateway because the project is designed to look and behave like a real backend infrastructure platform rather than a small demo. The gateway should make security policies explicit, support review, and produce evidence through tests, logs, metrics, and documentation.

A strong implementation should show not only that requests can be forwarded, but also why each request was allowed or denied. This is the difference between a simple proxy and a trustworthy gateway. The documentation should connect every feature to a problem, a design decision, an implementation artifact, and an evaluation method.

I.9 Operational Maturity

Operational Maturity is important for Sentra Gateway because the project is designed to look and behave like a real backend infrastructure platform rather than a small demo. The gateway should make security policies explicit, support review, and produce evidence through tests, logs, metrics, and documentation.

A strong implementation should show not only that requests can be forwarded, but also why each request was allowed or denied. This is the difference between a simple proxy and a trustworthy gateway. The documentation should connect every feature to a problem, a design decision, an implementation artifact, and an evaluation method.

Operational Maturity is important for Sentra Gateway because the project is designed to look and behave like a real backend infrastructure platform rather than a small demo. The gateway should make security policies explicit, support review, and produce evidence through tests, logs, metrics, and documentation.

A strong implementation should show not only that requests can be forwarded, but also why each request was allowed or denied. This is the difference between a simple proxy and a trustworthy gateway. The documentation should connect every feature to a problem, a design decision, an implementation artifact, and an evaluation method.

Operational Maturity is important for Sentra Gateway because the project is designed to look and behave like a real backend infrastructure platform rather than a small demo. The gateway should make security policies explicit, support review, and produce evidence through tests, logs, metrics, and documentation.

A strong implementation should show not only that requests can be forwarded, but also why each request was allowed or denied. This is the difference between a simple proxy and a trustworthy gateway. The

documentation should connect every feature to a problem, a design decision, an implementation artifact, and an evaluation method.

Operational Maturity is important for Sentra Gateway because the project is designed to look and behave like a real backend infrastructure platform rather than a small demo. The gateway should make security policies explicit, support review, and produce evidence through tests, logs, metrics, and documentation.

A strong implementation should show not only that requests can be forwarded, but also why each request was allowed or denied. This is the difference between a simple proxy and a trustworthy gateway. The documentation should connect every feature to a problem, a design decision, an implementation artifact, and an evaluation method.

I.10 Project Portfolio Value

Project Portfolio Value is important for Sentra Gateway because the project is designed to look and behave like a real backend infrastructure platform rather than a small demo. The gateway should make security policies explicit, support review, and produce evidence through tests, logs, metrics, and documentation.

A strong implementation should show not only that requests can be forwarded, but also why each request was allowed or denied. This is the difference between a simple proxy and a trustworthy gateway. The documentation should connect every feature to a problem, a design decision, an implementation artifact, and an evaluation method.

Project Portfolio Value is important for Sentra Gateway because the project is designed to look and behave like a real backend infrastructure platform rather than a small demo. The gateway should make security policies explicit, support review, and produce evidence through tests, logs, metrics, and documentation.

A strong implementation should show not only that requests can be forwarded, but also why each request was allowed or denied. This is the difference between a simple proxy and a trustworthy gateway. The documentation should connect every feature to a problem, a design decision, an implementation artifact, and an evaluation method.

Project Portfolio Value is important for Sentra Gateway because the project is designed to look and behave like a real backend infrastructure platform rather than a small demo. The gateway should make security policies explicit, support review, and produce evidence through tests, logs, metrics, and documentation.

A strong implementation should show not only that requests can be forwarded, but also why each request was allowed or denied. This is the difference between a simple proxy and a trustworthy gateway. The documentation should connect every feature to a problem, a design decision, an implementation artifact, and an evaluation method.

Project Portfolio Value is important for Sentra Gateway because the project is designed to look and behave like a real backend infrastructure platform rather than a small demo. The gateway should make security policies explicit, support review, and produce evidence through tests, logs, metrics, and documentation.

A strong implementation should show not only that requests can be forwarded, but also why each request was allowed or denied. This is the difference between a simple proxy and a trustworthy gateway. The documentation should connect every feature to a problem, a design decision, an implementation artifact, and an evaluation method.

I.11 Security Review Meetings

Security Review Meetings is important for Sentra Gateway because the project is designed to look and behave like a real backend infrastructure platform rather than a small demo. The gateway should make security policies explicit, support review, and produce evidence through tests, logs, metrics, and documentation.

A strong implementation should show not only that requests can be forwarded, but also why each request was allowed or denied. This is the difference between a simple proxy and a trustworthy gateway. The documentation should connect every feature to a problem, a design decision, an implementation artifact, and an evaluation method.

Security Review Meetings is important for Sentra Gateway because the project is designed to look and behave like a real backend infrastructure platform rather than a small demo. The gateway should make security policies explicit, support review, and produce evidence through tests, logs, metrics, and documentation.

A strong implementation should show not only that requests can be forwarded, but also why each request was allowed or denied. This is the difference between a simple proxy and a trustworthy gateway. The

documentation should connect every feature to a problem, a design decision, an implementation artifact, and an evaluation method.

Security Review Meetings is important for Sentra Gateway because the project is designed to look and behave like a real backend infrastructure platform rather than a small demo. The gateway should make security policies explicit, support review, and produce evidence through tests, logs, metrics, and documentation.

A strong implementation should show not only that requests can be forwarded, but also why each request was allowed or denied. This is the difference between a simple proxy and a trustworthy gateway. The documentation should connect every feature to a problem, a design decision, an implementation artifact, and an evaluation method.

Security Review Meetings is important for Sentra Gateway because the project is designed to look and behave like a real backend infrastructure platform rather than a small demo. The gateway should make security policies explicit, support review, and produce evidence through tests, logs, metrics, and documentation.

A strong implementation should show not only that requests can be forwarded, but also why each request was allowed or denied. This is the difference between a simple proxy and a trustworthy gateway. The documentation should connect every feature to a problem, a design decision, an implementation artifact, and an evaluation method.

I.12 Release Discipline

Release Discipline is important for Sentra Gateway because the project is designed to look and behave like a real backend infrastructure platform rather than a small demo. The gateway should make security policies explicit, support review, and produce evidence through tests, logs, metrics, and documentation.

A strong implementation should show not only that requests can be forwarded, but also why each request was allowed or denied. This is the difference between a simple proxy and a trustworthy gateway. The documentation should connect every feature to a problem, a design decision, an implementation artifact, and an evaluation method.

Release Discipline is important for Sentra Gateway because the project is designed to look and behave like a real backend infrastructure platform rather than a small demo. The gateway should make security policies explicit, support review, and produce evidence through tests, logs, metrics, and documentation.

A strong implementation should show not only that requests can be forwarded, but also why each request was allowed or denied. This is the difference between a simple proxy and a trustworthy gateway. The documentation should connect every feature to a problem, a design decision, an implementation artifact, and an evaluation method.

Release Discipline is important for Sentra Gateway because the project is designed to look and behave like a real backend infrastructure platform rather than a small demo. The gateway should make security policies explicit, support review, and produce evidence through tests, logs, metrics, and documentation.

A strong implementation should show not only that requests can be forwarded, but also why each request was allowed or denied. This is the difference between a simple proxy and a trustworthy gateway. The documentation should connect every feature to a problem, a design decision, an implementation artifact, and an evaluation method.

Release Discipline is important for Sentra Gateway because the project is designed to look and behave like a real backend infrastructure platform rather than a small demo. The gateway should make security policies explicit, support review, and produce evidence through tests, logs, metrics, and documentation.

A strong implementation should show not only that requests can be forwarded, but also why each request was allowed or denied. This is the difference between a simple proxy and a trustworthy gateway. The documentation should connect every feature to a problem, a design decision, an implementation artifact, and an evaluation method.

I.13 Documentation Hygiene

Documentation Hygiene is important for Sentra Gateway because the project is designed to look and behave like a real backend infrastructure platform rather than a small demo. The gateway should make security policies explicit, support review, and produce evidence through tests, logs, metrics, and documentation.

A strong implementation should show not only that requests can be forwarded, but also why each request was allowed or denied. This is the difference between a simple proxy and a trustworthy gateway. The

documentation should connect every feature to a problem, a design decision, an implementation artifact, and an evaluation method.

Documentation Hygiene is important for Sentra Gateway because the project is designed to look and behave like a real backend infrastructure platform rather than a small demo. The gateway should make security policies explicit, support review, and produce evidence through tests, logs, metrics, and documentation.

A strong implementation should show not only that requests can be forwarded, but also why each request was allowed or denied. This is the difference between a simple proxy and a trustworthy gateway. The documentation should connect every feature to a problem, a design decision, an implementation artifact, and an evaluation method.

Documentation Hygiene is important for Sentra Gateway because the project is designed to look and behave like a real backend infrastructure platform rather than a small demo. The gateway should make security policies explicit, support review, and produce evidence through tests, logs, metrics, and documentation.

A strong implementation should show not only that requests can be forwarded, but also why each request was allowed or denied. This is the difference between a simple proxy and a trustworthy gateway. The documentation should connect every feature to a problem, a design decision, an implementation artifact, and an evaluation method.

Documentation Hygiene is important for Sentra Gateway because the project is designed to look and behave like a real backend infrastructure platform rather than a small demo. The gateway should make security policies explicit, support review, and produce evidence through tests, logs, metrics, and documentation.

A strong implementation should show not only that requests can be forwarded, but also why each request was allowed or denied. This is the difference between a simple proxy and a trustworthy gateway. The documentation should connect every feature to a problem, a design decision, an implementation artifact, and an evaluation method.

I.14 Dashboard Storytelling

Dashboard Storytelling is important for Sentra Gateway because the project is designed to look and behave like a real backend infrastructure platform rather than a small demo. The gateway should make security policies explicit, support review, and produce evidence through tests, logs, metrics, and documentation.

A strong implementation should show not only that requests can be forwarded, but also why each request was allowed or denied. This is the difference between a simple proxy and a trustworthy gateway. The documentation should connect every feature to a problem, a design decision, an implementation artifact, and an evaluation method.

Dashboard Storytelling is important for Sentra Gateway because the project is designed to look and behave like a real backend infrastructure platform rather than a small demo. The gateway should make security policies explicit, support review, and produce evidence through tests, logs, metrics, and documentation.

A strong implementation should show not only that requests can be forwarded, but also why each request was allowed or denied. This is the difference between a simple proxy and a trustworthy gateway. The documentation should connect every feature to a problem, a design decision, an implementation artifact, and an evaluation method.

Dashboard Storytelling is important for Sentra Gateway because the project is designed to look and behave like a real backend infrastructure platform rather than a small demo. The gateway should make security policies explicit, support review, and produce evidence through tests, logs, metrics, and documentation.

A strong implementation should show not only that requests can be forwarded, but also why each request was allowed or denied. This is the difference between a simple proxy and a trustworthy gateway. The documentation should connect every feature to a problem, a design decision, an implementation artifact, and an evaluation method.

Dashboard Storytelling is important for Sentra Gateway because the project is designed to look and behave like a real backend infrastructure platform rather than a small demo. The gateway should make security policies explicit, support review, and produce evidence through tests, logs, metrics, and documentation.

A strong implementation should show not only that requests can be forwarded, but also why each request was allowed or denied. This is the difference between a simple proxy and a trustworthy gateway. The documentation should connect every feature to a problem, a design decision, an implementation artifact, and an evaluation method.